PENGEMBANGAN *FRAMEWORK* SISTEM PUBLIKASI ILMIAH BERBASIS *BLOCKCHAIN* UNTUK MENGURANGI KETERGANTUNGAN PADA *TRUSTED THIRD PARTY*

Laporan Tugas Akhir

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

MUHAMMAD ROIHAN

NIM : 13522152



PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

SEPTEMBER 2026

PENGEMBANGAN *FRAMEWORK* SISTEM PUBLIKASI ILMIAH BERBASIS *BLOCKCHAIN* UNTUK MENGURANGI KETERGANTUNGAN PADA *TRUSTED THIRD PARTY*

Laporan Tugas Akhir

Oleh

MUHAMMAD ROIHAN

NIM: 13522152

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir

di Bandung, pada tanggal 21 September 2026

Pembimbing



Dr. Phil. Eng. Hari Purnama, S.Si., M.Si.

NIP 118110072

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan laporan tugas akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Pelanggaran Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung khususnya Pasal 14.

Bandung, Senin, 21 September 2026

Muhammad Roihan

NIM 13522152

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya:

Nama/NIM : Muhammad Roihan / 13522152

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Judul : Pengembangan *Framework* Sistem Publikasi Ilmiah Berbasis *Blockchain* untuk Mengurangi Ketergantungan pada *Trusted Third Party*

Menyatakan bahwa saya menggunakan AI dalam pengerjaan maupun penyusunan luaran Tugas Akhir. Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tools**	**Tingkat Penggunaan**
Pemeriksaan Ejaan	Ya	Claude	20%
Pembuatan Teks	Ya	Claude	20%
Bantuan Diskusi dalam Penyusunan Konten	Ya	Claude	20%
Pembuatan Gambar, Video, dan/atau Grafik	Tidak	-	0%
Pencarian Referensi	Ya	Consensus ai	30%

Saya juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada Tugas Akhir telah dinyatakan di dalam surat ini.

Senin, 21 September 2026

Muhammad Roihan

NIM 13522152

ABSTRAK

PENGEMBANGAN *FRAMEWORK* SISTEM PUBLIKASI ILMIAH BERBASIS *BLOCKCHAIN* UNTUK MENGURANGI KETERGANTUNGAN PADA *TRUSTED THIRD PARTY*

Oleh

MUHAMMAD ROIHAN

NIM: 13522152

Sistem publikasi ilmiah konvensional bergantung pada penerbit sebagai pihak ketiga tepercaya (*trusted third party*). Penulis, penelaah, dan pembaca harus mempercayai bahwa penerbit mencatat riwayat naskah, keputusan editorial, dan pembagian insentif secara jujur, tanpa cara untuk memeriksanya secara independen. Penelitian terdahulu yang memanfaatkan *blockchain*, seperti *Eureka*, *PubChain*, dan *Scientific Journal Platform*, baru menyelesaikan sebagian aspek dan belum menyatukannya dalam satu alur kerja dari pengajuan naskah hingga penerbitan.

Tugas akhir ini merancang dan mengimplementasikan *framework* sistem publikasi ilmiah berbasis *blockchain* untuk mengurangi ketergantungan tersebut. Penelitian menggunakan pendekatan *Design Science Research* dengan tahapan penentuan kebutuhan, analisis masalah, perancangan, implementasi, dan evaluasi. Rancangan memakai arsitektur *blockchain* hibrida yang memisahkan data berdasarkan kebutuhannya. Riwayat siklus hidup naskah, keputusan penerimaan berdasarkan suara mayoritas penelaah, dan distribusi insentif dieksekusi oleh *smart contract*, sehingga tidak dapat diubah sepihak dan dapat diperiksa siapa pun. Berkas naskah disimpan pada IPFS dan hanya pengenal kontennya yang dicatat pada *blockchain*. Identitas penulis dan penelaah dijaga dengan alamat samaran, sedangkan penerbitan naskah menghasilkan DOI berupa *token non-fungible* yang tertaut pada berkas naskah. Rancangan ini diimplementasikan menjadi empat *smart contract* pada jaringan Ethereum beserta komponen pendukungnya.

Pengujian pada *smart contract* dan *backend* seluruhnya lulus. Alur publikasi lengkap juga ditelusuri pada sebuah naskah uji di jaringan uji publik Sepolia, mulai dari pengajuan hingga penerbitan. Hampir seluruh kebutuhan fungsional dan non-fungsional terpenuhi. Satu kebutuhan yang baru terpenuhi sebagian adalah desentralisasi penyimpanan, karena berkas masih disimpan melalui satu penyedia layanan IPFS. Meskipun demikian, sistem ini masih berupa implementasi awal karena mekanisme keputusan, insentif, dan pemilihan penelaah dibuat sederhana, serta belum diuji pada beban tinggi maupun audit keamanan.

Kata Kunci: Distributed Ledger Technology, Blockchain, Smart Contract, Publikasi Ilmiah, Trusted Third Party, Peer Review

ABSTRACT

DEVELOPMENT OF A BLOCKCHAIN-BASED SCIENTIFIC PUBLISHING SYSTEM FRAMEWORK TO REDUCE DEPENDENCE ON TRUSTED THIRD PARTIES

By

MUHAMMAD ROIHAN

NIM: 13522152

Conventional scientific publishing systems depend on publishers as trusted third parties. Authors, reviewers, and readers must trust that the publisher records the manuscript history, editorial decisions, and incentive payments honestly, with no means to verify them independently. Previous studies that use blockchain, such as *Eureka*, *PubChain*, and the *Scientific Journal Platform*, have addressed only some of these aspects and have not combined them into a single workflow from manuscript submission to publication.

This final project designs and implements a scientific publishing system framework based on blockchain to reduce this dependence. The research follows the Design Science Research approach, with the stages of requirements definition, problem analysis, design, implementation, and evaluation. The design uses a hybrid blockchain architecture that separates data according to its needs. The manuscript lifecycle history, acceptance decisions based on the majority vote of reviewers, and incentive distribution are executed by smart contracts, so they cannot be changed unilaterally and can be verified by anyone. Manuscript files are stored on IPFS and only their content identifiers are recorded on the blockchain. The identities of authors and reviewers are protected using pseudonymous addresses, while the publication of a manuscript produces a DOI in the form of a non-fungible token linked to the manuscript file. This design is implemented as four smart contracts on the Ethereum network together with supporting components.

Tests on the smart contracts and the backend all passed. The complete publication workflow was also traced on a test manuscript on the Sepolia public test network, from submission to publication. Almost all functional and non-functional requirements were fulfilled. One requirement is only partially fulfilled, namely decentralized storage, because files are still stored through a single IPFS service provider. Nevertheless, the system is still an initial implementation because the decision, incentive, and reviewer selection mechanisms are kept simple, and it has not been tested under heavy load or security audit.

Keywords: Distributed Ledger Technology, Blockchain, Smart Contract, Scientific Publishing, Trusted Third Party, Peer Review

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga tugas akhir dengan judul "Pengembangan *Framework* Sistem Publikasi Ilmiah Berbasis *Blockchain* untuk Mengurangi Ketergantungan pada *Trusted Third Party*" dapat diselesaikan sebagai salah satu syarat untuk memperoleh gelar sarjana. Pencapaian ini tidak lepas dari dukungan, bimbingan, dan bantuan dari berbagai pihak selama proses penyusunan dan pengerjaan tugas akhir. Pada kesempatan ini, penulis menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Phil. Eng. Hari Purnama, S.Si., M.Si., selaku dosen pembimbing atas segala bentuk dukungan yang telah diberikan dalam membimbing penulis;
2. Seluruh dosen tim tugas akhir atas tuntunannya terhadap mahasiswa program studi Teknik Informatika untuk mengerjakan tugas akhir;
3. Seluruh dosen program studi Teknik Informatika ITB yang telah memberikan ilmu pengetahuan yang sangat berharga bagi penulis;
4. Orang tua dan keluarga atas seluruh dukungan yang diberikan;
5. Teman-teman yang mendukung dan memberikan inspirasi baru; dan
6. Seluruh pihak lain yang tidak disebutkan di sini yang telah membantu dalam proses pengerjaan tugas akhir.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah terlibat dalam pengerjaan tugas akhir ini. Penulis juga menyadari bahwa Laporan Tugas Akhir ini masih memiliki berbagai kekurangan, sehingga dengan rendah hati penulis menerima kritik dan saran yang membangun untuk perbaikan di masa yang akan datang. Penulis berharap semoga tugas akhir ini dapat bermanfaat bagi pembaca dan dapat menjadi dasar untuk riset yang lebih baik ke depan.

DAFTAR ISI

LEMBAR PERNYATAAN ..... **iii**
SURAT PERNYATAAN PENGGUNAAN AI ..... **iv**
ABSTRAK ..... **v**
ABSTRACT ..... **vi**
KATA PENGANTAR ..... **vii**
BAB I Pendahuluan **1**
I.1 Latar Belakang 1
I.2 Rumusan Masalah 2
I.3 Tujuan dan Ukuran Keberhasilan Pencapaian 3
I.4 Batasan Masalah 3
I.5 Metodologi 4
BAB II Kajian Pustaka **6**
II.1 Sistem Publikasi Ilmiah 6
II.2 Karakteristik *Open* pada Publikasi Ilmiah 8
II.2.1 *Open Access Publication* dan *Open Peer Review* 8
II.3 *Distributed Ledger Technology* 9
II.3.1 *Blockchain* 9
II.4 *Smart Contract* 10
II.5 *InterPlanetary File System* (IPFS) 11
II.6 *Related Works* 12
II.6.1 *PubChain* 12
II.6.2 *Eureka* 15
II.6.3 *Scientific Journal Platform* 17
BAB III Analisis dan Perancangan **20**
III.1 Analisis Masalah 20
III.1.1 Analisis Masalah pada *Related Works* 20
III.2 Analisis Kebutuhan Sistem 24
III.2.1 Kebutuhan Fungsional 25
III.2.2 Kebutuhan Non-Fungsional 26
III.3 Rancangan Solusi 27

III.4 Perancangan Arsitektur Sistem 30
III.4.1 Arsitektur *Blockchain* Hibrida 30
III.4.2 Komponen Arsitektur Sistem 31
III.4.3 Klasifikasi Data *On-Chain* dan *Off-Chain* 32
III.4.4 Karakteristik *Framework* pada Rancangan 34
III.5 Perancangan Komponen 36
III.5.1 Perancangan Backend 38
III.5.2 Perancangan Smart Contract 41
III.5.3 Perancangan Penyimpanan *Off-Chain* 45
III.5.4 Perancangan Penyimpanan IPFS 46
III.5.5 Perancangan Modul Anonimitas 46
III.5.6 Perancangan Modul Pemilihan *Reviewer* 47
III.5.7 Perancangan Modul Distribusi Insentif 48
III.5.8 Perancangan Modul Konsensus 49
III.6 Alur Kerja Sistem 50
III.6.1 Alur Publikasi Naskah 50
III.6.2 Alur Verifikasi *Reviewer* 54
BAB IV Implementasi **56**
IV.1 Dasar dan Strategi Implementasi 56
IV.1.1 Pelapisan *On-Chain* dan *Off-Chain* 56
IV.1.2 Pemetaan Rancangan ke Unit Implementasi 57
IV.2 Lingkup, Batasan, dan Lingkungan Implementasi 58
IV.2.1 Lingkungan Implementasi 58
IV.2.2 Ruang Lingkup Implementasi 59
IV.2.3 Batasan Implementasi 60
IV.3 Implementasi *Smart Contract* 61
IV.3.1 Implementasi *PublicationRegistry* 61
IV.3.2 Implementasi *ReviewOracle* 66
IV.3.3 Implementasi *JournalToken* 67
IV.3.4 Implementasi *DOIToken* 68
IV.3.5 Konfigurasi Peran 68
IV.4 Implementasi *Backend* 70
IV.4.1 Implementasi *REST API* 70
IV.4.2 Implementasi *EthereumService* 73

IV.4.3 Implementasi Mekanisme Anonimitas (*Session Wallet*) 75
IV.4.4 Implementasi Modul Pemilihan *Reviewer* 76
IV.4.5 Implementasi *PinataService* 77
IV.4.6 Implementasi Manajemen Jurnal 78
IV.5 Implementasi *Oracle* 80
IV.5.1 *Event Listener* 80
IV.5.2 *Plagiarism Handler* 81
IV.5.3 *Reviewer Selector* 82
BAB V Pengujian dan Evaluasi **84**
V.1 Pengujian Implementasi 84
V.1.1 Strategi dan Lingkup Pengujian 84
V.1.2 Ringkasan Hasil Pengujian 86
V.1.3 Pengujian Lapisan *On-Chain* 87
V.1.4 Pengujian Lapisan *Backend* 89
V.1.5 *Traceability* Kebutuhan Fungsional dan Non-Fungsional 90
V.2 Evaluasi 92
V.2.1 Evaluasi Kualitatif 92
V.2.2 Kelebihan dan Keterbatasan 97
BAB VI Kesimpulan dan Saran **99**
VI.1 Kesimpulan 99
VI.2 Saran 100

DAFTAR LAMPIRAN

Lampiran A. Perbandingan Sistem Publikasi Ilmiah 104

Lampiran B. Repositori Implementasi 108

DAFTAR GAMBAR

Gambar II.1. Arsitektur *PubChain* (Wang dkk., 2020) **15**
Gambar II.2. Arsitektur *Eureka* (Niya dkk., 2019) **17**
Gambar II.3. Arsitektur *Scientific Journal Platform* (Beştaş dkk., 2023) **19**

Gambar III.1. Diagram Arsitektur Sistem Publikasi Ilmiah **37**
Gambar III.2. Diagram Komponen Backend **38**
Gambar III.3. Diagram Komponen Smart Contract **42**
Gambar III.4. Diagram Alur Publikasi Naskah **50**
Gambar III.5. Diagram Alur Verifikasi *Reviewer* **54**

DAFTAR TABEL

Tabel II.1. Alur Kerja Sistem Publikasi Ilmiah Konvensional **6**

Tabel III.1. Pemetaan masalah terhadap kebutuhan sistem **24**
Tabel III.2. Kebutuhan fungsional sistem **25**
Tabel III.3. Kebutuhan non-fungsional sistem **26**
Tabel III.4. Pemetaan kebutuhan terhadap rancangan solusi **26**
Tabel III.5. Klasifikasi data *on-chain* dan *off-chain* **33**

Tabel IV.1. Pemetaan unit implementasi ke rancangan dan kebutuhan sistem **57**
Tabel IV.2. Perangkat lunak dan versi yang digunakan pada setiap subsistem **59**
Tabel IV.3. Ringkasan *smart contract* yang diimplementasikan **61**
Tabel IV.4. Konstanta ekonomi pada *PublicationRegistry* **65**
Tabel IV.5. Matriks peran kontrol akses antarkontrak **69**
Tabel IV.6. Daftar *endpoint* REST API beserta penjagaan aksesnya **70**

Tabel V.1. Rekapitulasi eksekusi pengujian **86**
Tabel V.2. Contoh kasus uji dari setiap komponen pengujian **87**
Tabel V.3. Ringkasan cakupan pengujian lapisan *on-chain* **88**
Tabel V.4. *Traceability* kebutuhan fungsional dan non-fungsional **90**
Tabel V.5. Pemenuhan kebutuhan fungsional **92**
Tabel V.6. Pemenuhan kebutuhan non-fungsional **95**

Tabel A.1. Perbandingan tahap *Submission*, *Initial Check*, dan *Editorial Pre-check* **105**
Tabel A.2. Perbandingan tahap *Peer Review*, *Editorial Post-review*, dan *Revision and Resubmission* **106**
Tabel A.3. Perbandingan tahap *Final Acceptance*, *Publication*, dan *Post-Publication* **107**

BAB I

PENDAHULUAN

I.1 Latar Belakang

Sistem publikasi ilmiah tradisional telah menjadi tulang punggung penyebaran pengetahuan sejak kemunculan jurnal akademis pertama, *Journal des Sçavans* dan *Philosophical Transactions*, pada tahun 1665 (Banks, 2018). Sistem ini mengandalkan proses *peer review* yang dikelola penerbit terpusat, namun dinilai bermasalah karena membebankan biaya tinggi, menghilangkan hak cipta penulis, kurang transparan, dan minim insentif bagi *reviewer* (Beştaş dkk., 2023), serta mengalami krisis kredibilitas akibat rendahnya reproduktifitas penelitian (Niya dkk., 2019; Beştaş dkk., 2023).

Open Access Publication (OA) hadir sebagai solusi atas hambatan aksesibilitas dan biaya, didefinisikan sebagai literatur digital yang tersedia daring, bebas biaya, dan terbebas dari sebagian besar batasan hak cipta dan lisensi (Suber, 2012). Dalam ekosistem *Open Science*, konsep ini diperluas melalui *Open Peer Review* (OPR), yaitu istilah payung bagi model ulasan yang membuka identitas penulis dan penelaah serta mempublikasikan laporan ulasan untuk meningkatkan transparansi dan akuntabilitas (Ross-Hellauer, 2017). Namun (Gasparyan dkk., 2019) menegaskan bahwa OA memerlukan pendekatan komprehensif yang turut mencakup transparansi proses ulasan dan kepatuhan terhadap standar etika, bukan sekadar penyediaan akses gratis.

Meskipun OA dan OPR menawarkan perbaikan, akar permasalahan sistem publikasi ilmiah dapat ditelusuri pada hal yang lebih mendasar, yaitu ketergantungan terhadap *trusted third party* atau pihak ketiga tepercaya. *Trusted third party* adalah pihak yang harus dipercaya seluruh pihak lain untuk menyimpan, memvalidasi, dan menjaga kebenaran data suatu sistem. Pada sistem publikasi ilmiah, peran ini dipegang penerbit. Penulis, penelaah, maupun pembaca harus mempercayai bahwa penerbit mencatat dan menampilkan data secara jujur, tanpa cara memeriksa sendiri apakah kepercayaan itu benar-benar dijaga. Persoalannya bukan asumsi bahwa penerbit berniat buruk, melainkan struktur sistem yang tidak menyediakan mekanisme pemeriksaan independen. Apabila suatu catatan diubah, dihapus, atau ditampilkan secara selektif, tidak ada pihak luar yang dapat membuktikan-

nya. *Distributed Ledger Technology* (DLT) menawarkan pendekatan berbeda. Alih-alih menitipkan kepercayaan pada satu pihak, DLT memungkinkan banyak pihak independen menyimpan dan memeriksa salinan data yang sama, sehingga kebenaran suatu catatan ditentukan oleh kesepakatan bersama yang dapat diperiksa siapa pun (Rauchs dkk., 2018). Dengan kata lain, DLT memindahkan letak kepercayaan itu sendiri, dari satu pihak yang mengendalikan seluruh sistem menjadi mekanisme konsensus yang dapat diperiksa secara terbuka oleh setiap pihak yang terlibat.

Untuk menjawab persoalan ini, sejumlah penelitian telah mengusulkan pemanfaatan *blockchain* sebagai fondasi sistem publikasi ilmiah, di antaranya *Eureka* (Niya dkk., 2019), *PubChain* (Wang dkk., 2020), *Scientific Journal Platform* (Beştaş dkk., 2023), serta sistem berbasis IPFS yang diusulkan (Tenorio-Fornés dkk., 2019). Namun implementasinya secara umum masih bersifat parsial dan belum matang. (Tenorio-Fornés dkk., 2019) sendiri mengakui bahwa alternatif terdesentralisasi yang ada masih berada pada tahap awal pengembangan dan belum ada yang benar-benar berfungsi utuh, sementara (Janowicz dkk., 2018), yang merupakan editor jurnal ilmiah yang langsung mengevaluasi berbagai proposal serupa, menyimpulkan bahwa visi yang diusulkan umumnya tidak memberikan detail teknis yang memadai dan belum melibatkan kombinasi aktor publikasi ilmiah, yaitu penulis, editor, penelaah, dan penerbit, yang representatif untuk diuji dalam kondisi nyata.

Berdasarkan uraian di atas, berbagai penelitian yang dikaji telah menyelesaikan aspek-aspek tertentu permasalahan publikasi ilmiah secara terpisah. Tugas akhir ini mengembangkan rancangan tersebut dengan mengintegrasikan aspek-aspek itu ke dalam satu alur kerja *end-to-end* yang utuh, serta memvalidasi kelayakannya melalui pengujian kualitatif.

I.2 Rumusan Masalah

Berdasarkan persoalan *trusted third party* yang diuraikan pada Subbab I.1, sistem publikasi ilmiah konvensional masih menyandarkan kepercayaan pada *trusted third party* dalam berbagai aspek prosesnya, sehingga penulis, penelaah, maupun publik tidak dapat memverifikasinya secara independen, sementara solusi berbasis *blockchain* yang ada masih menyelesaikan aspek-aspek tersebut secara terpisah. Dalam tugas akhir ini, *framework* diartikan sebagai aplikasi yang dapat digunakan ulang dan bersifat semi-lengkap sehingga dapat dispesialisasikan lebih lanjut (Fayad dan Schmidt, 1997). Berdasarkan hal tersebut, rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Kebutuhan apa saja yang harus dipenuhi oleh sistem publikasi ilmiah berbasis *blockchain* agar integritas riwayat, keputusan editorial, dan distribusi insentif dapat diverifikasi secara independen, sementara anonimitas pihak yang terlibat tetap terjaga, tanpa sepenuhnya bergantung pada *trusted third party*?
2. Rancangan sistem publikasi ilmiah berbasis *blockchain* seperti apa yang dapat memenuhi kebutuhan tersebut dengan mengintegrasikan alur *submission*, *peer review*, hingga *publishing* ke dalam satu alur kerja *end-to-end*?

Adapun implementasi *framework* tidak dirumuskan sebagai masalah tersendiri, melainkan menjadi sarana untuk membuktikan bahwa rancangan tersebut dapat direalisasikan secara teknis dan berperilaku sesuai kebutuhan, yang diuji dan dibuktikan pada tahap evaluasi.

I.3 Tujuan dan Ukuran Keberhasilan Pencapaian

Tujuan tugas akhir ini adalah menghasilkan rancangan *framework* sistem publikasi ilmiah berbasis *blockchain* yang mengintegrasikan alur *submission*, *peer review*, hingga *publishing* dalam satu alur kerja *end-to-end*, sehingga integritas riwayat, keputusan editorial, dan distribusi insentif dapat diverifikasi secara independen oleh penulis, penelaah, maupun publik, sementara anonimitas pihak yang terlibat tetap terjaga tanpa seluruh proses bergantung sepenuhnya pada *trusted third party*, beserta implementasi awal sebagai pembuktian kelayakan rancangan. Keberhasilannya diukur melalui pemenuhan kebutuhan fungsional dan non-fungsional.

I.4 Batasan Masalah

Penelitian ini memiliki batasan kajian sebagai berikut.

1. Penelitian dibatasi pada alur editorial publikasi ilmiah sejak *submission* hingga *publishing*. Pengelolaan organisasi penerbit, administrasi keuangan institusi, pengelolaan sumber daya manusia, dan kegiatan operasional jurnal lain di luar alur editorial tidak termasuk dalam ruang lingkup penelitian.
2. Penelitian menggunakan suara mayoritas sederhana sebagai mekanisme keputusan, *token* bernilai tetap sebagai skema insentif, dan pengacakan berdasarkan kecocokan bidang sebagai algoritma pemilihan penelaah. Perancangan dan evaluasi mekanisme konsensus berbobot, model ekonomi *token*, algoritma reputasi, serta pemilihan

penelaah yang dapat diverifikasi secara *on-chain* tidak termasuk dalam ruang lingkup penelitian.

I.5 Metodologi

Penelitian ini berfokus pada perancangan dan pengujian sebuah *framework* baru sebagai artefak solusi, sehingga digunakan pendekatan metodologis yang berorientasi pada pembangunan dan evaluasi artefak. Penelitian menggunakan pendekatan *Design Science Research* (DSR), yaitu paradigma penelitian konstruktif yang bertujuan menghasilkan artefak inovatif sebagai solusi atas permasalahan yang diidentifikasi (vom Brocke dkk., 2020). Artefak yang dikembangkan berupa *framework* sistem publikasi ilmiah *end-to-end* berbasis *blockchain* yang mengintegrasikan integritas riwayat, keputusan editorial, insentif, dan anonimitas ke dalam satu alur kerja.

Metodologi yang digunakan terdiri dari lima tahap utama sebagai berikut.

1. **Pendefinisian spesifikasi dan kebutuhan.** Tahap ini mencakup identifikasi kebutuhan fungsional dan non-fungsional *framework*, termasuk alur kerja yang wajib didukung mulai dari *submission* hingga *publishing*, properti integritas dan verifikasi independen yang harus dipenuhi, serta sifat-sifat umum DLT yang dibutuhkan. Selain itu, ditetapkan batasan lingkup penelitian, pihak-pihak yang dimodelkan (penulis, penelaah, editor, dan pembaca), serta pemisahan konseptual data yang dicatat secara *on-chain* dan *off-chain*.

2. **Analisis persoalan.** Tahap ini mencakup analisis kesenjangan pada sistem publikasi konvensional maupun penelitian terkait berbasis *blockchain*. Selain itu, dilakukan analisis kebutuhan solusi dalam konteks DLT melalui pemetaan data dan peristiwa yang perlu dicatat secara publik pada *ledger on-chain* maupun disimpan secara privat pada penyimpanan *off-chain*.

3. **Perancangan *framework*.** Tahap ini mencakup penyusunan arsitektur sistem yang memisahkan komponen *on-chain* dan *off-chain*, perancangan struktur data dan alur eksekusi pada *smart contract*, serta perancangan alur kerja publikasi dari pengajuan naskah hingga penerbitan yang terintegrasi dengan mekanisme keputusan otomatis, distribusi insentif, dan anonimitas.

4. **Implementasi *framework*.** Tahap ini mewujudkan rancangan dalam bentuk implementasi awal pada lingkungan uji terbatas, untuk membuktikan bahwa rancangan

dapat direalisasikan secara teknis sesuai kebutuhan yang telah dianalisis. Selain itu, disusun skenario alur publikasi representatif yang dijalankan pada *testnet* untuk mendukung pengujian.

5. **Evaluasi.** Tahap ini mencakup pengujian kasus terhadap *framework* yang telah diimplementasikan untuk memverifikasi kebenaran perilakunya, serta evaluasi kualitatif terhadap pemenuhan kebutuhan fungsional dan non-fungsional, termasuk kajian kelebihan dan keterbatasan *framework* yang dihasilkan.

BAB II

KAJIAN PUSTAKA

II.1 Sistem Publikasi Ilmiah

Sistem publikasi ilmiah merupakan mekanisme fundamental dalam penyebaran pengetahuan yang telah berevolusi sejak penerbitan jurnal akademik pertama, *Journal des Sçavans* dan *Philosophical Transactions* pada tahun 1665. (Banks, 2018) menjelaskan bahwa motivasi awal publikasi ilmiah bergeser dari sekadar membangun reputasi pribadi dan berbagi temuan menjadi tuntutan karier yang mendesak di era modern. Secara operasional, (Donovan, 2018) menguraikan bahwa sebuah karya ilmiah melalui serangkaian tahapan ketat mulai dari persiapan, penyerahan (*submission*), peninjauan (*review*), revisi, hingga penerimaan dan publikasi akhir. Di dalam alur ini, proses *peer review* memegang peranan sentral sebagai mekanisme kendali mutu untuk memastikan validitas dan kebaruan sebuah temuan sebelum disebarluaskan kepada komunitas global (Rampelotto, 2023).

Berdasarkan (Rampelotto, 2023), alur kerja sistem publikasi ilmiah konvensional dapat didefinisikan sebagai tabel berikut:

Tabel II.1: Alur Kerja Sistem Publikasi Ilmiah Konvensional

No.	Nama Tahap	Deskripsi
1	***Submission***	Penulis mengajukan naskah melalui sistem *submission* jurnal beserta seluruh materi dan dokumentasi yang diperlukan.
2	***Initial Check***	Staf jurnal melakukan pemeriksaan awal, termasuk verifikasi kepatuhan terhadap kebijakan editorial, standar etika, format penulisan, serta *screening* plagiarisme.

Tabel II.1: Alur Kerja Sistem Publikasi Ilmiah Konvensional

No.	Nama Tahap	Deskripsi
3	***Editorial Pre-check***	Editor akademik mengevaluasi *merit* ilmiah, metodologi, kejelasan penulisan, dan kesesuaian naskah dengan ruang lingkup jurnal untuk menentukan kelanjutan ke tahap *peer review* atau penolakan awal.
4	***Peer Review***	*Reviewer* eksternal menilai kualitas, kekokohan ilmiah, dan signifikansi naskah dalam jangka waktu tertentu serta memberikan rekomendasi.
5	***Editorial Post-review***	Editor mempertimbangkan masukan *reviewer* dan menetapkan keputusan editorial: diterima, revisi minor/mayor, ditolak dengan opsi pengajuan ulang, atau penolakan final.
6	***Revision and Resubmission***	Penulis melakukan revisi naskah berdasarkan komentar *reviewer* dan mengajukan ulang disertai respons *point-by-point*.
7	***Final Acceptance***	Editor mengeluarkan keputusan penerimaan editorial bersifat *provisional* setelah substansi ilmiah dinilai memadai, sambil menunggu pemeriksaan teknis dan format akhir.
8	***Publication***	Artikel dipublikasikan secara resmi dan tersedia bagi pembaca.
9	***Post-Publication***	Laporan *open peer review* (jika tersedia) dipublikasikan bersama artikel.

Meskipun sistem konvensional ini telah mapan, berbagai literatur menyoroti tantangan signifikan yang menghambat efisiensi dan keadilan ekosistem publikasi saat ini. (Suber, 2012) mengidentifikasi adanya "krisis harga" di mana biaya berlangganan jurnal meningkat jauh melampaui inflasi, menciptakan hambatan akses bagi peneliti dan institusi. Selain faktor biaya, masalah transparansi dan insentif juga menjadi sorotan utama. (Rampelotto, 2023) mencatat bahwa kurangnya pengakuan terhadap kontribusi *reviewer* sering kali menyebabkan kelangkaan peninjau yang berkualitas, yang pada gilirannya memperlambat

proses *editorial*. Lebih jauh, model penerbitan terpusat yang ada saat ini dinilai rentan terhadap *bias editorial* dan ketidakefisienan proses yang dapat menghambat *diseminasi* ilmu pengetahuan secara tepat waktu.

II.2 Karakteristik *Open* pada Publikasi Ilmiah

Untuk mengatasi keterbatasan sistem tradisional yang tertutup dan terpusat, diperlukan pergeseran paradigma menuju model yang lebih modern. Penelitian ini mengusulkan karakteristik *Open* (terbuka) sebagai salah satu fondasi utama, yang menjamin aksesibilitas dan transparansi proses publikasi. Dua karakteristik fundamental lainnya yang diusung penelitian ini, yaitu *Autonomous* (otonom) dan *Distributed* (terdistribusi), dicapai melalui pemanfaatan teknologi *Distributed Ledger Technology* dan *smart contract*, sehingga kajian pustakanya dibahas pada Subbab II.3 dan II.4.

II.2.1 *Open Access Publication* dan *Open Peer Review*

Konsep keterbukaan dalam publikasi ilmiah diwujudkan melalui *Open Access* (*OA*).(Suber, 2012) mendefinisikan *OA* sebagai literatur digital yang tersedia secara daring, gratis, dan bebas dari sebagian besar hambatan hak cipta serta lisensi. Ia membedakan dua jenis utama *OA*: *Gratis OA* yang hanya menghilangkan hambatan harga, dan *Libre OA* yang juga menghilangkan hambatan izin penggunaan kembali. Sejalan dengan ini, (Gasparyan dkk., 2019) menekankan pentingnya inisiatif global seperti *Plan S* yang mewajibkan penelitian yang didanai publik untuk segera tersedia dalam akses terbuka, serta peran *platform pengindeksan* dalam meningkatkan visibilitas dan jangkauan karya ilmiah secara global.

Selain akses terhadap artikel, keterbukaan juga diterapkan pada proses evaluasi melalui *Open Peer Review* (*OPR*).(Ross-Hellauer, 2017) dalam tinjauan sistematisnya mengungkapkan bahwa meskipun definisi *OPR* beragam, terdapat tujuh ciri yang membedakannya dari *blind review* tradisional, dengan dua di antaranya yang paling dominan digunakan dalam berbagai definisi: *Open Identities*, di mana penulis dan peninjau saling mengetahui identitas masing-masing untuk meningkatkan akuntabilitas, dan *Open Reports*, di mana laporan peninjauan dipublikasikan berdampingan dengan artikel. Penerapan *OPR* bertujuan untuk meningkatkan kualitas ulasan, mengurangi *bias*, dan memfasilitasi diskusi konstruktif dalam komunitas ilmiah.

II.3 *Distributed Ledger Technology*

Distributed Ledger Technology (*DLT*) merupakan istilah payung yang merujuk pada sistem multi-pihak yang mampu beroperasi tanpa otoritas maupun operator pusat, meskipun sebagian pihak yang terlibat berpotensi tidak dapat dipercaya sepenuhnya, suatu kondisi yang disebut lingkungan adversarial (*adversarial environment*) (Rauchs dkk., 2018). Pada intinya, sebuah sistem *DLT* berfungsi sebagai "mesin konsensus", yakni sistem yang memungkinkan banyak pihak independen mencapai kesepakatan atas satu kumpulan data bersama tanpa bergantung pada koordinator tunggal (Rauchs dkk., 2018). Agar dapat disebut sebagai sistem *DLT*, terdapat lima properti utama yang harus dipenuhi (Rauchs dkk., 2018).

1. **Pencatatan bersama** (*shared recordkeeping*): banyak pihak secara kolektif memelihara satu kumpulan data resmi.
2. **Konsensus multi-pihak** (*multi-party consensus*): seluruh peserta sepakat atas isi catatan tersebut.
3. **Validasi independen** (*independent validation*): setiap peserta dapat memverifikasi sendiri keabsahan data.
4. **Bukti pengubahan** (*tamper evidence*): setiap peserta dapat dengan mudah mendeteksi perubahan yang tidak disepakati bersama pada catatan yang telah dikonfirmasi.
5. **Ketahanan terhadap pengubahan** (*tamper resistance*): satu pihak saja sulit mengubah catatan secara sepihak.

Sejalan dengan kelima properti tersebut, (Romero Ugarte, 2018) menjelaskan *DLT* sebagai basis data yang memiliki banyak salinan identik, didistribusikan ke berbagai partisipan (*node*) dalam jaringan *Peer-to-Peer* (*P2P*), yang disinkronkan melalui mekanisme konsensus kriptografis tanpa memerlukan administrator pusat. Karena sifatnya yang umum ini, *DLT* mencakup berbagai jenis arsitektur pencatatan terdistribusi, dan salah satu bentuk implementasinya yang paling dikenal luas adalah *blockchain* (Rauchs dkk., 2018).

II.3.1 *Blockchain*

Blockchain merupakan implementasi *DLT* yang paling umum digunakan, dengan struktur data khas berupa rangkaian blok yang saling tertaut secara kriptografis melalui fungsi *hash* (Rauchs dkk., 2018; Zheng dkk., 2020). (Nakamoto, 2008) memperkenalkan mekanisme

kerja *blockchain* secara konkret melalui sistem Bitcoin: setiap transaksi yang disiarkan ke jaringan dikumpulkan oleh *node* menjadi satu blok, kemudian *node* tersebut harus menemukan nilai tertentu (*nonce*) yang membuat *hash* blok tersebut memenuhi syarat kesulitan yang ditetapkan (*proof-of-work*) sebelum blok dapat disiarkan dan diterima oleh *node* lain. Setiap blok baru menyertakan *hash* dari blok sebelumnya sehingga membentuk satu rantai yang berurutan. Untuk mengubah isi satu blok lama, seorang penyerang harus mengulang kembali seluruh perhitungan *proof-of-work* blok tersebut beserta seluruh blok sesudahnya, sekaligus mengejar kecepatan seluruh *node* jujur yang terus memperpanjang rantai yang sah (Nakamoto, 2008). Melalui mekanisme inilah rantai terpanjang, yang merepresentasikan upaya komputasi terbesar, disepakati sebagai versi riwayat yang benar oleh mayoritas *node*, sehingga data yang telah tercatat menjadi sangat sulit diubah maupun dipalsukan. Karakteristik utama *blockchain* yang muncul dari mekanisme tersebut meliputi desentralisasi karena tidak ada satu pihak yang mengendalikan seluruh jaringan, kekekalan (*immutability*) karena data yang telah tercatat praktis tidak dapat diubah, persistensi karena setiap *node* menyimpan salinan penuh rantai blok, serta anonimitas karena peserta bertransaksi menggunakan alamat kunci publik alih-alih identitas asli (Zheng dkk., 2020).

II.4 *Smart Contract*

Smart contract adalah program komputer yang menuangkan klausul suatu perjanjian ke dalam kode, pertama kali diusulkan oleh Nick Szabo pada dekade 1990-an sebagai protokol transaksi terkomputerisasi yang mengeksekusi ketentuan perjanjian secara otomatis (Zheng dkk., 2020). Berbeda dengan *DLT* dan *blockchain* yang berperan sebagai infrastruktur pencatatan, *smart contract* dibangun dan dijalankan di atas infrastruktur tersebut: kode kontrak yang telah disetujui disimpan pada *blockchain*, dan begitu suatu kondisi yang telah ditetapkan di dalam kontrak terpenuhi, fungsi terkait akan tereksekusi secara otomatis tanpa memerlukan campur tangan pihak ketiga, dengan hasil eksekusinya turut tercatat sebagai transaksi yang tidak dapat diubah pada *blockchain* (Zheng dkk., 2020). Pengembang dapat menetapkan hak akses pada setiap fungsi kontrak, sehingga hanya pihak yang berwenang yang dapat memicu eksekusinya. Siklus hidup sebuah *smart contract* umumnya terdiri atas empat tahap, yaitu pembuatan (penerjemahan kesepakatan ke dalam kode program), penyebaran (*deployment*) ke jaringan *blockchain*, eksekusi ketika kondisi terpenuhi, dan penyelesaian ketika hasil akhirnya tercatat sebagai transaksi (Zheng dkk., 2020). Karena kode yang telah disebar pada *blockchain* nyaris tidak dapat direvisi kembali, kebenaran

logika kontrak perlu diverifikasi secara saksama sebelum tahap penyebaran. Perlu dicatat pula bahwa *blockchain* secara sengaja dirancang terisolasi dari jaringan luar, sehingga *smart contract* tidak dapat mengetahui kejadian di dunia nyata secara langsung. Untuk itu diperlukan *oracle*, yaitu perantara yang bertugas mencari, memverifikasi, dan meneruskan informasi dari luar jaringan ke dalam *smart contract* (Zheng dkk., 2020).

Sifat otomatis dan tidak dapat diubah inilah yang menjadikan *smart contract* sebagai wahana *otonomi* (*autonomous operation*): begitu kode disebar, ia berjalan persis seperti yang diprogram tanpa campur tangan pihak ketiga (Zheng dkk., 2020). Penerapan konsep ini dalam publikasi ilmiah memungkinkan otomatisasi sebagian besar proses *editorial* yang sebelumnya dilakukan secara manual oleh penerbit. (Beştaş dkk., 2023) mendemonstrasikan bagaimana *Smart Contracts* dapat menangani alur kerja mulai dari penyerahan naskah, penunjukan *reviewer*, hingga keputusan penerbitan. Dengan menghilangkan perantara terpusat, sistem otonom ini tidak hanya mempercepat waktu publikasi tetapi juga mengurangi biaya operasional secara signifikan dan menjamin integritas data melalui mekanisme eksekusi yang transparan dan tidak dapat diubah.

II.5 *InterPlanetary File System* (IPFS)

InterPlanetary File System (*IPFS*) merupakan sistem berkas terdistribusi *peer-to-peer* yang berupaya menghubungkan seluruh perangkat komputasi ke dalam satu sistem berkas yang sama (Benet, 2014). IPFS bukan bagian dari keluarga teknologi *DLT* maupun *blockchain*, melainkan teknologi penyimpanan terdistribusi yang berdiri sendiri, namun kerap dipadukan dengan *blockchain* karena keduanya sama-sama menghindari ketergantungan pada otoritas tunggal.

Setiap objek pada IPFS diberi pengenal unik berupa nilai *hash* kriptografis dari isinya sendiri, yang disebut *multihash*, sehingga alamat suatu objek ditentukan oleh isi kontennya dan bukan oleh lokasi penyimpanannya (*content addressing*) (Benet, 2014). Struktur data inti yang mendasari mekanisme ini adalah *Merkle Directed Acyclic Graph* (*Merkle DAG*), yaitu graf tanpa siklus dengan tautan antarobjek berupa *hash* kriptografis dari objek yang dituju (Benet, 2014). Struktur ini menghasilkan tiga sifat penting sebagai berikut (Benet, 2014).

1. **Pengalamatan konten** (*content addressing*): seluruh konten diidentifikasi secara unik melalui nilai *hash*-nya sendiri.

2. **Ketahanan terhadap pengubahan** (*tamper resistance*): isi objek diverifikasi terhadap *checksum*-nya, sehingga perubahan maupun kerusakan data dapat langsung terdeteksi.

3. **Deduplikasi**: objek dengan isi yang identik dianggap sebagai objek yang sama dan hanya perlu disimpan satu kali.

Objek pada IPFS juga bersifat *immutable*, karena perubahan sekecil apa pun terhadap isinya akan menghasilkan nilai *hash* yang berbeda dan dengan demikian dianggap sebagai objek yang sama sekali baru (Benet, 2014). Karena bersifat *peer-to-peer* tanpa *node* yang diistimewakan, IPFS tidak memiliki titik kegagalan tunggal (*single point of failure*) dan antar-*node* tidak perlu saling percaya (Benet, 2014).

Meskipun demikian, ketersediaan sebuah objek dalam jangka panjang tetap bergantung pada ada tidaknya *node* yang menyimpannya. Untuk itu, IPFS menyediakan mekanisme *pinning*, yaitu tindakan sebuah *node* menahan objek tertentu secara permanen pada penyimpanan lokalnya, termasuk seluruh objek turunan yang ditautkannya, sehingga objek tersebut tidak ikut hilang ketika *node* lain yang sebelumnya turut menyimpannya berhenti beroperasi (Benet, 2014).

II.6 *Related Works*

Penerapan teknologi *blockchain* dalam ekosistem publikasi ilmiah telah dieksplorasi melalui berbagai pendekatan arsitektur dan mekanisme insentif. Tiga penelitian utama yang menjadi rujukan dalam pengembangan sistem pada tugas akhir ini adalah *PubChain* (Wang dkk., 2020), *Eureka* (Niya dkk., 2019), dan *Scientific Journal Platform* (*SJP*) (Beştaş dkk., 2023). Ketiga sistem tersebut dipilih karena masing-masing merepresentasikan pendekatan yang berbeda dalam mendesentralisasi salah satu aspek utama publikasi ilmiah, yaitu penyimpanan dan akses terbuka, mekanisme insentif kontributor, serta otomatisasi alur editorial. Pada subbab berikut, masing-masing sistem dianalisis berdasarkan latar belakang dan tujuan, arsitektur dan komponen utama, alur kerja, kelebihan dan keterbatasan, serta relevansinya terhadap tugas akhir ini.

II.6.1 *PubChain*

Wang dkk. (Wang dkk., 2020) mengembangkan *PubChain*, sebuah platform publikasi akses terbuka terdesentralisasi yang berfokus pada penyelesaian tiga kelemahan utama

platform penerbitan terpusat. Ketiga kelemahan tersebut adalah *pay wall*, yaitu praktik penerbit membebankan biaya baik kepada penulis maupun pembaca, *information island*, yaitu kondisi di mana penulis diwajibkan menyerahkan hak cipta kepada penerbit yang tidak saling berbagi sumber daya literatur, serta disintegrasi proses *peer review* akibat tidak adanya insentif yang memadai bagi penelaah sehingga kualitas ulasan menurun. Tujuan utama *PubChain* adalah membangun ekosistem publikasi di mana penulis, penelaah, dan pembaca sama-sama memperoleh insentif atas kontribusinya melalui *token* berbasis *blockchain*.

Secara arsitektur, *PubChain* dibangun di atas dua komponen utama, yaitu jaringan *blockchain* untuk pencatatan kepemilikan dan insentif, serta jaringan *Inter-Planetary File System* (*IPFS*) sebagai sistem penyimpanan naskah dan komentar yang bersifat *content-addressed* dan terdistribusi, sehingga terhindar dari titik kegagalan tunggal (*single point of failure*). Jaringan *blockchain*-nya dirancang berjalan dalam dua fase secara berurutan, yaitu fase awal sebagai *consortium blockchain* dengan protokol konsensus *Proof-of-Authority* (*PoA*), lalu beralih menjadi *public blockchain* dengan protokol *Proof-of-Work* (*PoW*). Sistem ini menerbitkan *token* bernama *PubCoin* yang dirancang tanpa *Initial Coin Offering* (*ICO*), dan bernilai riil melalui mekanisme *two-way pegging* terhadap rantai induk (*parent chain*). Terdapat tiga entitas dalam ekosistem ini, yaitu pemain publikasi (penulis, penelaah, dan pembaca), penambang (*miner*) yang menjaga *blockchain*, serta *node* penyimpanan IPFS. Untuk mencegah serangan identitas ganda (*Sybil*), *PubChain* menerapkan sistem identitas nyata (*real-name*) di mana setiap alamat harus ditautkan pada surel institusi atau *ORCID*.

Alur kerja *PubChain* meliputi tahapan sebagai berikut:

1. **Pengajuan Naskah (*Submission*):** Penulis mengajukan naskah melalui transaksi *Tpost* yang memuat metadata seperti alamat penulis, alamat IPFS naskah, judul, kata kunci, serta *hash* transaksi dari makalah-makalah yang disitasi. Penulis membayar sejumlah *X PubCoin*, yang sebagian dialokasikan ke *reviewer bonus pool*, sebagian kepada penulis makalah yang disitasi, dan sisanya sebagai biaya transaksi bagi penambang.

2. **Pengajuan Ulasan (*Review*):** Penelaah mengirimkan ulasan melalui transaksi *Treview* yang memuat *hash* transaksi naskah, skor numerik, dan alamat penelaah. Teks komentar disimpan pada IPFS, sedangkan alamat IPFS-nya turut dicatat pada

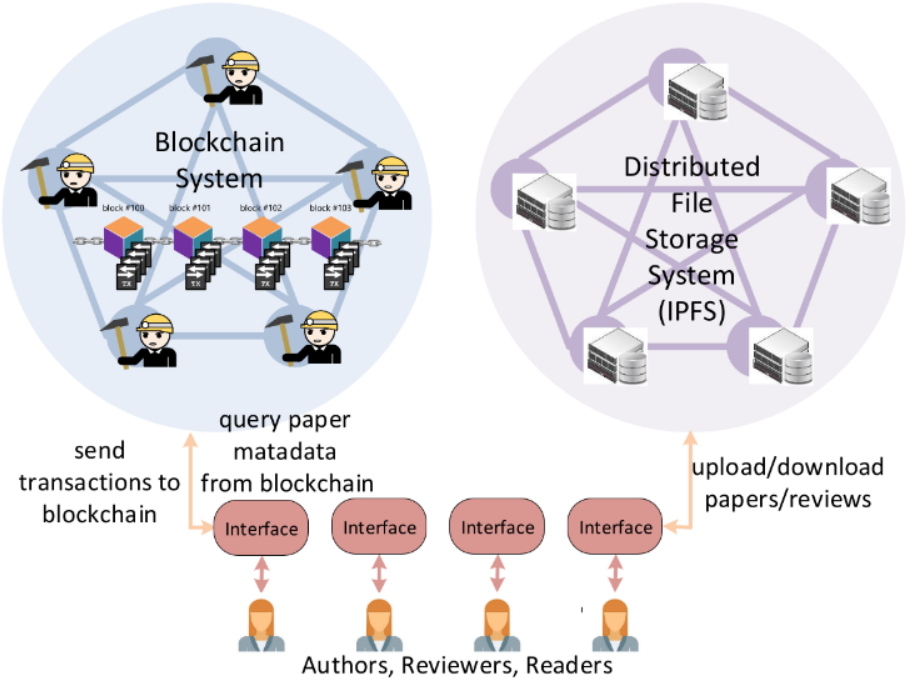
transaksi.

3. **Penilaian oleh Pembaca (*Reader Scoring*):** Pembaca memberikan penilaian terhadap ulasan penelaah. Skor efektif sebuah ulasan dihitung dengan membuang 10% skor tertinggi dan terendah lalu merata-ratakan sisanya, dan hanya berlaku apabila jumlah pembaca yang menilai telah mencukupi.
4. **Distribusi Insentif (*Reward Distribution*):** Penulis memperoleh imbalan berdasarkan skor ulasan makalahnya yang melampaui ambang kualitas tertentu, penelaah memperoleh imbalan dari *bonus pool* sesuai kualitas ulasannya yang dinilai pembaca, dan penulis makalah yang disitasi memperoleh imbalan secara berkelanjutan melalui mekanisme sitasi.

Kelebihan utama *PubChain* adalah kemampuannya menangani tiga permasalahan sekaligus, yaitu *pay wall*, titik kegagalan tunggal, dan lemahnya insentif penelaah, serta melibatkan pembaca sebagai aktor penilai melalui sistem penilaian terdesentralisasi yang terbukti tahan terhadap manipulasi berdasarkan hasil simulasinya. Meski demikian, sistem ini memiliki keterbatasan. Ketergantungan pada identitas nyata (surel institusi atau *ORCID*) menjadikannya tidak kompatibel dengan prinsip *blind review* karena identitas partisipan bersifat terbuka. Selain itu, keandalan sistem penilaiannya sangat bergantung pada besarnya jumlah pembaca, dan nilai ekonomi *PubCoin* bergantung pada keberhasilan mekanisme *pegging* yang belum terbukti pada skala nyata.

Relevansi *PubChain* terhadap tugas akhir ini terletak pada demonstrasinya mengenai integrasi penyimpanan terdesentralisasi berbasis IPFS dengan mekanisme insentif berbasis *token*, yang keduanya juga diadopsi pada sistem yang dirancang penulis. Namun, keputusan *PubChain* untuk menggunakan identitas nyata demi mencegah serangan *Sybil* justru menegaskan adanya ketegangan antara transparansi dan anonimitas, yang menjadi salah satu persoalan yang berusaha dijawab pada tugas akhir ini melalui pendekatan alamat samaran (*pseudo-anonymous*).

Gambaran arsitektur *PubChain* yang memperlihatkan interaksi antara pemain publikasi, jaringan *blockchain* dua fase, dan sistem penyimpanan IPFS diilustrasikan pada Gambar II.1.



Gambar II.1: Arsitektur *PubChain* (Wang dkk., 2020)

II.6.2 *Eureka*

Niya dkk. (Niya dkk., 2019) mengusulkan *Eureka*, sebuah platform publikasi ilmiah berbasis *Ethereum* yang berfokus pada penyelesaian tiga permasalahan sistem publikasi konvensional, yaitu minimnya insentif untuk mempublikasikan hasil penelitian yang gagal, tidak adanya pengakuan atas kontribusi penelaah dan editor, serta krisis reproduktifitas penelitian. Tujuan utama *Eureka* adalah menciptakan model insentif berbasis *token* yang mendistribusikan imbalan secara adil kepada seluruh pihak yang berkontribusi dalam proses editorial.

Secara arsitektur, *Eureka* terdiri atas tiga komponen utama. Pertama, antarmuka aplikasi terdesentralisasi (*DApp*) sebagai titik interaksi pengguna dengan *blockchain*. Kedua, *convenience layer*, yaitu lapisan *off-chain* yang terdiri atas *database* MongoDB, peladen Node.js, dan *node* Ethereum privat yang senantiasa memantau peristiwa (*event*) dari *smart contract*. Lapisan ini menangani notifikasi serta penyimpanan teks ulasan yang tidak efisien apabila disimpan langsung pada *blockchain*. Ketiga, kumpulan *smart contract* yang menghubungkan seluruh sistem ke jaringan Ethereum. *Eureka* menggunakan *token* utilitas *EKA* berstandar ERC-20 yang turut mengimplementasikan antarmuka ERC-865, sehingga pengguna dapat membayar biaya transaksi (*gas*) menggunakan *token EKA* alih-alih Ether.

Alur kerja sistem *Eureka* meliputi tahapan sebagai berikut:

1. **Pengajuan dan Pembayaran (*Submission & Payment*):** Penulis mengajukan naskah melalui *DApp* dan membayar biaya pengajuan menggunakan *token EKA*. Biaya ini dikunci (*locked*) di dalam *smart contract* untuk kemudian didistribusikan kepada para kontributor.

2. **Pemrosesan *Off-Chain* (*Convenience Layer*):** *Convenience layer* memantau peristiwa dari *smart contract*, memberi notifikasi kepada penelaah, serta menyimpan teks ulasan, sementara skor penilaian numerik dicatat pada *blockchain*.

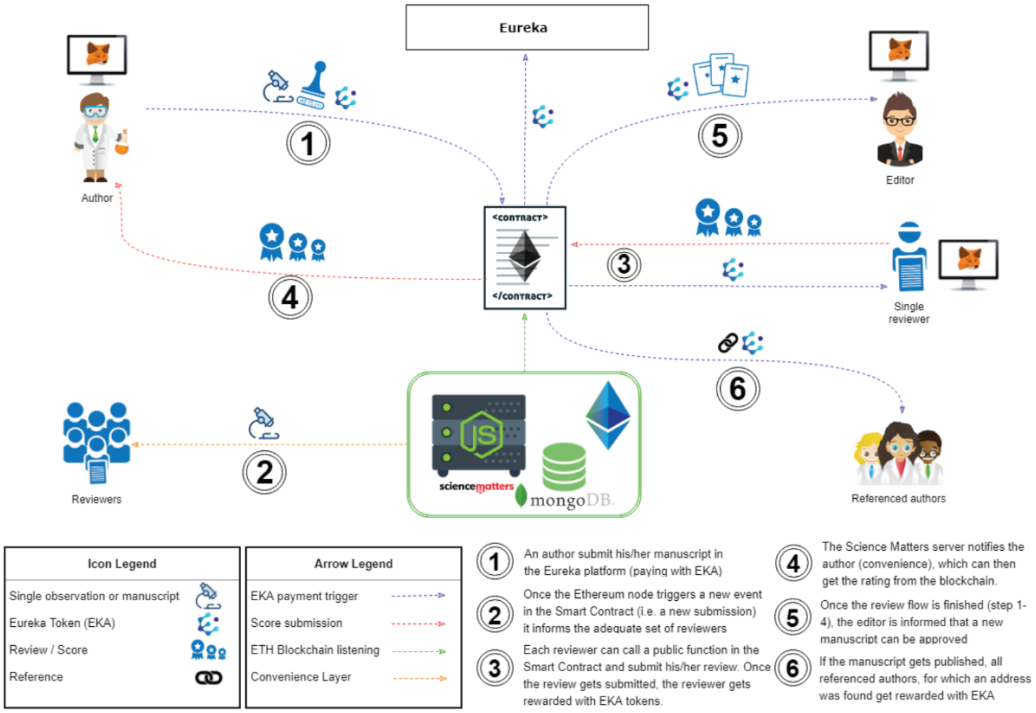
3. **Proses Ulasan (*Review Process*):** Penelaah memberikan penilaian atas naskah. Setelah jumlah ulasan minimum terpenuhi dan editor menyetujui, *smart contract* secara otomatis mendistribusikan *token EKA* yang terkunci sebagai imbalan kepada penelaah dan editor.

4. **Imbalan Sitasi (*Citation Reward*):** Sebagian biaya pengajuan didistribusikan secara otomatis kepada penulis dari makalah-makalah yang disitasi dalam naskah, dengan syarat penulis tersebut memiliki alamat dompet yang telah terdaftar pada sistem.

Kelebihan utama *Eureka* adalah model insentifnya yang komprehensif, yang tidak hanya menghargai penelaah dan editor tetapi juga penulis makalah yang disitasi, serta penerapan mekanisme transaksi tanpa Ether yang menurunkan hambatan penggunaan bagi pengguna awam. Namun, *Eureka* memiliki keterbatasan berupa ketergantungan pada *convenience layer* yang bersifat *off-chain* dan terpusat, sehingga desentralisasinya belum sepenuhnya terwujud. Selain itu, penulisnya sendiri mengakui bahwa menjadikan *smart contract* sebagai satu-satunya otoritas pengatur dalam publikasi ilmiah masih merupakan konsep yang ambisius, dan implementasi penuh pada kondisi nyata masih menjadi bagian dari pekerjaan lanjutan.

Relevansi *Eureka* terhadap tugas akhir ini terletak pada mekanisme distribusi insentif berbasis *token* yang dieksekusi secara otomatis oleh *smart contract*, serta pola pemisahan penyimpanan di mana data ringkas dicatat *on-chain* sementara data berukuran besar ditangani oleh komponen *off-chain*. Kedua pendekatan tersebut sejalan dengan rancangan sistem pada tugas akhir ini yang memisahkan data *on-chain* dan *off-chain* serta mengotomatiskan distribusi insentif kepada penelaah.

Gambaran arsitektur *Eureka* yang memperlihatkan hubungan antara *DApp*, *convenience layer off-chain*, dan *smart contract* pada jaringan Ethereum diilustrasikan pada Gambar II.2.



Gambar II.2: Arsitektur *Eureka* (Niya dkk., 2019)

II.6.3 *Scientific Journal Platform*

Beştaş dkk. (Beştaş dkk., 2023) merancang *Scientific Journal Platform* (*SJP*), sebuah sistem publikasi berbasis *Ethereum Virtual Machine* (*EVM*) yang bertujuan mengatasi berbagai kelemahan sistem publikasi tradisional, seperti biaya publikasi yang tinggi, proses evaluasi yang lambat dan *bias*, kepemilikan hak cipta yang berpindah ke penerbit, serta minimnya imbalan bagi kontributor. Tujuan utamanya adalah mengotomatiskan sebagian besar proses editorial menggunakan *smart contract* guna mempercepat proses, menekan biaya, serta mengurangi subjektivitas dalam evaluasi.

Secara arsitektur, *SJP* dibangun di atas *smart contract* pada jaringan Ethereum dan menggunakan *token SJP* sebagai alat pembayaran serta imbalan. Setiap pengguna memerlukan dompet yang memuat *token SJP* untuk dapat mengajukan naskah. Pemeriksaan plagiarisme pada tahap awal dilakukan dengan bantuan perkakas eksternal iThenticate. *Smart contract* berperan menyediakan daftar kandidat penelaah, menegakkan tenggat waktu ulasan, serta mengelola distribusi imbalan dan penalti.

Alur kerja sistem *SJP* meliputi tahapan sebagai berikut:

1. **Pencocokan Jurnal (*Journal Matching*):** Saat penulis mengunggah naskah, sistem merekomendasikan jurnal yang paling sesuai berdasarkan parameter naskah.

2. **Penyaringan Awal dan Penunjukan Penelaah:** Editor yang bertanggung jawab memeriksa kelayakan naskah dari segi ruang lingkup, format, dan plagiarisme. Apabila lolos, editor memilih penelaah dari daftar kandidat yang disediakan *smart contract* berdasarkan kesesuaian keahlian, dengan penyaringan konflik kepentingan (misalnya menolak penelaah dari institusi yang sama). Sistem juga secara otomatis mengganti penelaah apabila kandidat yang dipilih tidak tersedia dalam kurun waktu tertentu.

3. **Insentif dan Penalti (*Incentive & Penalty*):** Sistem memberikan imbalan lebih besar kepada penelaah yang menyelesaikan tugasnya lebih cepat dari tenggat waktu, dan sebaliknya menerapkan penalti atau biaya keterlambatan apabila melewati batas waktu.

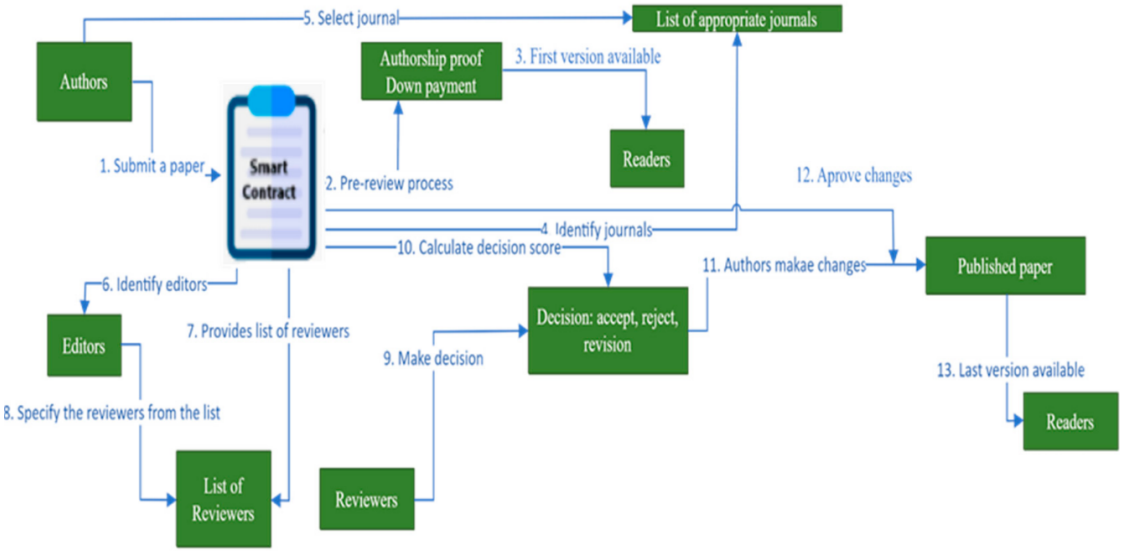
4. **Pemindahan Antar Jurnal (*Cross-Journal Transfer*):** Apabila naskah ditolak oleh satu jurnal dalam ekosistem *SJP*, naskah tersebut dapat dialihkan ke jurnal lain beserta riwayat ulasannya, sehingga proses ulasan tidak perlu diulang dari awal.

Kelebihan utama *SJP* adalah cakupan otomatisasinya yang luas, mulai dari pencocokan jurnal, penyaringan konflik kepentingan, penegakan tenggat waktu, hingga pemindahan naskah antar jurnal, yang secara langsung menekan waktu dan biaya publikasi. Meski demikian, *SJP* masih mempertahankan keterlibatan editor manusia pada tahap penyaringan awal dan pemilihan penelaah, sehingga otomatisasinya belum bersifat penuh. Penulisnya bahkan menyatakan bahwa peniadaan peran editor dan penelaah secara total baru direncanakan pada versi sistem berikutnya. Selain itu, sistem ini masih bergantung pada perkakas plagiarisme eksternal yang terpusat, serta belum menyertakan mekanisme perlindungan anonimitas partisipan.

Relevansi *SJP* terhadap tugas akhir ini sangat kuat pada aspek otomatisasi alur editorial menggunakan *smart contract*, khususnya pada gagasan pencocokan penelaah berdasarkan bidang keahlian dan penyaringan konflik kepentingan. Gagasan ini diadaptasi pada tugas akhir ini melalui mekanisme pencocokan bidang keahlian penelaah yang telah diverifikasi editor. Namun, berbeda dengan *SJP* yang menargetkan peniadaan editor, tugas akhir ini secara sadar mempertahankan peran editor pada tahap penyaringan awal sebagai bentuk kendali mutu yang terukur dan dapat diaudit.

Gambaran arsitektur *SJP* yang memperlihatkan interaksi antara pengguna, *smart contract* pengelola alur editorial, dan perkakas eksternal pada jaringan Ethereum diilustrasikan pada

Gambar II.3.



Gambar II.3: Arsitektur *Scientific Journal Platform* (Beştaş dkk., 2023)

Ketiga sistem di atas menunjukkan bahwa masing-masing telah menyelesaikan sebagian aspek permasalahan publikasi ilmiah secara terpisah, yaitu *PubChain* pada akses terbuka dan insentif berbasis IPFS, *Eureka* pada model insentif kontributor yang komprehensif, serta *SJP* pada otomatisasi alur editorial. Namun, belum ada satu pun di antaranya yang secara utuh memadukan otomatisasi editorial, insentif kontributor, penyimpanan terdesentralisasi, dan jaminan anonimitas dalam satu ekosistem *end-to-end*. Kesenjangan inilah yang menjadi dasar perancangan sistem pada tugas akhir ini, sebagaimana dianalisis lebih lanjut pada Bab III.

BAB III

ANALISIS DAN PERANCANGAN

III.1 Analisis Masalah

Sistem publikasi ilmiah konvensional maupun sebagian besar solusi berbasis *blockchain* yang telah dikaji pada Bab II pada dasarnya telah menyelesaikan sejumlah persoalan permukaan dalam publikasi ilmiah. Namun, terdapat beberapa persoalan yang masih belum terselesaikan. Subbab III.1.1 menganalisis kesenjangan yang tersisa pada penelitian terkait dan sistem publikasi yang ada, sebagai dasar bagi analisis kebutuhan sistem pada Subbab III.2 dan rancangan solusi pada Subbab III.3.

III.1.1 Analisis Masalah pada *Related Works*

Berdasarkan telaah terhadap sistem publikasi konvensional dan penelitian terkait pada Bab II, teridentifikasi lima permasalahan yang saling berkaitan dan secara spesifik menuntut penyelesaian berbasis *blockchain*. Kelima permasalahan tersebut diuraikan sebagai berikut.

III.1.1.1 Integritas Riwayat Publikasi

Dalam sistem publikasi konvensional, seluruh jejak siklus hidup naskah, mulai dari waktu pengajuan, hasil pemeriksaan awal, laporan ulasan, keputusan editorial, riwayat revisi, hingga metadata publikasi akhir, dicatat dan disimpan sepenuhnya pada basis data yang dikendalikan oleh penerbit. Akibatnya, penerbit menjadi satu-satunya pihak yang menentukan kebenaran catatan tersebut, sedangkan penulis, penelaah, maupun publik tidak memiliki cara independen untuk memverifikasi bahwa suatu catatan tidak diubah, dihapus, atau dimanipulasi secara diam-diam setelah dibuat. Persoalan ini penting karena integritas riwayat publikasi merupakan fondasi kepercayaan terhadap rekam ilmiah, sebuah keputusan penerimaan yang direvisi tanpa jejak, laporan ulasan yang disunting kemudian, atau metadata publikasi yang diubah secara sepihak dapat meruntuhkan akuntabilitas keseluruhan proses.

Pendekatan terpusat tidak memadai untuk menjawab persoalan ini bukan karena keterbatasan ketersediaan data, melainkan karena persoalan kepercayaan dimana sekuat apa

pun mekanisme pengelolaan basis data yang diterapkan, operator sistem tetap memegang wewenang penuh untuk mengubah catatan, dan tidak tersedia bukti kriptografis yang memungkinkan pihak luar membuktikan keutuhan sebuah catatan secara mandiri. Penelitian terkait pun belum menutup celah ini secara menyeluruh. *Eureka* (Niya dkk., 2019) hanya mencatatkan skor numerik ulasan pada *blockchain*, sedangkan teks ulasan disimpan pada basis data *off-chain* (MongoDB) di dalam *convenience layer* yang dikelola oleh satu pihak, sehingga sebagian jejak proses evaluasi tetap berada di luar jangkauan verifikasi publik dan bergantung pada satu pihak pengelola. Di sinilah karakteristik *blockchain* sebagai buku besar terdistribusi yang bersifat *append-only* dan tahan-usik (*tamper-resistant*) (Romero Ugarte, 2018) menjadi tepat guna. Setiap peristiwa dalam siklus hidup naskah dapat dicatat sebagai transaksi permanen yang dapat ditelusuri asal-usulnya dan diverifikasi kebenarannya oleh siapa pun tanpa memerlukan kepercayaan pada perantara mana pun.

III.1.1.2 Kepercayaan pada Pengambilan Keputusan

Permasalahan kedua berkaitan dengan cara keputusan penerimaan naskah dihasilkan. Pada sistem konvensional, keputusan ini sepenuhnya berada di tangan editor sebagai penjaga gerbang (*gatekeeper*), sehingga membuka peluang penyalahgunaan wewenang maupun keputusan yang tidak konsisten dan tidak dapat diverifikasi objektivitasnya oleh pihak luar. Persoalan ini penting karena objektivitas keputusan penerimaan merupakan inti legitimasi proses publikasi.

Persoalan serupa juga masih tersisa pada sebagian penelitian terkait. *Eureka* (Niya dkk., 2019) tetap menggantungkan keputusan akhir pada persetujuan editor secara manual, yaitu editor harus menyetujui naskah pada akhir proses ulasan sebelum naskah dapat diterbitkan, sehingga unsur subjektivitas penjaga gerbang belum sepenuhnya hilang. Sementara itu, *SJP* (Beştaş dkk., 2023) telah mengarahkan keputusan pascaulasan pada eksekusi otomatis melalui *smart contract*, namun makalahnya tidak merinci kriteria kuantitatif yang mendasari keputusan tersebut, sehingga objektivitas keputusannya tetap sulit diverifikasi secara independen. Perlu ditegaskan bahwa sekadar mencatatkan keputusan pada basis data biasa tidak menyelesaikan akar persoalannya, sebab tindakan mencatat itu sendiri tetap menuntut kepercayaan pada pihak pencatat.

III.1.1.3 Kepercayaan pada Distribusi Insentif

Permasalahan ketiga berkaitan dengan cara penghargaan atas kontribusi penelaah didistribusikan. Pada sistem konvensional, penghargaan ini bergantung sepenuhnya pada kebijakan sepihak penerbit, sehingga penelaah tidak memiliki jaminan atas imbalan kontribusinya. Persoalan ini penting karena keadilan ekonomi bagi kontributor turut menentukan ketersediaan penelaah berkualitas dalam jangka panjang.

Berbeda dengan persoalan pengambilan keputusan, persoalan ini justru telah dijawab dengan baik oleh penelitian terkait. *Eureka* (Niya dkk., 2019), *PubChain* (Wang dkk., 2020), dan *SJP* (Beştaş dkk., 2023) sama-sama menunjukkan bahwa imbalan dapat didistribusikan secara otomatis melalui *smart contract*, sehingga pembayaran tidak lagi bergantung pada itikad baik pengelola terpusat. Dengan demikian, kelemahan distribusi insentif pada dasarnya merupakan persoalan sistem konvensional, sementara pendekatan berbasis *blockchain* telah terbukti kelayakannya sebagai solusi. Meskipun demikian, persoalan ini tetap dibahas pada tugas akhir ini karena penyelesaiannya belum pernah dipadukan dalam satu sistem yang sama dengan penyelesaian keempat persoalan lain pada subbab ini, sebagaimana diuraikan pada Subbab III.1.1.5. Oleh karena itu, rancangan pada tugas akhir ini tidak mengusulkan mekanisme baru untuk persoalan ini, melainkan mengadopsi pola yang telah terbukti tersebut sebagai bagian dari satu ekosistem publikasi yang terpadu.

III.1.1.4 Ketegangan antara Transparansi dan Anonimitas

Permasalahan keempat justru muncul sebagai konsekuensi dari upaya memperoleh transparansi yang tahan-usik melalui *blockchain*. Karakteristik keterbukaan pada *Open Peer Review* (Ross-Hellauer, 2017) menuntut agar jejak proses evaluasi dapat diaudit secara publik. Namun transparansi penuh tanpa perlindungan identitas berpotensi memicu bias sosial, misalnya penelaah yang segan mengkritik penulis senior. Persoalan ini bersifat khas *blockchain* karena tidak seperti basis data terpusat yang dapat dengan mudah menyembunyikan identitas di balik lapisan aplikasi, *blockchain* pada dasarnya bersifat transparan sehingga setiap aksi harus dikaitkan dengan suatu alamat. (Janowicz dkk., 2018) menegaskan bahwa penerapan anonimitas penelaah pada *blockchain* tak-terenkripsi seperti Ethereum bukan perkara sederhana, sebab penelaah tetap harus dikaitkan dengan alamat publik untuk keperluan penugasan dan distribusi imbalan. Dengan demikian, diperlukan mekanisme yang dirancang khusus agar anonimitas dapat terjamin tanpa mengorbankan akuntabilitas maupun ketahanan terhadap serangan identitas ganda. Di antara solusi yang

ada, *PubChain* (Wang dkk., 2020) bahkan secara sengaja mengorbankan anonimitas dengan mewajibkan identitas nyata demi mencegah serangan identitas ganda, sedangkan *Eureka* dan *SJP* tidak menyediakan mekanisme anonimitas sama sekali. Dengan demikian, menjaga transparansi yang dapat diaudit sekaligus melindungi anonimitas peran secara bersamaan tetap menjadi persoalan terbuka yang memerlukan pendekatan asli *blockchain*.

III.1.1.5 Fragmentasi Solusi yang Ada

Analisis atas keempat permasalahan di atas menunjukkan bahwa penelitian terkait cenderung menyelesaikan aspek publikasi ilmiah secara terpisah-pisah. *PubChain* unggul pada akses terbuka dan penyimpanan terdistribusi berbasis IPFS, *Eureka* pada kelengkapan model insentif kontributor, dan *SJP* pada otomatisasi alur editorial. Namun, belum ada satu pun yang memadukan integritas riwayat publikasi, minimalisasi kepercayaan pada eksekusi keputusan dan insentif, serta jaminan anonimitas ke dalam satu ekosistem *end-to-end* yang utuh. Fragmentasi inilah yang menjadi kesenjangan utama, dan menuntut adanya rancangan sistem yang menyatukan seluruh nilai tambah *blockchain* tersebut secara terintegrasi. Perbandingan penanganan setiap tahap alur kerja publikasi ilmiah oleh sistem konvensional, ketiga sistem tersebut, dan solusi yang diajukan pada tugas akhir ini dirinci pada Lampiran A.

Ketiga sistem tersebut, yaitu *PubChain*, *Eureka*, dan *SJP*, dengan demikian menjadi (*baseline*) bagi perancangan pada tugas akhir ini, bukan karena salah satunya diadopsi secara utuh, melainkan karena masing-masing merepresentasikan pendekatan solusi yang paling maju pada aspeknya masing-masing di antara penelitian sejenis yang telah dikaji pada Bab II. Kesenjangan yang teridentifikasi pada subbab ini, yaitu belum terpadunya integritas riwayat, minimalisasi kepercayaan pada keputusan dan insentif, serta anonimitas ke dalam satu alur kerja, menjadi acuan langsung bagi kebutuhan sistem pada Subbab III.2 dan rancangan solusi pada Subbab III.3, karena setiap mekanisme yang diusulkan di sana secara eksplisit ditelusuri kembali ke salah satu *baseline* di atas, baik sebagai kelanjutan dari pendekatan yang telah terbukti (misalnya penyimpanan berbasis IPFS pada *PubChain* atau insentif berbasis *token* pada *Eureka* dan *SJP*), maupun sebagai jawaban atas kekurangan yang belum ditutup oleh ketiganya (misalnya anonimitas yang tidak disediakan *Eureka* maupun *SJP*, dan dikorbankan secara sengaja oleh *PubChain*).

III.2 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dilakukan untuk menerjemahkan permasalahan yang diidentifikasi pada Subbab III.1 ke dalam spesifikasi teknis untuk sistem publikasi ilmiah yang akan dibangun, sebagai dasar bagi rancangan solusi pada Subbab III.3. Kebutuhan ini dibagi menjadi dua kategori utama, yaitu kebutuhan fungsional dan kebutuhan non-fungsional. Agar keterkaitan antara persoalan yang diidentifikasi pada Subbab III.1.1 dengan kebutuhan yang dirumuskan tetap dapat ditelusuri, Tabel III.1 memetakan setiap masalah ke kode kebutuhan yang menjawabnya.

Tabel III.1: Pemetaan masalah terhadap kebutuhan sistem

Masalah	**Kebutuhan Terkait**	**Keterkaitan**
Integritas Riwayat Publikasi	FR-01, FR-04, FR-07, FR-10, NFR-01, NFR-02, NFR-03	Pencatatan siklus hidup naskah, ulasan, dan berkas secara permanen dan dapat diaudit tanpa bergantung pada satu penerbit.
Kepercayaan pada Pengambilan Keputusan	FR-02, FR-03, FR-05, FR-09, NFR-04	Keputusan editorial dan penunjukan penelaah dijalankan berdasarkan aturan otomatis, bukan kebijaksanaan sepihak editor.
Kepercayaan pada Distribusi Insentif	FR-06	Insentif penelaah dijamin oleh eksekusi kode, bukan itikad baik pengelola.
Ketegangan Transparansi dan Anonimitas	FR-08	Identitas penulis dan penelaah terlindungi tanpa mengorbankan keterlacakan proses.
Fragmentasi Solusi yang Ada	Seluruh kebutuhan FR-01 s.d. FR-10 dan NFR-01 s.d. NFR-04	Dijawab melalui integrasi seluruh kebutuhan di atas ke dalam satu sistem, bukan oleh satu kebutuhan spesifik.

III.2.1 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan fitur-fitur dan fungsionalitas yang harus tersedia pada sistem publikasi. Daftar kebutuhan fungsional sistem diberikan pada Tabel III.2.

Tabel III.2: Kebutuhan fungsional sistem

Kode	Kebutuhan fungsional
FR-01	Sistem dapat menerima pengajuan naskah dari penulis dan melakukan pencatatan ke dalam *blockchain*.
FR-02	Sistem dapat melakukan pemeriksaan awal secara otomatis terhadap naskah yang diajukan, dilanjutkan dengan penyaringan kelayakan oleh editor sebelum naskah dilanjutkan ke tahap *peer review*.
FR-03	Sistem dapat melakukan penunjukan penelaah secara otomatis.
FR-04	Sistem dapat memfasilitasi proses penyampaian laporan ulasan dari penelaah dan mencatat hasil ulasan secara *immutable* dan *transparent* pada *blockchain*.
FR-05	Sistem dapat mengambil keputusan penerimaan, penolakan, atau permintaan revisi terhadap naskah secara otonom.
FR-06	Sistem dapat mengeksekusi distribusi insentif berupa *reward token* secara otomatis kepada penelaah yang berkontribusi pada saat naskah diterbitkan.
FR-07	Sistem dapat menyimpan berkas asli naskah pada jaringan penyimpanan terdistribusi IPFS dan menautkan referensinya ke dalam *blockchain*.
FR-08	Sistem menyediakan fitur anonimitas di mana informasi profil penulis dan penelaah saling disembunyikan satu sama lain selama proses ulasan berlangsung. Identitas penulis dapat dibuka kembali setelah keputusan akhir ulasan ditetapkan sebagai bentuk transparansi pascapublikasi, sedangkan identitas penelaah tetap disembunyikan secara permanen, tidak terikat status naskah.
FR-09	Sistem dapat memfasilitasi proses verifikasi bidang keahlian penelaah oleh editor sebelum penelaah tersebut dapat ditugaskan pada naskah yang sesuai dengan bidang keahliannya.
FR-10	Sistem menyediakan fitur bagi pembaca untuk memberikan komentar terhadap naskah yang telah dipublikasikan.

III.2.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mendefinisikan batasan, aspek keamanan, kriteria kualitas, serta metrik kinerja dari sistem. Daftar kebutuhan non-fungsional sistem diberikan pada Tabel III.3.

Tabel III.3: Kebutuhan non-fungsional sistem

Kode	Kebutuhan non-fungsional
NFR-01	**Transparansi.** Seluruh keputusan dan hasil tahapan alur kerja (hasil penunjukan penelaah, ulasan, distribusi pendanaan) harus tersimpan sebagai *on-chain data* yang dapat dilacak dan diaudit secara publik, kecuali informasi identitas yang memang sengaja disamarkan demi menjaga anonimitas sebagaimana disebutkan pada FR-08.
NFR-02	**Kekekalan.** Data penting sistem, khususnya riwayat ulasan, nilai *hash* dokumen, dan log transaksi tidak dapat dihapus, diubah, maupun dimodifikasi secara sepihak oleh aktor manapun di dalam jaringan.
NFR-03	**Desentralisasi.** Penyimpanan fisik dokumen naskah tidak boleh bergantung pada peladen tunggal (*single point of failure*).
NFR-04	**Kontrol Akses.** Setiap aksi penting dalam sistem hanya dapat dilakukan oleh aktor yang memiliki hak akses atau peran yang sesuai.

Setelah kebutuhan fungsional dan non-fungsional dirumuskan, Tabel III.4 memetakan setiap kebutuhan ke rancangan solusi yang memenuhinya, sebagai jembatan menuju Subbab III.3.

Tabel III.4: Pemetaan kebutuhan terhadap rancangan solusi

Rancangan Solusi	Kebutuhan yang Dipenuhi	Masalah Terkait
Pencatatan siklus hidup naskah sebagai rekam *on-chain*	FR-01, FR-04, NFR-01, NFR-02	Integritas Riwayat Publikasi
Penyimpanan berkas berbasis *content-addressing*	FR-07, NFR-03	Integritas Riwayat Publikasi

Tabel III.4: Pemetaan kebutuhan terhadap rancangan solusi

Rancangan Solusi	**Kebutuhan yang Dipenuhi**	**Masalah Terkait**
Validasi keputusan penerimaan secara otonom	FR-02, FR-03, FR-05, FR-09, NFR-04	Kepercayaan pada Pengambilan Keputusan
Distribusi insentif melalui *smart contract*	FR-06	Kepercayaan pada Distribusi Insentif
DOI sebagai *token non-fungible* tertaut CID	FR-01, FR-07, FR-10	Integritas Riwayat Publikasi
Mekanisme anonimitas berbasis alamat samaran	FR-08	Ketegangan Transparansi dan Anonimitas

III.3 Rancangan Solusi

Untuk menjawab kebutuhan yang telah dirumuskan pada Subbab III.2, sekaligus permasalahan yang mendasarinya pada Subbab III.1.1, dirancang sistem publikasi ilmiah yang menambahkan sejumlah mekanisme dan komponen, yang masing-masing secara langsung bertumpu pada nilai tambah *blockchain*. Setiap rancangan berikut diuraikan berdasarkan fungsinya, keterkaitannya dengan permasalahan yang telah diidentifikasi, serta justifikasi teknis dan keterkaitannya dengan manfaat *blockchain*.

1. **Pencatatan siklus hidup naskah sebagai rekam *on-chain* yang tak dapat diubah.** Sistem mencatatkan setiap peristiwa penting dalam siklus hidup naskah, yaitu pengajuan, hasil pemeriksaan awal, keputusan penyaringan editor, setiap putusan ulasan, revisi, keputusan akhir, hingga penerbitan, sebagai transaksi dan *event* permanen pada *blockchain*. Mekanisme ini secara langsung menjawab permasalahan integritas dan keterverifikasian riwayat publikasi karena sifat *blockchain* yang *append-only* dan tahan-usik, keseluruhan jejak proses membentuk satu rekam yang saling tertaut, dapat ditelusuri asal-usulnya, dan dapat diverifikasi secara independen oleh siapa pun tanpa bergantung pada penerbit. Justifikasinya bertumpu pada karakteristik kekekalan (*immutability*) dan keterlacakan (*traceability*) yang menjadi inti nilai tambah teknologi *blockchain*.

2. **Penyimpanan berkas berbasis *content-addressing* dengan referensi *on-chain*.** Berkas berukuran besar seperti naskah, ulasan, dan dokumen keputusan editor disimpan pada jaringan penyimpanan terdistribusi IPFS, sedangkan pengenal kontennya

(*Content Identifier*, CID) dicatatkan pada *blockchain*. Fungsi utama mekanisme ini bukan sekadar mendistribusikan penyimpanan, melainkan menjamin integritas berkas karena CID merupakan sidik jari kriptografis dari isi berkas, setiap perubahan sekecil apa pun terhadap berkas akan menghasilkan CID yang berbeda, sehingga upaya manipulasi berkas dapat langsung terdeteksi ketika dibandingkan dengan CID yang tercatat *on-chain* (Benet, 2014). Pendekatan pemaduan penyimpanan terdistribusi dengan *blockchain* ini telah terbukti pada *PubChain* (Wang dkk., 2020) dan diadaptasi pada tugas akhir ini sebagai penopang keterverifikasian artefak ilmiah.

3. **Validasi keputusan penerimaan secara otonom melalui *smart contract*.** Alih-alih sekadar mencatatkan keputusan yang dihasilkan di luar rantai, sistem menanamkan logika pengambilan keputusan langsung ke dalam *smart contract*, di mana status *ACCEPTED* hanya dapat dieksekusi apabila kriteria kuantitatif terpenuhi. Mekanisme ini menjawab permasalahan minimalisasi kepercayaan pada eksekusi keputusan: keputusan tidak hanya tercatat, tetapi benar-benar dihasilkan dan diverifikasi oleh logika *on-chain* yang deterministik dan bersifat *self-enforcing* (Zheng dkk., 2017). Dengan demikian, tidak ada satu aktor pun, termasuk editor, yang dapat memaksakan keputusan publikasi secara sepihak, sehingga objektivitas keputusan dapat diverifikasi secara publik. Pendekatan ini memperkuat gagasan pencatatan keputusan *on-chain* pada *SJP* (Beştaş dkk., 2023) dengan memindahkan logika keputusan itu sendiri ke dalam *smart contract*.

4. **Distribusi insentif melalui *smart contract*.** Sistem mendistribusikan insentif kepada penelaah secara otomatis melalui *smart contract* pada saat naskah resmi diterbitkan, dalam satu transaksi yang sama dengan pencetakan pengenal publikasi. Mekanisme ini menjawab persoalan keadilan ekonomi bagi kontributor karena distribusi dijamin oleh kode yang dieksekusi secara atomik, penelaah tidak perlu mempercayai itikad baik pihak pengelola untuk memperoleh haknya. Justifikasi ini melanjutkan gagasan insentif berbasis *token* pada *Eureka* (Niya dkk., 2019) dan *PubChain* (Wang dkk., 2020), dengan penekanan pada sifat eksekusi yang tahan-usik dan minim-kepercayaan.

5. **Pengenal objek digital (DOI) sebagai token *non-fungible* yang tertaut CID.** Setiap naskah yang telah diterbitkan memperoleh pengenal objek digital yang dicetak sebagai token berstandar ERC-721 dan ditautkan langsung dengan CID naskah pada IPFS. Perlu ditegaskan bahwa mekanisme ini tidak menambah proteksi terhadap

pengubahan data, karena kontrak tidak lagi mengizinkan CID naskah diubah begitu status mencapai *PUBLISHED*, sebagaimana telah ditegakkan oleh mesin status pada mekanisme pertama. Fungsi mekanisme ini bersifat komplementer, bukan mengulangi jaminan integritas yang sudah ada: memberikan naskah yang telah terbit sebuah pengenal berformat DOI yang dapat disitasi secara standar, sejalan dengan tahap *Publication* pada alur kerja konvensional (Tabel II.1), sekaligus merepresentasikan kepemilikannya melalui token yang dapat dikenali dompet maupun perkakas yang mendukung standar ERC-721 tanpa memerlukan integrasi khusus terhadap struktur data sistem. Token ini juga menjadi tempat melekatnya fitur komentar pembaca pascapublikasi.

6. **Mekanisme anonimitas berbasis alamat samaran.** Untuk menjawab ketegangan antara transparansi dan anonimitas, sistem menjamin anonimitas peran penulis dan penelaah menggunakan alamat samaran yang dibangkitkan khusus untuk setiap pengajuan maupun penugasan, sementara pemetaan terhadap identitas asli disimpan secara terpisah di luar rantai. Mekanisme ini memungkinkan jejak proses evaluasi tetap tercatat secara transparan dan dapat diaudit pada *blockchain*, tanpa mengungkap identitas asli partisipan, sehingga objektivitas penilaian tetap terjaga dari bias personal maupun institusional. Perlakuan atas kedua peran ini sengaja dibuat tidak simetris pada tahap pascakeputusan: identitas penulis dapat dibuka kembali setelah keputusan akhir ditetapkan sebagai bentuk transparansi pascapublikasi, sedangkan identitas penelaah tetap dirahasiakan secara permanen. Ketidaksimetrisan ini diperlukan karena penelaah yang mengetahui identitasnya akan dibuka pada akhirnya berpotensi menyesuaikan putusannya secara tidak jujur selama proses ulasan, misalnya menjadi segan mengkritik naskah dari penulis yang lebih senior karena mengantisipasi konsekuensi sosial di kemudian hari, persis persoalan yang diidentifikasi pada Subbab III.1.1.4.

Keenam rancangan di atas dimaksudkan untuk bekerja secara terpadu dalam satu ekosistem *end-to-end*, sehingga sekaligus menjawab persoalan fragmentasi solusi yang telah diidentifikasi pada Subbab III.1.1. Rancangan solusi yang telah diuraikan pada subbab ini selanjutnya diterjemahkan menjadi arsitektur sistem yang dijabarkan pada Subbab III.4.

III.4 Perancangan Arsitektur Sistem

Pada tugas akhir ini, arsitektur sistem publikasi ilmiah dirancang dengan mengadopsi pendekatan arsitektur *blockchain* hibrida berbasis Web3.

III.4.1 Arsitektur *Blockchain* Hibrida

Arsitektur *blockchain* hibrida adalah pendekatan yang memisahkan operasi suatu sistem menjadi dua kelompok, yaitu operasi yang dieksekusi dan ditegakkan secara terdesentralisasi pada *blockchain*, serta operasi yang dieksekusi secara terpusat pada komponen *off-chain* (Solaiman dkk., 2021). Solaiman dkk. (Solaiman dkk., 2021) menunjukkan bahwa pendekatan murni terpusat maupun murni terdesentralisasi masing-masing memiliki kelemahan mendasar untuk kelas aplikasi yang cukup luas. Pendekatan terpusat sepenuhnya bergantung pada pihak ketiga tepercaya (*trusted third party*) sebagai satu-satunya penjamin kebenaran data, sehingga menghadirkan titik kegagalan tunggal (*single point of failure*). Sebaliknya, pendekatan terdesentralisasi penuh membebankan seluruh operasi, termasuk operasi yang sebenarnya tidak krusial secara kepercayaan, pada mekanisme konsensus *blockchain* yang mahal dan terbatas kapasitasnya, sehingga menjadi tidak praktis untuk aplikasi dengan volume data maupun frekuensi transaksi yang tinggi. Pemisahan operasi pada arsitektur hibrida didasarkan pada beberapa pertimbangan, antara lain biaya transaksi, performa, kemampuan bahasa pemrograman *smart contract*, dan kebutuhan privasi data (Solaiman dkk., 2021).

Pendekatan ini dinilai cocok untuk sistem publikasi ilmiah pada tugas akhir ini karena kedua kelemahan yang diuraikan di atas merupakan dua persoalan yang secara eksplisit ingin dihindari sekaligus diselesaikan, sebagaimana dianalisis pada Subbab III.1.1. Pendekatan murni terpusat akan menghadirkan kembali persoalan kepercayaan pada pihak ketiga yang justru menjadi kesenjangan utama sistem publikasi konvensional, sedangkan pendekatan murni terdesentralisasi akan membebankan seluruh data, termasuk berkas naskah dan data profil pengguna yang berukuran besar dan tidak memerlukan bukti tidak dapat diubah secara publik, pada *blockchain* yang mahal dan lambat untuk keperluan tersebut. Arsitektur hibrida memungkinkan kedua kebutuhan yang tampak bertentangan ini dipenuhi sekaligus, kepercayaan minimum untuk data dan keputusan yang krusial, serta performa dan efisiensi biaya untuk data dan operasi yang tidak memerlukan jaminan tersebut.

Penerapannya pada rancangan tugas akhir ini mengikuti prinsip pemisahan operasi yang

sama, dengan kriteria yang disesuaikan pada kebutuhan sistem publikasi ilmiah. Operasi yang menjadi inti kepercayaan sistem, yaitu pencatatan siklus hidup naskah, evaluasi keputusan mayoritas, dan distribusi insentif, dieksekusi dan ditegakkan oleh *smart contract* pada jaringan *blockchain* sebagai komponen terdesentralisasi. Sementara itu, operasi yang menuntut performa tinggi, menangani data berukuran besar, atau memerlukan interaksi dengan sumber data eksternal ditangani oleh subsistem *off-chain*. Kriteria pemisahan data antara kedua kelompok komponen ini dirinci lebih lanjut pada Subbab III.4.3.

III.4.2 Komponen Arsitektur Sistem

Pembagian sistem ini ke dalam beberapa subsistem bukanlah keputusan sembarang, melainkan mengikuti dua pertimbangan utama yang lahir langsung dari kebutuhan pada Subbab III.2. Pertimbangan pertama adalah batas antara data yang *harus* memiliki bukti tidak dapat diubah secara publik dengan data yang tidak memerlukan hal tersebut, sebagaimana dirinci klasifikasinya pada Subbab III.4.3. Pertimbangan kedua adalah arah pemicu interaksi. Komponen yang bertugas meneruskan aksi *dari* pengguna *menuju blockchain* memerlukan wewenang untuk menandatangani dan membiayai transaksi serta mengetahui pemetaan identitas pengguna, sedangkan komponen yang bertugas membawa data *dari* dunia luar *menuju blockchain* hanya perlu membaca permintaan dan meneruskan hasilnya, tanpa perlu mengetahui identitas pengguna mana pun. Karena kedua arah interaksi ini memiliki batas kepercayaan yang berbeda, keduanya dipisahkan menjadi subsistem tersendiri alih-alih digabung menjadi satu komponen *off-chain* tunggal.

Berdasarkan kedua pertimbangan tersebut, arsitektur sistem ini dibagi menjadi lima subsistem utama, yaitu lapisan antarmuka aplikasi (*frontend*), lapisan layanan aplikasi (*backend*), *oracle*, *smart contract*, dan basis data *off-chain*. Peran dan tanggung jawab masing-masing entitas pada sistem yang dikembangkan adalah sebagai berikut:

1. **Antarmuka Aplikasi / *Frontend*:** Lapisan presentasi yang memfasilitasi interaksi langsung antara pengguna dan sistem, sekaligus menyembunyikan seluruh kerumitan *blockchain* dari pengguna akhir.
2. **Layanan Aplikasi / *Backend*:** Komponen *off-chain* terpusat yang meningkatkan performa sistem dan menjadi satu-satunya pihak yang berwenang meneruskan aksi pengguna menjadi transaksi *blockchain*, sejalan dengan pertimbangan arah interaksi di atas.

3. ***Oracle*:** Komponen *off-chain* yang menyuplai data eksternal kepada *smart contract*, mengingat sifat jaringan *blockchain* yang terisolasi dari dunia luar.

4. ***Smart Contract*:** Program yang berjalan secara otonom di atas jaringan *blockchain*, berperan sebagai *backend* terdesentralisasi utama untuk menjamin kekekalan dan transparansi data.

5. **Basis Data *Off-Chain*:** Komponen yang menyimpan data sensitif yang tidak memiliki padanan *on-chain* maupun tidak aman jika dicatatkan pada jaringan *blockchain* publik.

Rincian tiap komponen *backend* dan keempat *smart contract* di atas dijelaskan lebih lanjut pada Subbab III.5.

III.4.3 Klasifikasi Data *On-Chain* dan *Off-Chain*

Subbab ini merinci penerapan konkret pertimbangan pertama pada Subbab III.4.2, yaitu pemilihan data yang dicatat pada tiap lapisan mengikuti sifat masing-masing jenis data, bukan keputusan sembarang.

Data yang dirancang untuk disimpan *on-chain* hanyalah data berukuran kecil yang menjadi inti keputusan sistem. Seluruh data tersebut cukup dicatat sebagai representasi, bukan salinan utuh, karena yang perlu dijamin keasliannya adalah fakta bahwa suatu peristiwa terjadi (naskah diajukan, penelaah memutuskan, insentif dibayarkan), bukan seluruh isi berkas yang menyertainya. Berkas naskah, isi ulasan, dan dokumen keputusan editor sendiri, yang dapat berukuran ratusan kilobita hingga beberapa megabita, dirancang untuk disimpan pada IPFS melalui Pinata dan hanya direpresentasikan *on-chain* melalui CID-nya, sebagaimana dirancang lebih lanjut pada Subbab III.5.3 dan diimplementasikan pada Subbab IV.4.5.

Keputusan ini didasari dua pertimbangan. Pertama, biaya transaksi: setiap satuan data yang ditulis *on-chain* dikenakan biaya *gas*, sehingga menyimpan berkas utuh secara *on-chain* akan membuat biaya pengajuan naskah menjadi tidak praktis. Kedua, ukuran data: jaringan *blockchain* publik seperti Ethereum tidak dirancang sebagai media penyimpanan berkas besar, karena setiap simpul pada jaringan harus turut menyimpan salinan seluruh riwayat data *on-chain*, sehingga penyimpanan berkas besar akan membebani seluruh jaringan alih-alih hanya sistem ini.

Sebaliknya, tidak seluruh data dapat diserahkan ke lapisan *off-chain*, karena basis data *off-chain* bersifat dapat diubah (*mutable*) dan berada di bawah kendali penyelenggara sistem. Fakta-fakta yang menjadi dasar kepercayaan sistem, yaitu siapa mengajukan apa, kapan, dan keputusan apa yang dihasilkan, harus dicatat pada media yang tidak dapat diubah sepihak, yaitu *ledger blockchain*, sebagaimana dirancang lebih lanjut pada Subbab III.5.1 dan diimplementasikan pada Subbab IV.4.1.

Pertimbangan sebaliknya berlaku untuk data pribadi pengguna, seperti kata sandi dan alamat surel. Karena data yang tercatat pada *blockchain* publik bersifat transparan dan dapat dibaca oleh siapa pun tanpa izin, data semacam ini justru tidak mungkin disimpan *on-chain* tanpa mengorbankan privasi pengguna, sehingga wajib disimpan pada basis data *off-chain* yang aksesnya dapat dibatasi.

Tabel III.5 merangkum pemetaan lengkap data sistem ke lapisan penyimpanannya.

Tabel III.5: Klasifikasi data *on-chain* dan *off-chain*

Data	**Lokasi**	**Keterangan**
Alamat penulis dan penelaah	*On-Chain*	Merepresentasikan keterlibatan pada siklus hidup naskah tanpa mengungkap identitas asli.
Status naskah	*On-Chain*	Nilai mesin status siklus hidup publikasi.
Penghitung suara ulasan	*On-Chain*	Dasar evaluasi keputusan mayoritas.
CID berkas naskah, ulasan, dan dokumen editor	*On-Chain*	Sidik jari kriptografis berkas, bukan isi berkas itu sendiri.
Skor plagiarisme	*On-Chain*	Angka ringkas hasil pemeriksaan *oracle*.
Nomor DOI	*On-Chain*	Pengenal objek digital naskah terbit.
Hash komentar pembaca	*On-Chain*	Bukti integritas komentar tanpa menyimpan isinya.
Berkas asli naskah	*Off-Chain* (IPFS)	Berukuran besar, direferensikan *on-chain* melalui CID.

Tabel III.5: Klasifikasi data *on-chain* dan *off-chain*

Data	Lokasi	Keterangan
Isi ulasan penelaah	*Off-Chain* (IPFS)	Berukuran besar, direferensikan *on-chain* melalui CID.
Dokumen keputusan editor	*Off-Chain* (IPFS)	Berukuran besar, direferensikan *on-chain* melalui CID.
Isi teks komentar pembaca	*Off-Chain* (PostgreSQL)	Hanya *hash*-nya yang tercatat *on-chain*.
Data akun pengguna	*Off-Chain* (PostgreSQL)	Data pribadi sensitif, tidak memiliki padanan *on-chain*.

III.4.4 Karakteristik *Framework* pada Rancangan

Rumusan masalah pada Subbab I.2 menempatkan artefak yang dihasilkan sebagai sebuah *framework*, bukan sekadar aplikasi. Fayad dan Schmidt (1997) mendefinisikan *framework* sebagai aplikasi semi-lengkap yang dapat digunakan ulang dan dapat dispesialisasikan untuk menghasilkan aplikasi yang sesuai dengan kebutuhan penggunanya. Dalam penelitian ini, karakteristik *framework* pada artefak yang dikembangkan dianalisis melalui tiga aspek berikut.

1. **Memiliki bagian inti yang stabil dan terkendali serta bagian yang dapat disesuaikan atau diganti.** *Framework* memisahkan antarmuka dan perilaku inti yang stabil dari bagian-bagian yang bervariasi sesuai kebutuhan setiap instansiasinya, serta membungkus detail yang mudah berubah di balik antarmuka yang stabil (Fayad dan Schmidt, 1997).

2. **Dapat disesuaikan dan dikonfigurasi.** Cara penyesuaian *framework* berada dalam suatu rentang antara pendekatan *white-box* dan *black-box* (Fayad dan Schmidt, 1997). Pada pendekatan *white-box*, fungsionalitas diperluas melalui pewarisan dan penimpaan metode *hook* yang telah disediakan, sehingga menuntut pemahaman atas struktur internal *framework*. Pada pendekatan *black-box*, fungsionalitas disesuaikan dengan menyusun komponen yang mengikuti antarmuka tertentu lalu mengintegrasikannya ke dalam *framework* melalui komposisi, sehingga umumnya lebih mudah digunakan (Fayad dan Schmidt, 1997).

3. **Dapat digunakan kembali.** Antarmuka yang stabil memungkinkan komponen

generik pada *framework* dipakai ulang untuk membangun aplikasi baru (Fayad dan Schmidt, 1997).

Uraian berikut menunjukkan wujud ketiga aspek tersebut pada rancangan tugas akhir ini.

Bagian inti yang stabil. Operasi yang menjadi inti kepercayaan sistem ditempatkan pada *smart contract* dan tidak dapat dikonfigurasi secara bebas pada setiap instansiasi *framework*, sedangkan komponen di luar inti ditempatkan pada lapisan *off-chain* agar dapat disesuaikan.

Jenis penyesuaian. Penyesuaian terhadap *framework* dibedakan menjadi lima jenis berdasarkan bagian yang diubah, mulai dari yang tidak memerlukan perubahan kode hingga yang menyentuh bagian inti.

1. **Konfigurasi data.** Katalog jurnal, kategori bidang kajian, serta editor beserta bidang keahliannya disimpan sebagai data pada basis data *off-chain* (Subbab III.5.1.5). Penyelenggara dapat menambahkan jurnal, kategori, maupun editor tanpa mengubah kode maupun *smart contract*.

2. **Konfigurasi *deployment*.** Alamat *smart contract* dan titik akses RPC jaringan *blockchain* ditentukan melalui konfigurasi pada saat penyebaran, sehingga *framework* dapat dihubungkan dengan kontrak dan jaringan yang berbeda tanpa mengubah kode.

3. **Substitusi komponen.** Penyedia penyimpanan berkas (Subbab III.5.1.3), penyedia verifikasi identitas (Subbab III.5.1.4), dan layanan pemeriksaan plagiarisme (Subbab III.5.2.2) dapat diganti tanpa mengubah *smart contract*, karena kontrak hanya mencatat hasilnya, seperti CID berkas dan skor plagiarisme, tanpa bergantung pada penyedia yang menghasilkannya. Namun, cara penggantian ketiganya berada pada posisi yang berbeda dalam rentang *white-box* hingga *black-box*. Layanan pemeriksaan plagiarisme paling mendekati pendekatan *black-box*, karena dapat diganti cukup melalui konfigurasi alamat layanan selama layanan pengganti mengikuti format permintaan dan respons yang sama. Penyedia penyimpanan berkas dipasang melalui antarmuka yang telah ditentukan, tetapi pemasangan penyedia baru masih memerlukan penambahan implementasi pada lapisan *off-chain*. Adapun penyedia verifikasi identitas belum dapat dikategorikan *black-box*, karena penggantiannya masih memerlukan perubahan pada beberapa bagian implementasi.

4. **Modifikasi kode.** Aturan penugasan editor (Subbab III.5.1.5) dan pemilihan

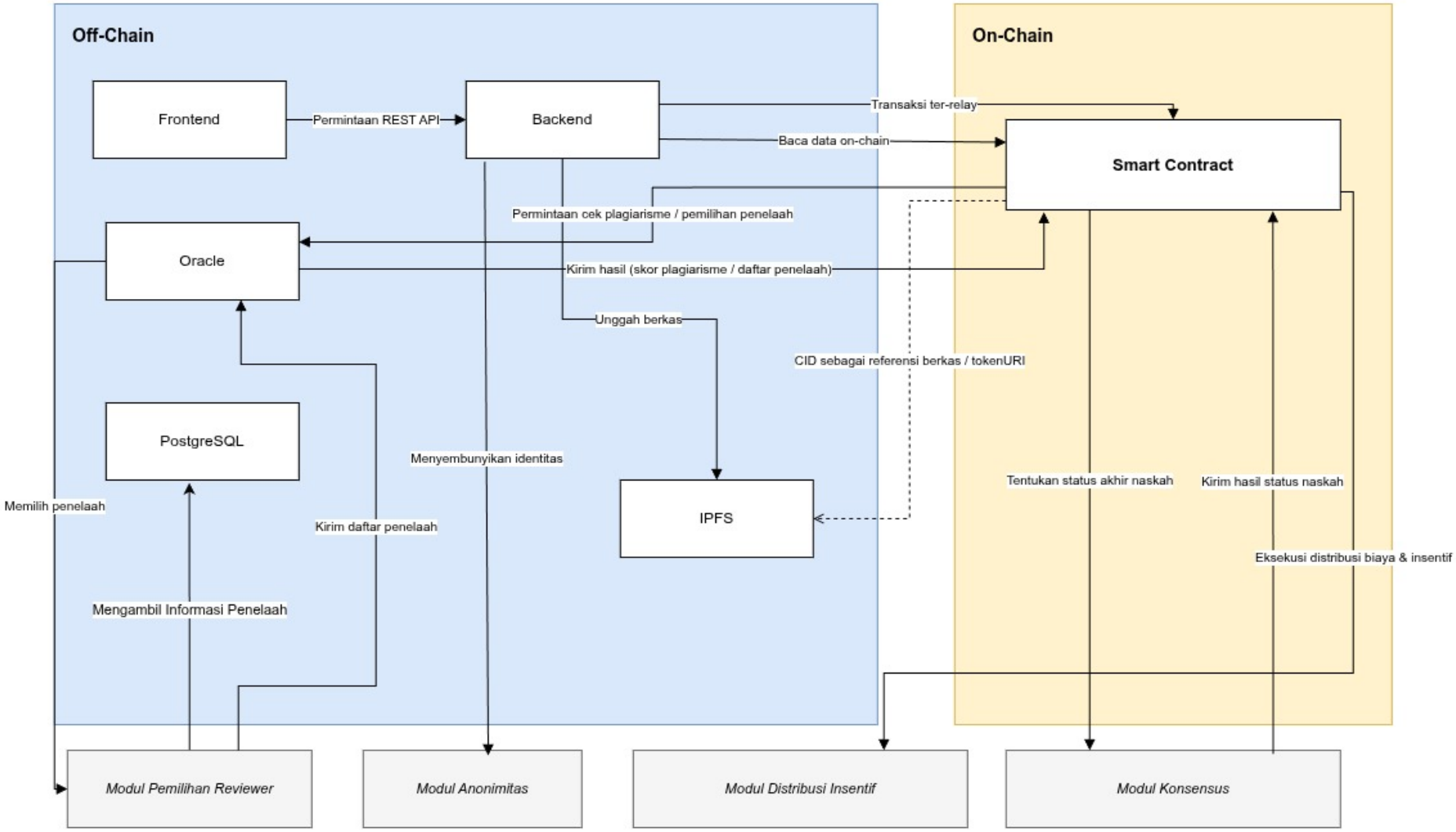
penelaah (Subbab III.5.6) dijalankan pada lapisan *off-chain* sebagai solusi awal. Perubahan algoritmanya, misalnya dengan menambahkan pertimbangan reputasi atau konflik kepentingan, dilakukan dengan memodifikasi kode pada lapisan *off-chain* tanpa menyentuh bagian inti.

5. **Pembaruan inti melalui UUPS.** Perubahan terhadap logika inti hanya dapat dilakukan melalui mekanisme pembaruan *proxy* UUPS oleh pemegang *ADMIN_ROLE*. Konsekuensi berupa titik kepercayaan terpusat pada kewenangan pembaruan tersebut diakui sebagai keterbatasan rancangan pada Subbab III.5.2.1.

Penggunaan kembali. Satu instans *framework* dirancang untuk menaungi banyak jurnal sekaligus, alih-alih hanya melayani satu jalur publikasi tunggal. Karena bagian inti tidak terikat pada jurnal maupun penyedia layanan tertentu, *framework* juga dapat diinstansiasi ulang oleh penyelenggara lain dengan menyebarkan ulang *smart contract* serta mengisi katalog jurnal dan editornya sendiri.

III.5 Perancangan Komponen

Setelah rancangan arsitektur sistem dan klasifikasi data secara umum dijabarkan pada subbab sebelumnya, bagian ini menguraikan rancangan tiap komponen utama secara lebih mendalam, yaitu *backend*, *smart contract*, dan mekanisme penyimpanan data *off-chain*. Perancangan komponen ini disusun berdasarkan kebutuhan fungsional dan non-fungsional yang telah dirumuskan pada Subbab III.2.

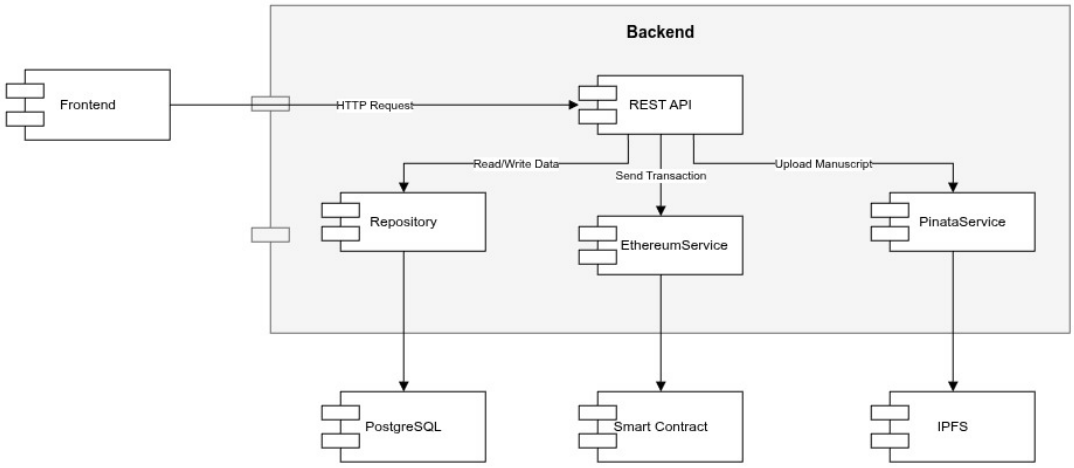


Gambar III.1: Diagram Arsitektur Sistem Publikasi Ilmiah

Modul-modul yang digambarkan di luar kotak *Off-Chain* dan *On-Chain* pada Gambar III.1 sengaja diletakkan terpisah karena komponen penyusun tiap modul dapat berasal dari lapisan *off-chain* maupun *on-chain*, sehingga tidak selalu dapat dikelompokkan ke dalam salah satu kotak saja.

III.5.1 Perancangan Backend

Sejalan dengan komponen *backend* yang telah diperkenalkan pada subbab sebelumnya, yaitu *REST API*, *EthereumService*, dan *PinataService*, bagian ini menguraikan rancangan tiap komponen tersebut secara lebih rinci, ditambah dengan rancangan mekanisme verifikasi identitas pengguna serta manajemen jurnal sebagai pelengkap komponen *backend*.



Gambar III.2: Diagram Komponen Backend

III.5.1.1 REST API

Komponen *REST API* bertindak sebagai pintu masuk utama bagi seluruh permintaan yang dikirim *frontend*, menyediakan seluruh kebutuhan data maupun aksi pengguna melalui satu antarmuka terpadu. Untuk data yang bersumber dari *blockchain*, seperti status naskah dan riwayat ulasan, komponen ini membaca langsung dari *smart contract* pada saat permintaan diterima. Untuk data sensitif yang tidak dicatatkan *on-chain*, seperti identitas pengguna dan pemetaan dompet sesi, komponen ini membaca dari basis data *off-chain* melalui *Repository*. Permintaan yang mengubah status sistem, seperti pengajuan naskah dan pengiriman ulasan, diteruskan kepada *EthereumService* untuk direlai sebagai transaksi *blockchain*.

III.5.1.2 EthereumService

Komponen *EthereumService* dirancang untuk menjalankan peran sebagai *relayer* tepercaya yang meneruskan data tanda tangan EIP-712 yang telah dibubuhkan penulis maupun penelaah ke dalam transaksi *blockchain* yang sesungguhnya, sekaligus menanggung biaya *gas* atas nama pengguna pada aksi tersebut. Komponen ini memegang kunci privat dari sebuah dompet operator yang secara khusus digunakan untuk menanggung biaya *gas* atas setiap transaksi yang direlai, mencakup pengajuan manuskrip, pengajuan ulasan, keputusan penyaringan editor, dan verifikasi bidang penelaah. Keputusan agar *sistem*, bukan pengguna, yang menanggung biaya *gas* pada aksi-aksi tersebut bertumpu pada alasan yang sama dengan desain antarmuka pengguna pada Subbab III.4.2. Target pengguna sistem ini adalah peneliti, penelaah, dan editor yang umumnya tidak memiliki pengalaman dengan dompet kripto maupun mata uang kripto, sehingga mewajibkan mereka memiliki saldo Ether hanya untuk mengajukan naskah atau mengirim ulasan akan menghadirkan kembali kurva pembelajaran yang curam yang justru ingin dihindari. Perlu ditegaskan bahwa pendekatan ini tidak menghilangkan seluruh interaksi pengguna dengan aset kripto pada sistem. Penulis tetap perlu memiliki *token* JRT dan menggunakan dompet aslinya sendiri pada tahap pelunasan biaya publikasi (Subbab III.6.1.4), karena tahap tersebut melibatkan pertanggungjawaban langsung atas dana milik pengguna sehingga tidak dapat diwakilkan kepada *relayer*.

Wewenang merelai transaksi ini sengaja dibatasi hanya untuk dompet operator *backend*, tidak dibuka bagi pihak mana pun yang kebetulan memegang tanda tangan sah sebagaimana lazimnya pola *meta-transaction*. Peran sebagai *relayer* ini secara alami meluas menjadi wewenang eksklusif. Dompet operator *backend* menjadi satu-satunya alamat yang diberi izin melalui kontrol akses berbasis peran (*onlyRole*) pada kontrak untuk memicu fungsi-fungsi yang mengubah status sistem. Alasan pembatasan ini berbeda untuk dua kelompok fungsi yang direlai. Untuk fungsi yang membawa tanda tangan pengguna, seperti pengajuan naskah dan pengiriman ulasan, keabsahan identitas penanda tangan tetap terjaga oleh tanda tangan itu sendiri sekalipun direlai pihak lain, sehingga pembatasan pada fungsi-fungsi ini terutama menjaga konsistensi arsitektur satu pemilik relai, bukan sebagai satu-satunya penjaga keamanan. Sebaliknya, untuk keputusan penyaringan editor dan verifikasi bidang keahlian penelaah, yang dipanggil *backend* tanpa tanda tangan pengguna sama sekali sebagaimana dijelaskan pada Subbab IV.4.2, pembatasan ini justru menjadi satu-satunya penjaga keabsahan. Kedua fungsi tersebut tidak memiliki mekanisme auten-

tikasi kriptografis independen, sehingga keabsahannya sepenuhnya bertumpu pada fakta bahwa hanya alamat operator *backend* yang dapat memanggilnya, setelah *backend* lebih dahulu memverifikasi peran editor melalui otentikasi JWT. Apabila wewenang memanggil kedua fungsi tersebut dibuka bagi sembarang pihak, siapa pun dapat mengklaim dirinya sebagai editor tanpa dapat dibedakan oleh kontrak. Dengan demikian, pembatasan ini turut menjawab kebutuhan kontrol akses pada NFR-04.

III.5.1.3 PinataService

Komponen *PinataService* bertanggung jawab menjembatani *backend* dengan jaringan penyimpanan terdistribusi IPFS melalui layanan Pinata. Komponen ini digunakan untuk mengunggah berkas asli manuskrip pada saat pengajuan, isi ulasan beserta komentar penelaah, serta dokumen keputusan penyaringan editor, sehingga seluruh konten yang berukuran besar tidak perlu disimpan langsung pada *blockchain*, melainkan cukup direferensikan melalui *Content Identifier* (CID) yang dikembalikan oleh IPFS.

III.5.1.4 Verifikasi Identitas Pengguna

Untuk memastikan identitas peneliti yang menggunakan sistem merupakan identitas asli, dirancang mekanisme verifikasi identitas pengguna secara opsional. Mekanisme ini idealnya menggunakan ORCID (*Open Researcher and Contributor ID*), yaitu pengenal unik standar yang lazim digunakan kalangan akademik untuk mengidentifikasi peneliti secara konsisten lintas publikasi dan institusi. Pengguna dapat memasangkan identitas terverifikasinya melalui halaman profil, yang kemudian dikirimkan sebagai token identitas (*ID token*) kepada *backend* untuk diverifikasi.

Backend memvalidasi token tersebut terhadap kunci publik penyedia identitas (*JSON Web Key Set*), memastikan tanda tangannya sah, penerbit (*issuer*) dan audiens (*audience*) sesuai, serta subjek dan surel pengguna tersedia. Subjek unik dari identitas tersebut kemudian dipetakan secara permanen ke satu akun pengguna, sehingga satu identitas terverifikasi hanya dapat dipasangkan pada satu akun sistem.

Surel yang telah terverifikasi ini turut ditampilkan kepada editor pada alur verifikasi bidang keahlian penelaah (Subbab III.6.2), memberikan sinyal tambahan atas kredibilitas identitas calon penelaah saat editor mempertimbangkan persetujuan. Mekanisme ini melengkapi verifikasi bidang keahlian pada FR-09 dan berkaitan dengan persoalan kepercayaan pada pengambilan keputusan yang dianalisis pada Subbab III.1.1.2.

III.5.1.5 Manajemen Jurnal

Agar *framework* ini dapat menaungi banyak jurnal dengan kategori dan kebijakannya masing-masing, alih-alih hanya melayani satu jalur publikasi tunggal, dirancang mekanisme manajemen jurnal. Penulis memilih satu jurnal dari katalog jurnal yang tersedia pada saat mengajukan naskah, dan jurnal yang dipilih ini turut disertakan sebagai bagian dari data metadata naskah yang ditandatangani penulis secara EIP-712 dan dicatatkan *on-chain* sebagaimana dirancang pada Subbab III.5.1.1, alih-alih ditambahkan sebagai kolom tersendiri pada *smart contract*. Pendekatan ini dipilih karena metadata yang sama sudah ditandatangani penulis secara utuh, sehingga pilihan jurnal ikut memperoleh jaminan keaslian yang sama tanpa perlu mengubah struktur data maupun fungsi kontrak yang telah dirancang.

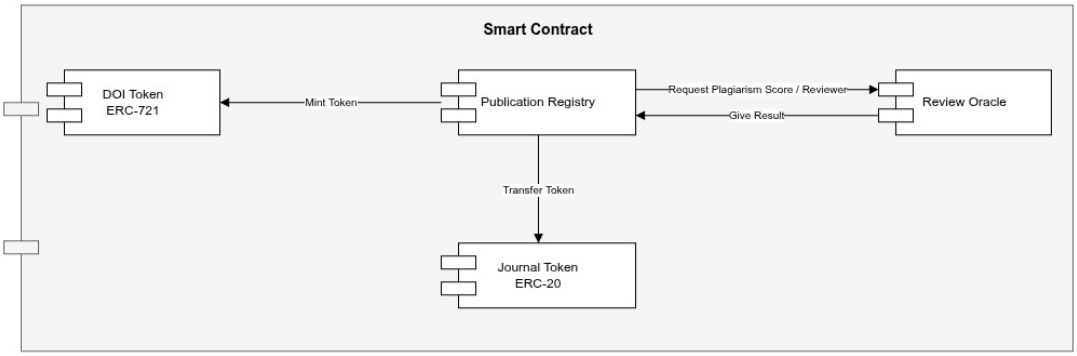
Begitu naskah tercatat *on-chain*, *backend* menurunkan kembali jurnal yang dipilih dari metadata tersebut dan menyimpannya pada basis data *off-chain*, lalu secara otomatis menugaskan naskah kepada seorang editor yang bidang keahliannya tercatat mencakup kategori jurnal tersebut.

Perlu ditegaskan bahwa penugasan editor pada rancangan ini merupakan solusi awal yang sengaja dipilih karena kesederhanaannya, bukan solusi akhir yang tertutup dari pengembangan lebih lanjut. Idealnya, sebagaimana ditunjukkan pada studi *Scientific Journal Platform* (SJP) oleh Beştaş dkk. (Beştaş dkk., 2023), penugasan editor dapat mempertimbangkan kesesuaian bidang penelitian dan reputasi sitasi calon editor, disertai penyaringan agar penulis yang disitasi, berasal dari institusi, maupun wilayah yang sama dengan naskah tidak ditugaskan sebagai editor guna menghindari konflik kepentingan. Namun, demi membuktikan lebih dahulu bahwa penugasan editor dapat berjalan otomatis berdasarkan jurnal yang dipilih penulis, dipilih skema penugasan yang paling sederhana sebagai pembuktian awal, yaitu memilih secara acak satu editor di antara editor yang kategorinya sesuai dengan jurnal yang dipilih, sehingga penyempurnaan kriteria penugasan editor menjadi salah satu arah pengembangan lanjutan yang terbuka.

III.5.2 Perancangan Smart Contract

Pada tugas akhir ini, dirancang dan dikembangkan 4 (empat) buah *smart contract* utama, yaitu *PublicationRegistry*, *ReviewOracle*, *JournalToken*, dan *DOIToken*. Masing-masing *smart contract* ini memiliki fungsionalitas spesifik yang saling berinteraksi untuk mem-

bentuk satu ekosistem sistem publikasi ilmiah yang utuh.



Gambar III.3: Diagram Komponen Smart Contract

III.5.2.1 PublicationRegistry

Sebagai pusat dari sistem, *PublicationRegistry* dirancang sebagai mesin status (*state machine*) utama yang mengendalikan seluruh siklus hidup sebuah manuskrip. Dalam rancangan tersebut, sebuah manuskrip dapat memiliki delapan status berbeda, yaitu *SUBMITTED*, *CHECKING*, *PENDING_EDITOR*, *UNDER_REVIEW*, *REVISION_REQUESTED*, *ACCEPTED*, *REJECTED*, dan *PUBLISHED*. Saat seorang peneliti mengajukan naskahnya melalui fungsi *submitManuscript()*, status naskah tersebut secara otomatis menjadi *CHECKING*. Idealnya, tahap pemeriksaan awal ini mencakup beberapa parameter validasi tambahan selain plagiarisme. Namun, untuk membatasi ruang lingkup pada tugas akhir ini, tahap tersebut diwakilkan secara khusus oleh proses evaluasi plagiarisme yang memicu permintaan kepada *ReviewOracle*. Apabila hasil pengecekan menunjukkan tingkat plagiarisme di bawah batas toleransi, naskah akan maju ke tahap *PENDING_EDITOR* untuk disaring oleh editor. Namun, jika tingkat plagiarisme melebihi batas tersebut, naskah akan langsung ditolak (*REJECTED*). Pemeriksaan tahap awal ini dirancang untuk memenuhi kebutuhan fungsional FR-02.

Kontrak ini juga dirancang agar dapat diperbarui (*upgradeable*) menggunakan pola *proxy* UUPS, berbeda dengan ketiga kontrak pendukung lainnya yang bersifat tetap. Keputusan ini diambil karena *PublicationRegistry* menampung sebagian besar logika bisnis sistem, sehingga paling rentan memerlukan perbaikan cacat logika setelah disebarkan, sedangkan tiga kontrak lainnya memiliki cakupan fungsi yang jauh lebih sempit dan stabil sehingga risiko tersebut jauh lebih kecil. Wewenang memperbarui kontrak ini dibatasi hanya untuk pemegang *ADMIN_ROLE*, sebab tanpa pembatasan tersebut, sembarang pihak berpotensi

mengganti seluruh logika kontrak, termasuk aturan mesin status maupun logika keputusan mayoritas, secara sepihak. Perlu diakui bahwa pembatasan ini menghadirkan satu titik kepercayaan yang terpusat pada pemegang *ADMIN_ROLE*, sebuah ketegangan dengan premis dasar tugas akhir ini pada Subbab I.1.

Ketika naskah berada pada status *PENDING_EDITOR*, editor bertugas melakukan penyaringan awal. Pada tahap ini, editor menentukan bidang kajian naskah dan memutuskan apakah naskah tersebut layak untuk dilanjutkan ke tahap *peer review*. Jika editor menyetujuinya, status naskah berubah menjadi *UNDER_REVIEW*, dan *smart contract* akan otomatis meminta *ReviewOracle* untuk mencarikan penelaah yang sesuai, sebagaimana dirinci pada Subbab III.5.6. Sebaliknya, jika ditolak, naskah akan berstatus *REJECTED*. Bidang kajian yang dipilih oleh editor ini sangat penting karena akan menjadi acuan utama sistem dalam mencocokkan naskah dengan keahlian penelaah.

Evaluasi keputusan akhir naskah berdasarkan suara mayoritas penelaah dirancang sebagai mekanisme tersendiri, dijelaskan lebih lanjut pada Subbab III.5.8.

Apabila naskah mendapatkan status *REVISION_REQUESTED*, penulis dapat memperbaiki dan mengajukan ulang naskahnya. Fungsi ini akan menaikkan versi naskah, menyetel ulang riwayat ulasan, dan mengembalikan status ke *CHECKING* agar naskah diperiksa ulang dari awal. Di sisi lain, jika naskah telah *ACCEPTED*, penulis dapat menyelesaikan proses dengan melakukan pembayaran. Pembayaran ini akan secara otomatis memicu pembagian insentif (Subbab III.5.7) dan pembuatan sertifikat DOI, sebelum akhirnya naskah resmi berstatus *PUBLISHED*.

Interaksi penulis dan penelaah dengan *smart contract* melalui dompet sesi (*session wallet*), tanpa mengharuskan dompet tersebut menanggung biaya *gas*-nya sendiri, dirancang sebagai bagian dari mekanisme anonimitas, dijelaskan lebih lanjut pada Subbab III.5.5.

III.5.2.2 ReviewOracle

Untuk mendukung fungsionalitas tersebut, *ReviewOracle* dirancang murni sebagai jembatan komunikasi antara sistem di *blockchain* dan program *oracle* di luar jaringan (*off-chain*), sehingga logika komunikasi eksternal ini tetap terpisah dari logika siklus publikasi pada *PublicationRegistry* dan dapat diperiksa maupun diuji secara independen. Kontrak ini tidak menyimpan data naskah sama sekali. Setiap kali ada permintaan pengecekan plagiarisme atau pemilihan penelaah, kontrak ini hanya akan memancarkan *event* yang

berisi ID permintaan, guna menjawab kebutuhan FR-02 dan FR-03. Ketika *oracle off-chain* mengirimkan hasilnya kembali, kontrak *ReviewOracle* akan memverifikasi keabsahan ID permintaan tersebut sebelum meneruskan datanya kembali ke *PublicationRegistry*. Khusus untuk pemilihan penelaah, *ReviewOracle* juga memvalidasi bahwa jumlah penelaah yang dikembalikan bernilai ganjil dan minimal tiga orang, sebelum meneruskan daftar tersebut kembali ke *PublicationRegistry*. Penjagaan ini mencegah kontrak menerima daftar penelaah yang berpotensi menghasilkan keputusan seri atau ditentukan oleh terlalu sedikit penelaah.

Seluruh permintaan maupun pemenuhan pada alur ini dibatasi melalui kontrol akses berbasis peran. Hanya *PublicationRegistry* yang dapat mengirimkan permintaan ke *ReviewOracle*, dan hanya dompet operator *oracle* yang dapat mengirimkan pemenuhannya. Pembatasan ini penting karena kedua jenis pemanggilan tersebut tidak memiliki mekanisme autentikasi independen lain seperti tanda tangan digital, sehingga tanpa kontrol akses ini, sembarang pihak dapat langsung memasukkan skor plagiarisme maupun daftar penelaah palsu, melewati proses pemeriksaan *oracle* yang sesungguhnya. Pola yang sama diterapkan secara timbal balik pada *PublicationRegistry*, yang hanya menerima hasil pemenuhan dari alamat kontrak *ReviewOracle* itu sendiri, bukan dari dompet operator *oracle* secara langsung, sehingga kedua kontrak saling memvalidasi identitas satu sama lain di setiap tahap alur permintaan.

III.5.2.3 JournalToken

Dalam ekosistem ini, digunakan *JournalToken* (JRT), yaitu sebuah *smart contract* berstandar ERC-20 yang berfungsi sebagai alat tukar atau mata uang sistem, sekaligus menjadi mekanisme yang memenuhi kebutuhan FR-06 pada Masalah 3 (Kepercayaan pada Distribusi Insentif). Token ini digunakan untuk membayar biaya publikasi dan memberikan imbalan kepada penelaah. Idealnya, sistem insentif seperti ini akan jauh lebih efektif jika menggunakan koin kripto asli yang memiliki nilai moneter riil, karena imbalan finansial yang nyata tentu akan lebih memotivasi masyarakat untuk ikut berkontribusi secara aktif di sistem publikasi. Namun, demi kelancaran dan kemudahan proses pengembangan pada tugas akhir ini, diputuskan untuk merancang token tersendiri berbasis ERC-20, yang fungsi dan perilakunya dapat mensimulasikan aset kripto asli serta tetap dapat ditransfer antarpihak secara bebas. Pada saat sistem pertama kali dijalankan (*deployment*), sebanyak 1.000.000 JRT dicetak dan diberikan kepada akun administrator. Pencetakan token baru

di masa mendatang hanya bisa dilakukan oleh akun yang memiliki hak akses khusus (*MINTER_ROLE*), guna mencegah penerbitan token secara tidak terkendali oleh sembarang pihak yang dapat merusak fungsinya sebagai alat pembayaran maupun insentif dalam sistem.

III.5.2.4 DOIToken

Terakhir, *DOIToken* dirancang sebagai *Non-Fungible Token* (NFT) berstandar ERC-721. Token ini merepresentasikan identitas objek digital atau DOI dari naskah yang telah terbit. Di dalam setiap token ini, tersimpan tautan (*tokenURI*) yang mengarah ke dokumen naskah di jaringan IPFS. Dengan demikian, kepemilikan token ini bertindak layaknya sertifikat resmi bahwa sebuah naskah telah dipublikasikan secara sah (FR-01 dan FR-07). Selain itu, ditambahkan pula fitur *postComment()* pada *DOIToken* ini, yang memungkinkan pembaca publik memberikan komentar pada naskah yang terbit tanpa dibatasi oleh peran mana pun. Fitur ini memperluas akuntabilitas proses publikasi melampaui tahap *peer review* formal yang tertutup, karena kritik maupun tanggapan atas naskah tetap dapat disampaikan secara terbuka oleh siapa pun setelah naskah terbit, tanpa gerbang persetujuan editor maupun penelaah. Demi efisiensi biaya penyimpanan, isi teks komentar tidak disimpan langsung di *blockchain*, melainkan hanya *hash*-nya saja yang dicatat sebagai *event* publik, sehingga keaslian komentar yang ditampilkan tetap dapat diverifikasi terhadap catatan permanen tersebut. Kedua sifat inilah, keterbukaan partisipasi dan keterlacakan keasliannya, yang menjadikan mekanisme ini bentuk transparansi pascapublikasi. Pencetakan token ini dibatasi hanya untuk pemegang *MINTER_ROLE*, yang diberikan khusus kepada *PublicationRegistry*, sehingga DOI hanya dapat diterbitkan melalui alur penerbitan yang sah dan tidak dapat dicetak sembarangan oleh pihak lain yang ingin mengklaim penerbitan naskah secara palsu.

III.5.3 Perancangan Penyimpanan *Off-Chain*

Sejalan dengan klasifikasi data yang telah dijabarkan pada Subbab III.4.3, basis data *off-chain* pada sistem ini dirancang untuk menyimpan data aplikasi yang bersifat sensitif dan tidak memiliki padanan di *blockchain*, mencakup data akun pengguna, pemetaan identitas asli terhadap alamat dompet sesi (baik milik penulis maupun penelaah), permintaan verifikasi bidang keahlian penelaah yang belum maupun telah diputuskan oleh editor, serta isi teks komentar pembaca yang hanya *hash*-nya tercatat pada *blockchain*. Kategori

data ini bersifat sensitif dan sengaja tidak dicatatkan pada *blockchain* publik, baik karena menyangkut privasi pengguna (kata sandi, alamat surel) maupun karena berkaitan langsung dengan mekanisme anonimitas yang dijelaskan pada Subbab III.5.5, sehingga menjawab kebutuhan NFR-03 mengenai desentralisasi tanpa mengorbankan kerahasiaan data pribadi pengguna.

III.5.4 Perancangan Penyimpanan IPFS

Sejalan dengan klasifikasi data yang telah dijabarkan pada Subbab III.4.3, jaringan penyimpanan terdistribusi IPFS pada sistem ini dirancang khusus untuk menyimpan berkas berukuran besar yang tidak layak dicatatkan langsung pada *blockchain*, mencakup berkas asli naskah, isi ulasan penelaah, dan dokumen keputusan penyaringan editor. Ketiga jenis berkas ini diunggah melalui *PinataService* sebagaimana telah dirancang pada Subbab III.5.1, yang menjembatani *backend* dengan jaringan IPFS melalui layanan Pinata.

Setiap berkas yang diunggah menghasilkan *Content Identifier* (CID), yaitu pengenal unik yang dibentuk langsung dari isi berkas melalui fungsi *hash* kriptografis. Sifat ini menjadikan CID sebagai sidik jari digital dari konten yang direpresentasikannya, sehingga perubahan sekecil apa pun terhadap berkas akan menghasilkan CID yang sama sekali berbeda. Hanya CID inilah yang dicatatkan pada *blockchain*, bukan isi berkas itu sendiri, sehingga integritas berkas tetap dapat diverifikasi kapan pun dengan mencocokkan CID hasil pengunduhan ulang terhadap CID yang tercatat *on-chain*, tanpa perlu menyimpan salinan berkas pada *ledger* yang mahal dan terbatas kapasitasnya.

Pendekatan ini menjawab kebutuhan FR-07 sekaligus menjaga konsistensi dengan pertimbangan biaya dan ukuran data yang mendasari klasifikasi *on-chain* dan *off-chain* pada Subbab III.4.3.

III.5.5 Perancangan Modul Anonimitas

Untuk memenuhi kebutuhan *double-blind review* pada FR-08, dirancang mekanisme dompet sesi (*session wallet*), yaitu sepasang kunci kriptografi sementara yang dibangkitkan khusus untuk setiap pengajuan manuskrip maupun penugasan ulasan. Alamat dompet sesi inilah yang tercatat pada *blockchain* sebagai penulis maupun penelaah suatu manuskrip, sehingga identitas asli kedua belah pihak tidak dapat diketahui satu sama lain maupun oleh publik hanya dari data *on-chain*. *Backend* bertanggung jawab menyimpan pemetaan antara identitas asli pengguna dan alamat dompet sesi miliknya secara terpisah pada basis data

off-chain yang tidak dapat diakses oleh pihak lain, sehingga transparansi *blockchain* dan anonimitas proses ulasan dapat terjaga secara bersamaan.

Dompet sesi ini pun sengaja dirancang tanpa saldo Ether sama sekali, sehingga secara struktural tidak mungkin menanggung biaya *gas* atas transaksinya sendiri. Kebutuhan inilah yang mendasari perancangan *EthereumService* sebagai *relayer* pada Subbab III.5.1.2. Setiap aksi atas nama dompet sesi cukup ditandatangani secara EIP-712, tanpa perlu dompet tersebut memiliki saldo apa pun agar dapat dieksekusi ke *blockchain*.

Pada *PublicationRegistry*, penulis dan penelaah berinteraksi dengan *smart contract* melalui dompet sesi tersebut, bukan dompet kripto asli miliknya. Karena dompet sesi tidak memiliki saldo Ether, fungsi *submitManuscript()*, *submitReview()*, dan *reviseManuscript()* dirancang untuk menerima tanda tangan digital dompet sesi yang direlai oleh *backend*, sehingga identitas penanda tangan tetap dapat diautentikasi secara kriptografis tanpa mengharuskan dompet sesi menanggung biaya *gas*-nya sendiri. Mekanisme relai ini diimplementasikan pada Subbab IV.3.1.3 serta IV.4.2.

Perlu ditegaskan bahwa pendekatan penyimpanan pemetaan identitas secara terpusat pada *backend* ini merupakan solusi awal yang sengaja dipilih karena kesederhanaannya, bukan solusi akhir yang tertutup dari pengembangan lebih lanjut. Idealnya, mekanisme anonimitas dapat dirancang dengan mengombinasikan teknik kriptografis terdesentralisasi seperti pseudonim *ephemeral*, *stealth address*, skema *commit–reveal*, serta kredensial anonim berbasis *Zero-Knowledge Proof* (ZKP) dalam model *triple-blind peer review* berbasis *Distributed Ledger Technology* (DLT). Namun, demi membuktikan lebih dahulu bahwa anonimitas dan transparansi dapat berjalan bersamaan pada sistem publikasi berbasis *blockchain*, dipilih pendekatan pemetaan terpusat yang paling sederhana sebagai pembuktian awal, sehingga penguatan mekanisme anonimitas menjadi salah satu arah pengembangan lanjutan yang terbuka.

III.5.6 Perancangan Modul Pemilihan *Reviewer*

Pemilihan penelaah dipicu secara otomatis oleh *PublicationRegistry* setelah editor menyetujui naskah pada tahap penyaringan awal, sebagaimana dirancang pada Subbab III.5.2.1. Kontrak mengirimkan permintaan kepada *ReviewOracle* berdasarkan bidang kajian yang telah ditetapkan editor, yang selanjutnya menjadi acuan utama dalam mencocokkan naskah dengan keahlian penelaah.

Pencarian penelaah yang sesuai bidang kajian itu sendiri dijalankan oleh *Oracle* bersama *backend* di luar jaringan *blockchain*, sebagaimana telah diperkenalkan pada Subbab III.4.2 dan dirinci alurnya pada Subbab III.6.1.2.

Perlu ditegaskan bahwa pemilihan penelaah melalui pengacakan sederhana pada basis data *backend* ini merupakan solusi awal yang sengaja dipilih karena kesederhanaannya, bukan solusi akhir yang tertutup dari pengembangan lebih lanjut, sebagaimana dinyatakan pada Subbab IV.2.3. Idealnya, pemilihan penelaah dapat mempertimbangkan aspek yang lebih kaya, misalnya pembobotan berdasarkan reputasi atau beban kerja penelaah. Namun, demi membuktikan lebih dahulu bahwa penugasan penelaah dapat berjalan otomatis tanpa campur tangan editor, dipilih skema pengacakan yang paling sederhana sebagai pembuktian awal, sehingga penyempurnaan algoritma pemilihan penelaah menjadi salah satu arah pengembangan lanjutan yang terbuka.

III.5.7 Perancangan Modul Distribusi Insentif

Distribusi insentif dirancang untuk dieksekusi oleh *PublicationRegistry* pada saat penulis melunasi biaya publikasi setelah naskahnya berstatus *ACCEPTED*. Biaya publikasi ditetapkan sebesar 100 JRT, dengan insentif untuk masing-masing penelaah sebesar 10 JRT, menggunakan *token* JRT (*JournalToken*) sebagai alat tukar, sebagaimana telah dirancang pada Subbab III.5.2.3.

Ketika penulis membayarkan biaya publikasi, kontrak secara otomatis memotong dana, mendistribusikannya kepada seluruh penelaah yang ditugaskan, dan mencetak DOI secara bersamaan dalam satu transaksi, guna memenuhi kebutuhan FR-06. Ketiga operasi ini, yaitu penarikan biaya, distribusi insentif, dan pencetakan DOI, dirancang untuk berjalan atomik dalam satu transaksi yang sama, sehingga tidak ada kondisi antara yang membiarkan insentif terdistribusi sebagian maupun DOI tercetak tanpa pembayaran yang sah.

Perlu ditegaskan bahwa skema insentif bernilai tetap ini merupakan solusi awal yang sengaja dipilih karena kesederhanaannya, bukan solusi akhir yang tertutup dari pengembangan lebih lanjut, sebagaimana dinyatakan pada Subbab IV.2.3. Idealnya, insentif dirancang secara adaptif dan proporsional berdasarkan evaluasi *Quality of Service* (QoS) multidimensi dan *Multi-Criteria Decision Making* (MCDM) dengan menggabungkan pembobotan hibrida (BWM dan ECWM), perangkingan TOPSIS, serta pemetaan fungsi non-linear sigmoid untuk distribusi imbalan ganda berupa ETH dan token reputasi *Soulbound Token*

(SBT) *non-transferable* dengan peluruhan eksponensial. Namun, demi membuktikan lebih dahulu bahwa distribusi insentif dapat berjalan otomatis dan atomik tanpa campur tangan pihak pengelola, dipilih skema bernilai tetap yang paling sederhana sebagai pembuktian awal, sehingga penyempurnaan skema insentif menjadi salah satu arah pengembangan lanjutan yang terbuka.

III.5.8 Perancangan Modul Konsensus

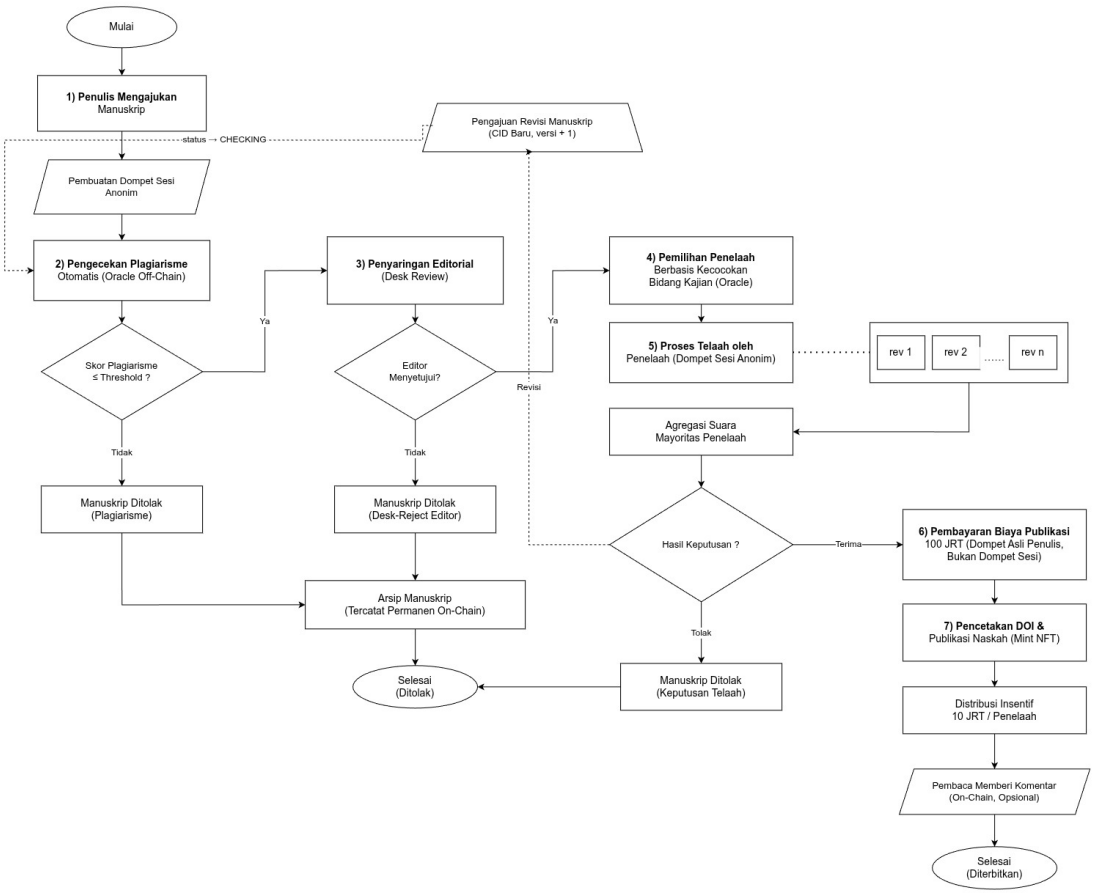
Setelah seluruh penelaah yang ditugaskan menyampaikan ulasan, *PublicationRegistry* secara otomatis mengevaluasi hasilnya menggunakan mekanisme pemungutan suara mayoritas (*majority vote*). Sistem menganggap sebuah keputusan sah apabila jumlah suara telah mencapai setengah dari total penelaah ditambah satu. Jika suara *ACCEPT* mencapai mayoritas, naskah dinyatakan *ACCEPTED*. Jika suara *REJECT* yang dominan, naskah berstatus *REJECTED*. Sementara itu, jika suara *REVISE* yang mendominasi, atau jika tidak ada satu pun keputusan yang mencapai mayoritas mutlak, sistem mengambil langkah aman dengan mengubah status menjadi *REVISION_REQUESTED*. Pendekatan ini diimplementasikan untuk menjawab kebutuhan FR-05.

Untuk memastikan sistem pemungutan suara ini tidak berujung seri, ditetapkan aturan bahwa jumlah penelaah untuk setiap naskah harus bernilai ganjil dan minimal terdiri dari tiga orang, sebagaimana ditegakkan *ReviewOracle* pada Subbab III.5.2.2. Perlu ditegaskan bahwa mekanisme suara mayoritas ini merupakan solusi awal yang sengaja dipilih karena kesederhanaannya, bukan solusi akhir yang tertutup dari pengembangan lebih lanjut. Idealnya, keputusan penerimaan naskah dapat mempertimbangkan aspek yang lebih kaya, misalnya pembobotan suara berdasarkan reputasi atau rekam jejak penelaah, tingkat keyakinan penelaah atas putusannya sendiri, atau substansi argumen di balik putusan tersebut, bukan sekadar menghitung jumlah suara secara setara antarpenelaah. Namun, demi membuktikan lebih dahulu bahwa keputusan editorial dapat dieksekusi secara otonom dan diverifikasi publik sebagaimana dirancang pada Subbab III.3, dipilih skema penghitungan suara yang paling sederhana sebagai pembuktian awal, sehingga penyempurnaan skema pemungutan suara menjadi salah satu arah pengembangan lanjutan yang terbuka.

III.6 Alur Kerja Sistem

Bagian ini menjelaskan alur kerja utama sistem publikasi ilmiah yang telah dirancang, mulai dari pengajuan manuskrip oleh penulis hingga penerbitan naskah dan pencetakan DOI. Setiap alur digambarkan berdasarkan interaksi antar komponen yang telah dijabarkan pada Subbab III.4.2 dan III.5.

III.6.1 Alur Publikasi Naskah



Gambar III.4: Diagram Alur Publikasi Naskah

III.6.1.1 Alur *Submission* Naskah

Alur *submission* naskah diawali ketika penulis mengunggah berkas manuskrip melalui antarmuka *frontend*. Berkas tersebut pertama-tama dikirimkan ke *backend* untuk diunggah ke jaringan IPFS melalui *PinataService*, yang kemudian mengembalikan CID sebagai representasi berkas tersebut. Selanjutnya, *frontend* membangkitkan sepasang dompet sesi baru khusus untuk pengajuan ini, membaca nilai *nonce* milik alamat dompet sesi tersebut dari *smart contract PublicationRegistry*, kemudian membubuhkan tanda tangan digital

berbasis EIP-712 terhadap data CID, metadata naskah, dan *nonce* tersebut menggunakan kunci dompet sesi.

Tanda tangan beserta data terkait kemudian dikirimkan ke *backend*, yang selanjutnya bertindak sebagai *relayer* dengan meneruskan permintaan tersebut ke fungsi *submitManuscript()* pada *PublicationRegistry* menggunakan dompet operator. *Smart contract* memulihkan kembali identitas penanda tangan melalui algoritma ECDSA, mencatatkan manuskrip baru dengan status *CHECKING*, lalu secara otomatis mengirimkan permintaan pemeriksaan plagiarisme kepada *ReviewOracle* melalui fungsi *requestPlagiarismCheck()*, yang kemudian memancarkan *event PlagiarismCheckRequested*.

Oracle off-chain yang senantiasa memantau *event* tersebut akan menerima permintaan ini, lalu mengirimkan CID naskah kepada layanan pemeriksa plagiarisme eksternal untuk memperoleh skor plagiarisme. Skor yang diperoleh kemudian dikirimkan kembali ke *blockchain* melalui fungsi *fulfillPlagiarismCheck()* pada *ReviewOracle*, yang selanjutnya meneruskannya ke fungsi *fulfillPlagiarism()* pada *PublicationRegistry*. Apabila skor yang diterima tidak melebihi ambang batas, status manuskrip berubah menjadi *PENDING_EDITOR* untuk menunggu penyaringan editor. Sebaliknya, apabila skor melampaui ambang batas tersebut, manuskrip langsung berstatus *REJECTED* dan alur berhenti pada tahap ini.

Bersamaan dengan pencatatan naskah tersebut, *backend* menurunkan jurnal tujuan yang dipilih penulis dari data metadata yang tercatat, lalu secara otomatis menugaskan naskah kepada editor yang bidang keahliannya sesuai dengan kategori jurnal tersebut, sebagaimana dirancang pada Subbab III.5.1.5.

III.6.1.2 Alur Pemilihan *Reviewer* secara Otonom

Alur ini dimulai ketika sebuah manuskrip telah berstatus *PENDING_EDITOR* setelah lolos pemeriksaan plagiarisme pada alur sebelumnya. Editor yang bertugas meninjau naskah tersebut, yang telah ditugaskan otomatis berdasarkan jurnal tujuan naskah (Subbab III.5.1.5), menentukan bidang kajian yang paling sesuai serta memutuskan kelayakan naskah untuk dilanjutkan ke tahap ulasan sejawat. Keputusan ini disampaikan melalui fungsi *submitEditorReview()*, yang turut menyertakan referensi dokumen keputusan editor yang telah diunggah ke IPFS. Apabila editor menyetujui, bidang kajian tersebut tersimpan pada data manuskrip, status berubah menjadi *UNDER_REVIEW*, dan *smart contract* se-

cara otomatis mengirimkan permintaan pemilihan penelaah kepada *ReviewOracle* melalui fungsi *requestRandomReviewers()*. Apabila ditolak, manuskrip langsung berstatus *REJECTED*.

Oracle off-chain yang menangkap *event ReviewerSelectionRequested* kemudian membaca bidang kajian manuskrip yang telah ditetapkan editor langsung dari *PublicationRegistry*. Berdasarkan bidang tersebut, *oracle* meminta *backend* untuk menyiapkan dompet sesi bagi penelaah-penelaah yang bidang keahliannya telah diverifikasi oleh editor sesuai alur pada Subbab III.6.2 dan sesuai dengan bidang kajian manuskrip yang bersangkutan. *Backend* kemudian membangkitkan dompet sesi bagi penelaah-penelaah terpilih tersebut dan mengembalikan daftar alamatnya kepada *oracle*.

Daftar alamat penelaah tersebut selanjutnya dikirimkan kembali ke *blockchain* melalui fungsi *fulfillReviewerSelection()* pada *ReviewOracle*. Pada tahap ini, *ReviewOracle* memvalidasi bahwa jumlah penelaah yang diberikan bernilai ganjil dan berjumlah minimal tiga, sebelum meneruskan daftar tersebut ke fungsi *fulfillRandomReviewers()* pada *PublicationRegistry*. *PublicationRegistry* kemudian menyimpan daftar penelaah tersebut sebagai penelaah resmi yang ditugaskan pada manuskrip yang bersangkutan, sehingga manuskrip siap memasuki tahap pemberian ulasan sebagaimana dijelaskan pada Subbab III.6.1.3. Rancangan alur ini menjawab kebutuhan FR-03.

III.6.1.3 Alur Pemberian *Review* dan Keputusan Editorial

Penelaah memberikan ulasan melalui *frontend* dengan memilih putusan (*ACCEPT*, *REJECT*, atau *REVISE*) beserta komentarnya. *Frontend* mengambil kunci dompet sesi penelaah dari *backend*, membaca *nonce* terkini dari *PublicationRegistry*, lalu menandatangani data ulasan tersebut secara EIP-712 menggunakan kunci dompet sesi itu.

Data ulasan beserta tanda tangannya dikirim ke *backend*. *Backend* mengunggah komentar ulasan ke IPFS melalui *PinataService* untuk memperoleh CID, kemudian meneruskannya sebagai transaksi relai ke fungsi *submitReview()* pada *PublicationRegistry*. *Smart contract* memulihkan identitas penelaah dari tanda tangan tersebut, memvalidasi *nonce*, memastikan penelaah memang ditugaskan pada manuskrip yang bersangkutan dan belum pernah mengulas, lalu mencatat putusannya.

Setelah seluruh penelaah yang ditugaskan menyampaikan ulasan, *smart contract* otomatis mengevaluasi keputusan mayoritas sebagaimana dijelaskan pada Subbab III.5.8. Mayoritas

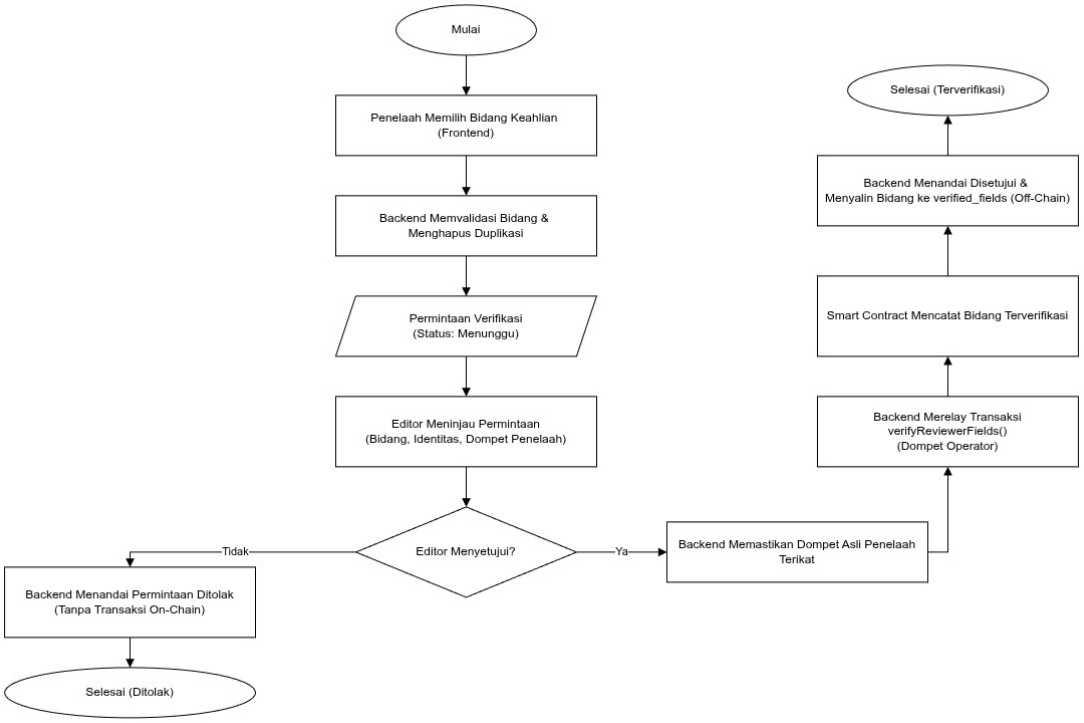
ACCEPT mengubah status menjadi *ACCEPTED* dan manuskrip lanjut ke tahap penerbitan pada Subbab III.6.1.4. Mayoritas *REJECT* mengubah status menjadi *REJECTED* dan alur berakhir. Sementara itu, mayoritas *REVISE*, atau tidak ada mayoritas mutlak, mengubah status menjadi *REVISION_REQUESTED* sehingga penulis dapat mengajukan revisi melalui *reviseManuscript()* dan kembali ke tahap pemeriksaan plagiarisme pada Subbab III.6.1.1 dengan nomor versi baru. Rancangan alur ini menjawab kebutuhan FR-04 dan FR-05.

III.6.1.4 Alur Distribusi *Reward* dan Pencetakan DOI

Alur ini berlaku bagi manuskrip berstatus *ACCEPTED*. Berbeda dengan alur-alur sebelumnya yang direlai *backend* atas nama dompet sesi, pelunasan biaya publikasi dilakukan langsung oleh penulis menggunakan dompet aslinya sendiri, karena melibatkan *token* JRT yang harus dipertanggungjawabkan langsung oleh pemilik dana. *Frontend* lebih dulu memeriksa *allowance* JRT penulis terhadap *PublicationRegistry*, apabila belum mencukupi 100 JRT, penulis memberikan persetujuan (*approve*) melalui dompetnya sendiri sebelum memanggil *payPublicationFee()*.

Smart contract memastikan manuskrip berstatus *ACCEPTED* dan pemanggilnya adalah penulis manuskrip tersebut, lalu menarik 100 JRT dari dompet penulis ke kontrak melalui *transferFrom()*. Insentif sebesar 10 JRT kemudian didistribusikan kepada setiap penelaah yang ditugaskan. Setelah seluruh insentif terdistribusi, kontrak membentuk DOI dari nomor dan versi manuskrip, mencetak *DOIToken* yang tertaut ke berkas manuskrip pada IPFS, lalu mengirimkannya ke dompet penulis. Penarikan biaya, distribusi insentif, dan pencetakan DOI dieksekusi dalam satu transaksi yang sama, sehingga status manuskrip berubah menjadi *PUBLISHED* secara atomik, *frontend* kemudian memantau status ini secara berkala untuk menampilkan DOI yang telah terbit. Rancangan alur ini menjawab kebutuhan FR-01 dan FR-06.

III.6.2 Alur Verifikasi *Reviewer*



Gambar III.5: Diagram Alur Verifikasi *Reviewer*

Berbeda dengan alur-alur lain pada subbab ini, alur verifikasi penelaah tidak terikat pada manuskrip tertentu, melainkan merupakan prasyarat yang harus dipenuhi seorang penelaah sebelum dapat ditugaskan pada naskah apa pun, sesuai kebutuhan FR-09. Penelaah memilih satu atau lebih bidang keahlian dari daftar yang tersedia melalui *frontend*, yang mengirimkan pilihan tersebut ke *backend* melalui *POST /reviewer/specialization*. *Backend* memvalidasi bidang yang diajukan terhadap daftar bidang yang diperbolehkan, menghapus duplikasi, lalu membuat permintaan verifikasi berstatus menunggu.

Editor memantau daftar permintaan yang menunggu melalui *GET /editor/reviewer-verifications*, yang mengembalikan identitas penelaah, alamat dompet, dan bidang yang diajukan untuk setiap permintaan, kemudian memutuskan menyetujui atau menolak setiap permintaan melalui *POST /editor/reviewer-verifications/:id/decision*. Apabila editor menolak, *backend* cukup menandai permintaan tersebut sebagai ditolak tanpa interaksi dengan *blockchain*.

Apabila editor menyetujui, *backend* terlebih dahulu memastikan penelaah telah mengikat alamat dompet aslinya, kemudian meneruskan permintaan sebagai transaksi relai ke fungsi *verifyReviewerFields()* pada *PublicationRegistry* menggunakan dompet operator. Berbeda

dengan alur-alur lain yang membawa tanda tangan pengguna, panggilan ini tidak ditandatangani penelaah karena kewenangannya sepenuhnya berasal dari otoritas operator, bukan dari identitas penelaah itu sendiri, sebagaimana telah dijelaskan pada Subbab IV.4.2. *Smart contract* mencatat bidang yang disetujui pada pemetaan *_verifiedReviewerFields* milik alamat penelaah dan memancarkan *event ReviewerFieldsVerified*. *Backend* kemudian menandai permintaan sebagai disetujui dan menyalin bidang tersebut ke kolom *verified_fields* milik penelaah pada basis data *off-chain*.

Bidang yang tersimpan pada *verified_fields* inilah yang menjadi penyaring utama pada alur pemilihan penelaah (Subbab III.6.1.2): seorang penelaah hanya dapat dipilih untuk suatu manuskrip apabila salah satu bidang terverifikasinya sesuai dengan bidang kajian manuskrip tersebut. Rancangan alur ini menjawab kebutuhan FR-09.

BAB IV

IMPLEMENTASI

Bab ini memaparkan hasil implementasi sistem publikasi ilmiah berbasis *blockchain* yang telah dirancang pada Bab III. Implementasi mencakup dua lapisan utama, yaitu lapisan *on-chain* berupa empat *smart contract* yang berperan sebagai pencatat *immutable* pada jaringan Ethereum, serta lapisan *off-chain* berupa *backend*, *oracle*, dan *frontend* yang mengorkestrasikan alur publikasi sekaligus menjadi perantara antara pengguna dan lapisan *on-chain*.

Implementasi yang disajikan merupakan realisasi dari *framework* yang dirancang pada Bab III, disusun untuk memvalidasi bahwa mekanisme yang diusulkan memenuhi kebutuhan fungsional dan non-fungsional pada Subbab III.2, dengan lingkup sebagaimana dirumuskan pada Subbab I.2 dan dibatasi pada Subbab I.4. Pengujian dan evaluasi terhadap hasil implementasi ini dibahas pada Bab V.

Bab ini dibuka dengan dasar dan strategi implementasi yang memetakan komponen rancangan ke unit implementasi konkret, diikuti oleh lingkup, batasan, dan lingkungan implementasi, kemudian implementasi tiap komponen, dan diakhiri dengan integrasi antarkomponen.

IV.1 Dasar dan Strategi Implementasi

Subbab ini menjelaskan alasan pemilihan pelapisan *on-chain* dan *off-chain* sebagai inti implementasi, serta bagaimana komponen rancangan pada Bab III dipetakan ke dalam unit implementasi yang konkret. Pemetaan ini menjadi jembatan antara rancangan yang bersifat konseptual dan implementasi yang menggunakan teknologi spesifik.

IV.1.1 Pelapisan *On-Chain* dan *Off-Chain*

Lapisan *on-chain* direalisasikan sebagai empat *smart contract* yang masing-masing memuat aturan pencatatan dan transisi status untuk siklus hidup naskah, permintaan layanan *oracle*, alat tukar insentif, serta pengenal objek digital naskah terbit. Keempat kontrak ini menjadi satu-satunya komponen yang bersifat *immutable* setelah disebarkan (*deploy*), sehingga aturan inti rancangan tertanam secara permanen pada *ledger*.

Lapisan *off-chain* direalisasikan sebagai *backend* yang mengorkestrasikan alur publikasi dan bertindak sebagai *relayer*, *oracle* yang menjembatani kontrak dengan layanan eksternal, serta *frontend* sebagai antarmuka pengguna. Dengan pelapisan ini, data dan aturan yang tercatat pada *ledger* langsung mencerminkan rancangan, sedangkan komputasi berat, pengelolaan berkas, dan pemetaan identitas ditempatkan pada lingkungan yang dapat berkembang tanpa mengubah kontrak yang telah disebarkan.

Kombinasi kedua lapisan tersebut mewakili inti rancangan sistem publikasi berbasis *blockchain*, yaitu transparansi pencatatan, akuntabilitas melalui jejak audit, desentralisasi melalui eksekusi kontrak pada jaringan publik, serta imutabilitas data yang telah dikonfirmasi.

Pemilihan data yang dicatat pada tiap lapisan, beserta alasan pemisahannya dari data *off-chain*, telah dianalisis pada Subbab III.4.3. Seluruh klasifikasi pada Tabel III.5 diikuti sesuai rancangan tanpa penyimpangan, dengan realisasi penyimpanan berkas besar pada IPFS melalui Pinata dijelaskan lebih lanjut pada Subbab IV.4.5.

IV.1.2 Pemetaan Rancangan ke Unit Implementasi

Tabel IV.1 menyajikan pemetaan ringkas dari setiap unit implementasi ke subbab perancangan pada Bab III dan kode kebutuhan yang dipenuhinya, sehingga hubungan antara rancangan, implementasi, dan kebutuhan dapat ditelusuri secara langsung.

Tabel IV.1: Pemetaan unit implementasi ke rancangan dan kebutuhan sistem

Unit Implementasi	**Rancangan**	**Kebutuhan**
REST API (*backend*)	III.5.1.1	FR-01, FR-04, FR-08
EthereumService (*backend*)	III.5.1.2	FR-01, FR-04, FR-09, NFR-04
PinataService (*backend*)	III.5.1.3	FR-07, NFR-03
PublicationRegistry	III.5.2.1	FR-01, FR-02, FR-04, FR-09, NFR-01, NFR-02, NFR-04
ReviewOracle	III.5.2.2	FR-02, FR-03, NFR-04
JournalToken	III.5.2.3	FR-06, NFR-04
DOIToken	III.5.2.4	FR-01, FR-07, FR-10, NFR-02
Penyimpanan IPFS	III.5.4	FR-07
Penyimpanan *Off-Chain*	III.5.3	NFR-03

Tabel IV.1: Pemetaan unit implementasi ke rancangan dan kebutuhan sistem

Unit Implementasi	Rancangan	Kebutuhan
Modul Anonimitas	III.5.5	FR-08
Modul Pemilihan *Reviewer*	III.5.6	FR-03
Modul Distribusi Insentif	III.5.7	FR-06
Modul Konsensus	III.5.8	FR-05
Frontend	III.4.2	FR-01, FR-04, FR-08, FR-10

IV.2 Lingkup, Batasan, dan Lingkungan Implementasi

Implementasi pada bab ini merupakan realisasi *framework* yang mencakup seluruh kontrak *on-chain* beserta komponen *off-chain* pendukungnya. Lingkup ini dibatasi pada komponen yang diperlukan untuk menjalankan alur publikasi secara *end-to-end* dan memverifikasi pemenuhan kebutuhan pada Subbab III.2, tanpa membangun ekosistem produksi yang lengkap.

IV.2.1 Lingkungan Implementasi

Bagian ini menjelaskan spesifikasi perangkat lunak dan jaringan yang digunakan dalam mengimplementasikan serta menguji sistem.

IV.2.1.1 Jaringan *Blockchain*

Sistem diuji pada dua jenis jaringan Ethereum yang berbeda sesuai kebutuhannya masing-masing:

- **Hardhat Network (*local*):** Jaringan *blockchain* lokal yang disediakan oleh *framework* Hardhat. Jaringan ini digunakan untuk seluruh pengujian otomatis (*unit test* dan *integration test*) karena setiap blok dapat ditambang secara instan tanpa biaya *gas* yang sesungguhnya, sehingga proses pengujian dapat dilakukan berulang kali secara cepat.
- **Ethereum Sepolia Testnet:** Jaringan uji publik yang digunakan untuk *deployment* pada lingkungan yang lebih menyerupai kondisi produksi. Ether pengujian pada jaringan ini diperoleh melalui *faucet* publik, sedangkan interaksi dengan jaringan dilakukan melalui *RPC endpoint* pihak ketiga yang dikonfigurasi pada berkas *hardhat.config.js*.

IV.2.1.2 Perangkat Lunak

Tabel IV.2 merangkum perangkat lunak, *framework*, dan versi pustaka utama yang digunakan pada setiap subsistem.

Tabel IV.2: Perangkat lunak dan versi yang digunakan pada setiap subsistem

Subsistem	**Perangkat Lunak**	**Versi**
Smart Contract	Solidity	0.8.24
	Hardhat	2.22
	@openzeppelin/contracts	5.1
	@openzeppelin/contracts-upgradeable	5.1
	@openzeppelin/hardhat-upgrades	3.6
Backend	Go	1.23
	Gin	1.10
Oracle	Node.js	20
Frontend	Next.js (*App Router*)	16.2
	React	19.2
Basis Data	PostgreSQL	15

IV.2.1.3 Layanan Pihak Ketiga

Sistem ini memanfaatkan Pinata sebagai layanan *pinning* IPFS untuk menyimpan berkas naskah, isi ulasan, dan dokumen keputusan editor, sebagaimana telah dijelaskan pada Subbab III.5.1.

IV.2.2 Ruang Lingkup Implementasi

Ruang lingkup implementasi *framework* ini berfokus pada:

1. **Alur publikasi *end-to-end*:** Sistem menjalankan seluruh tahapan sebagaimana didefinisikan pada Subbab III.6, mulai dari pengajuan naskah, pemeriksaan plagiarisme otomatis, penyaringan editor, pemilihan penelaah secara otomatis, pengumpulan ulasan, keputusan mayoritas, hingga distribusi insentif dan pencetakan DOI.
2. **Verifikasi melalui pengujian otomatis:** Sistem dilengkapi rangkaian pengujian otomatis pada lapisan *smart contract* dan pengujian terintegrasi terhadap basis data untuk membuktikan kebenaran perilaku sistem.

IV.2.3 Batasan Implementasi

Batasan implementasi yang relevan untuk interpretasi hasil pengujian dan evaluasi adalah sebagai berikut.

1. **Nilai ekonomi *token*:** *JournalToken* (JRT) merupakan *token* ERC-20 yang dirancang khusus untuk sistem ini dan tidak memiliki nilai ekonomi riil. *Token* ini berfungsi mensimulasikan aset kripto pada alur pembayaran dan insentif, sebagaimana telah dijelaskan pada Subbab III.5.2.3.

2. **Ketergantungan pada penyedia *pinning* tunggal:** Penyimpanan berkas pada IPFS dilakukan melalui satu penyedia layanan *pinning*, yaitu Pinata. Meskipun protokol IPFS bersifat terdistribusi, ketersediaan berkas dalam jangka panjang masih bergantung pada satu penyedia layanan.

3. **Model kepercayaan *backend*:** *Backend* diperlakukan sebagai komponen tepercaya yang mengetahui pemetaan antara identitas asli pengguna dan alamat dompet sesi, serta memegang kunci dompet operator sebagai *relayer*.

4. **Kesederhanaan mekanisme konsensus, insentif, dan pemilihan penelaah:** Sesuai fokus *framework* sebagaimana dinyatakan pada Subbab III.1.1, keputusan penerimaan diwujudkan sebagai suara mayoritas sederhana, insentif berupa pembagian *token* tetap, dan pemilihan penelaah berdasarkan pengacakan sederhana pada basis data *backend* yang disaring berdasarkan kecocokan bidang keahlian, tanpa pendalaman terhadap model konsensus, skema insentif, maupun algoritma pemilihan penelaah yang lebih kompleks, seperti pembobotan berdasarkan reputasi atau beban kerja, maupun keacakan yang dapat diverifikasi secara *on-chain*.

5. **Penyedia identitas pada verifikasi pengguna:** Mekanisme verifikasi identitas pengguna yang dirancang pada Subbab III.5.1.4 idealnya menggunakan ORCID sebagai pengenal peneliti, sesuai standar yang lazim di kalangan akademik. Namun, demi mempermudah implementasi pada tugas akhir ini, penyedia identitas yang digunakan adalah Google *Sign-In*, yang menerapkan alur verifikasi *token* identitas yang serupa (OAuth 2.0/OIDC) tanpa memerlukan proses pendaftaran ke lembaga ORCID.

IV.3 Implementasi *Smart Contract*

Keempat *smart contract* yang dirancang pada Subbab III.5.2 diimplementasikan menggunakan bahasa Solidity versi 0.8.24 dan dikembangkan dengan *framework* Hardhat. *PublicationRegistry* merupakan kontrak terbesar karena berperan sebagai pusat logika bisnis sistem. Tabel IV.3 merangkum peran utama setiap kontrak.

Tabel IV.3: Ringkasan *smart contract* yang diimplementasikan

Kontrak	Peran Utama
PublicationRegistry	Mesin status siklus hidup naskah, verifikasi tanda tangan, evaluasi keputusan, serta distribusi biaya dan insentif.
ReviewOracle	Jembatan permintaan dan pemenuhan antara kontrak dan *oracle off-chain*.
DOIToken	Pengenal objek digital naskah terbit beserta kanal komentar pembaca.
JournalToken	Alat tukar untuk biaya publikasi dan insentif penelaah.

Seluruh kontrak menerapkan pola keamanan yang seragam, yaitu kontrol akses berbasis peran menggunakan pustaka *AccessControl* dari OpenZeppelin. Dengan pola ini, setiap fungsi yang mengubah status sistem dibatasi oleh *modifier onlyRole* sehingga hanya dapat dipanggil oleh pemegang peran yang berwenang, sebagaimana dirinci pada Subbab IV.3.5.

IV.3.1 Implementasi *PublicationRegistry*

PublicationRegistry merupakan kontrak inti yang mengendalikan seluruh siklus hidup naskah. Subbab-subbab berikut menjelaskan lima mekanisme utama pada kontrak ini beserta potongan kode yang relevan.

IV.3.1.1 Pola *Proxy* UUPS dan Inisialisasi

Sesuai rancangan pada Subbab III.5.2.1, kontrak ini diimplementasikan sebagai kontrak yang dapat diperbarui menggunakan pola *proxy* UUPS dengan mewarisi *Initializable*, *UUPSUpgradeable*, *AccessControlUpgradeable*, dan *EIP712Upgradeable*. Karena kontrak berada di balik *proxy*, konstruktor tidak dapat digunakan untuk mengisi data awal. Konstruktor justru memanggil *_disableInitializers()* agar kontrak implementasi tidak dapat diinisialisasi secara langsung, sedangkan pengisian data awal dilakukan melalui fungsi

initialize() yang hanya dapat dieksekusi satu kali dan menetapkan alamat ketiga kontrak pendukung beserta pemberian peran awal.

Wewenang pembaruan kontrak dibatasi melalui fungsi *_authorizeUpgrade()* yang diwajibkan oleh *UUPSUpgradeable*. Fungsi ini sengaja dibiarkan berbadan kosong karena seluruh penjagaannya terletak pada *modifier onlyRole*, sebagaimana ditunjukkan pada Kode IV.1.

```
function _authorizeUpgrade(
    address newImplementation
) internal override onlyRole(ADMIN_ROLE) {}
```

Kode IV.1: Pembatasan wewenang pembaruan *proxy* UUPS

Dengan demikian, hanya pemegang *ADMIN_ROLE* yang dapat mengganti logika kontrak, sementara alamat *proxy* beserta seluruh data yang tersimpan di dalamnya tetap tidak berubah.

IV.3.1.2 Struktur Data dan Mesin Status

Delapan status naskah yang dirancang pada Subbab III.5.2.1 direalisasikan sebagai enumerasi *Status* sebagaimana ditunjukkan pada Kode IV.2.

```
enum Status {
    SUBMITTED,
    CHECKING,
    UNDER_REVIEW,
    REVISION_REQUESTED,
    ACCEPTED,
    REJECTED,
    PUBLISHED,
    PENDING_EDITOR
}
```

Kode IV.2: Enumerasi status naskah

Terdapat satu hal yang perlu dijelaskan pada Kode IV.2. Secara alur kerja, status *PENDING_EDITOR* sesungguhnya berada di antara *CHECKING* dan *UNDER_REVIEW*, karena naskah memasuki status tersebut setelah lolos pemeriksaan plagiarisme dan sebelum disetujui editor. Namun, pada Solidity nilai enumerasi disimpan sebagai bilangan bulat

berdasarkan urutan deklarasinya. Apabila status baru disisipkan di tengah, seluruh nilai status yang telah tersimpan pada *ledger* akan bergeser maknanya dan menjadi tidak valid setelah kontrak diperbarui. Oleh sebab itu, *PENDING_EDITOR* sengaja ditempatkan pada indeks terakhir agar nilai status yang sudah tercatat sebelumnya, yaitu indeks 0 sampai 6, tetap sahih. Alasan yang sama mendasari penempatan atribut *field*, yang menyimpan bidang kajian dari editor, pada bagian akhir *struct Manuscript*.

Adapun *struct Manuscript* menyimpan seluruh atribut naskah dalam satu satuan data, meliputi pengenal dan alamat penulis, CID berkas beserta metadata, status dan nomor versi, skor plagiarisme, daftar alamat penelaah yang ditugaskan, penghitung suara untuk masing-masing putusan, pengenal DOI, serta bidang kajian naskah.

IV.3.1.3 Verifikasi Tanda Tangan EIP-712

Mekanisme *meta-transaction* yang dirancang pada Subbab III.5.5 diimplementasikan melalui verifikasi tanda tangan EIP-712. Fungsi *submitManuscript()* hanya dapat dipanggil oleh pemegang *OPERATOR_ROLE*, yaitu dompet operator *backend* yang menanggung biaya *gas*. Meskipun demikian, identitas penulis yang sesungguhnya tidak diambil dari pemanggil transaksi, melainkan dipulihkan dari tanda tangan digital, sebagaimana ditunjukkan pada Kode IV.3.

```
bytes32 structHash = keccak256(abi.encode(
    SUBMIT_MANUSCRIPT_TYPEHASH,
    keccak256(bytes(cid)),
    keccak256(bytes(metadata)),
    nonce
));
address actualAuthor = ECDSA.recover(_hashTypedDataV4(structHash), v, r, s);
if (actualAuthor == address(0)) revert InvalidSignature();
if (nonces[actualAuthor] != nonce) revert InvalidSignature();
unchecked { nonces[actualAuthor]++; }
```

Kode IV.3: Pemulihan identitas penanda tangan dan penjagaan *nonce*

Pada Kode IV.3, data yang ditandatangani terlebih dahulu disusun menjadi *structHash* sesuai definisi tipe pada *SUBMIT_MANUSCRIPT_TYPEHASH*, kemudian dicocokkan dengan tanda tangan menggunakan *ECDSA.recover()* untuk memperoleh alamat penanda tangan. Alamat inilah yang dicatat sebagai penulis naskah, bukan alamat *relayer*. Dua baris

berikutnya menjalankan penjagaan terhadap serangan *replay*, yaitu memastikan *nonce* yang disertakan sama dengan *nonce* tersimpan milik penanda tangan, lalu menaikkannya sehingga tanda tangan yang sama tidak dapat digunakan untuk kedua kalinya. Pola identik diterapkan pada fungsi *submitReview()* dan *reviseManuscript()* dengan *typehash* yang berbeda.

IV.3.1.4 Evaluasi Keputusan Mayoritas

Setiap kali seorang penelaah mengirimkan ulasannya melalui *submitReview()*, kontrak memvalidasi bahwa pemanggil merupakan penelaah yang benar-benar ditugaskan dan belum pernah memberikan ulasan sebelumnya, kemudian menaikkan penghitung suara sesuai putusan yang diberikan. Ketika jumlah ulasan yang masuk telah sama dengan jumlah penelaah yang ditugaskan, fungsi internal *_evaluateDecision()* otomatis dipanggil untuk menetapkan keputusan akhir. Inti logikanya ditunjukkan pada Kode IV.4, dengan pemancaran *event DecisionMade* pada setiap cabang tidak ditampilkan demi keringkasan.

```
uint256 majority = (ms.reviewers.length / 2) + 1;

if (ms.acceptCount >= majority) {
    ms.status = Status.ACCEPTED;
} else if (ms.rejectCount >= majority) {
    ms.status = Status.REJECTED;
} else if (ms.reviseCount >= majority) {
    ms.status = Status.REVISION_REQUESTED;
} else {
    ms.status = Status.REVISION_REQUESTED;
}
```

Kode IV.4: Inti logika penetapan keputusan berbasis suara mayoritas

Ambang mayoritas dihitung sebagai setengah jumlah penelaah ditambah satu menggunakan pembagian bilangan bulat. Untuk tiga penelaah, ambangnya menjadi dua suara, sedangkan untuk lima penelaah menjadi tiga suara. Cabang terakhir pada Kode IV.4 merupakan penanganan kondisi ketika tidak ada satu pun putusan yang mencapai ambang tersebut, misalnya ketika tiga penelaah memberikan tiga putusan yang berbeda. Dalam kondisi tersebut sistem mengambil langkah konservatif dengan menetapkan status *REVISION_REQUESTED*, sehingga naskah tidak pernah diterima maupun ditolak secara otomatis tanpa dukungan mayoritas yang jelas.

IV.3.1.5 Distribusi Insentif dan Pencetakan DOI

Konstanta ekonomi yang mengatur biaya dan insentif ditetapkan sebagai konstanta kontrak sebagaimana dirangkum pada Tabel IV.4.

Tabel IV.4: Konstanta ekonomi pada *PublicationRegistry*

Konstanta	Nilai	Keterangan
PLAGIARISM_THRESHOLD	30	Ambang batas skor plagiarisme. Skor di atas nilai ini menyebabkan naskah langsung ditolak.
PUBLICATION_FEE	100 JRT	Biaya publikasi yang dibayarkan penulis ketika naskah telah diterima.
REVIEWER_INCENTIVE	10 JRT	Insentif yang diterima setiap penelaah yang telah menyelesaikan ulasannya.

Ketika penulis melunasi biaya publikasi melalui *payPublicationFee()*, kontrak menarik biaya dari dompet penulis, lalu mendistribusikan insentif kepada seluruh penelaah yang ditugaskan melalui perulangan sebagaimana ditunjukkan pada Kode IV.5.

```
address[] memory reviewers = _manuscripts[msId].reviewers;
for (uint256 i = 0; i < reviewers.length; i++) {
    bool paid = journalToken.transfer(reviewers[i], REVIEWER_INCENTIVE);
    if (!paid) revert TransferFailed();
    emit IncentivePaid(reviewers[i], REVIEWER_INCENTIVE);
}
```

Kode IV.5: Distribusi insentif kepada seluruh penelaah

Setiap pemindahan *token* diperiksa nilai baliknya dan akan membatalkan seluruh transaksi apabila gagal. Setelah seluruh insentif terdistribusi, kontrak membentuk pengenal DOI dan mencetak *token* ERC-721 sebagaimana ditunjukkan pada Kode IV.6.

```
string memory doi = DOILib.buildDOI(msId, ms.version);
ms.doi = doi;
string memory tokenURI_ = string(
    abi.encodePacked("ipfs://", ms.cid, "?doi=", doi)
);
uint256 doiTokenId = doiToken.mint(msg.sender, tokenURI_);
```

Kode IV.6: Pembentukan DOI dan pencetakan *token* penanda publikasi

Pustaka internal *DOILib* membentuk DOI dengan format berikut.

10.55121/<tahun>.<chainId>.<msId>.<versi>

Komponen tahun pada format tersebut didekati dari *block.timestamp*, sedangkan komponen *chainId* diambil dari *block.chainid* sehingga DOI yang dihasilkan bersifat unik untuk setiap jaringan tempat kontrak disebarkan. Nilai *tokenURI* yang tersimpan pada *token* menautkan berkas naskah pada IPFS sekaligus DOI yang baru dibentuk. Seluruh rangkaian ini, mulai dari penarikan biaya, distribusi insentif, pembentukan DOI, hingga perubahan status menjadi *PUBLISHED*, dieksekusi dalam satu transaksi yang sama sehingga bersifat atomik.

Perlu ditegaskan bahwa mekanisme pembentukan DOI di atas merupakan versi minimum yang dipilih demi kemudahan implementasi, bukan mekanisme yang menempuh proses penerbitan DOI yang sesungguhnya. Pada praktik penerbitan ilmiah yang sebenarnya, DOI hanya dapat diperoleh melalui pendaftaran resmi. Sebuah penerbit terlebih dahulu harus menjadi anggota (*registrant-assigner*) pada salah satu lembaga pendaftaran, misalnya Crossref untuk artikel atau DataCite untuk data penelitian, yang beroperasi di bawah tata kelola *International DOI Foundation* (IDF) dan mengikuti standar ISO 26324:2012 (Carter-Templeton dkk., 2021). Melalui keanggotaan tersebut, penerbit diberikan awalan (*prefix*) unik yang membedakan hasil terbitannya dari penerbit lain, kemudian setiap objek yang diterbitkan didaftarkan beserta metadatanya ke jaringan global lembaga tersebut, sehingga DOI yang dihasilkan benar-benar dapat ditelusuri melalui sistem resolusi *doi.org* dari mana pun objek tersebut diakses (Carter-Templeton dkk., 2021).

IV.3.2 Implementasi *ReviewOracle*

Sesuai rancangan pada Subbab III.5.2.2, *ReviewOracle* diimplementasikan sebagai kontrak *AccessControl* biasa yang tidak dapat diperbarui, dan berfungsi murni sebagai jembatan permintaan dan pemenuhan antara *PublicationRegistry* dan program *oracle off-chain*. Kontrak ini tidak menyimpan data naskah sama sekali, melainkan hanya memetakan nomor permintaan terhadap nomor naskah beserta penanda apakah permintaan tersebut telah dipenuhi.

Setiap permintaan diberi nomor melalui penghitung yang naik secara berurutan. Nomor

tersebut berfungsi sebagai penanda idempotensi, yaitu memastikan satu permintaan hanya dapat dipenuhi satu kali. Percobaan pemenuhan terhadap nomor permintaan yang tidak dikenal maupun yang telah dipenuhi sebelumnya akan ditolak.

Khusus untuk pemilihan penelaah, kontrak menegakkan aturan jumlah penelaah sebagaimana ditunjukkan pada Kode IV.7.

```
1 if (reviewers.length < 3 || reviewers.length % 2 == 0)
2     revert InvalidReviewerCount(reviewers.length);
```

Kode IV.7: Penjagaan jumlah penelaah pada pemenuhan permintaan

Penjagaan pada Kode IV.7 menuntut jumlah penelaah bernilai ganjil dan minimal tiga orang. Syarat ganjil memastikan mekanisme suara mayoritas pada Kode IV.4 selalu dapat mencapai keputusan tanpa kemungkinan seri antara dua putusan, sedangkan syarat minimal tiga memastikan penilaian tidak ditentukan oleh satu orang penelaah saja.

Relasi kepercayaan antara kedua kontrak diatur melalui dua peran. *REGISTRY_ROLE* hanya diberikan kepada alamat *PublicationRegistry* yang sedang aktif melalui fungsi *setPublicationRegistry()*, yang juga otomatis mencabut peran tersebut dari alamat sebelumnya apabila kontrak diganti. Sementara itu, *ORACLE_ROLE* dipegang oleh dompet operator program *oracle off-chain* yang berwenang mengirimkan hasil pemeriksaan kembali ke rantai.

IV.3.3 Implementasi *JournalToken*

JournalToken diimplementasikan sebagai kontrak ERC-20 sederhana yang mewarisi *ERC20* dan *AccessControl* dari OpenZeppelin dalam varian non-*upgradeable*, sesuai rancangan pada Subbab III.5.2.3. Kontrak ini merupakan yang paling ringkas di antara keempat kontrak karena sebagian besar perilakunya diwarisi langsung dari pustaka standar.

Pada saat kontrak disebarkan, konstruktor mencetak seluruh suplai awal sebanyak 1.000.000 JRT kepada akun *deployer* sekaligus memberikan *DEFAULT_ADMIN_ROLE* dan *MINTER_ROLE* kepada akun tersebut. Pencetakan *token* tambahan di kemudian hari hanya dapat dilakukan oleh pemegang *MINTER_ROLE*, dengan penjagaan tambahan berupa penolakan pencetakan ke alamat nol maupun pencetakan bernilai nol.

Token JRT inilah yang digunakan pada alur pembayaran biaya publikasi. Penulis terlebih dahulu memanggil *approve()* untuk memberikan izin kepada *PublicationRegistry* menarik

dan mendistribusikan *token* tersebut, sebagaimana telah dijelaskan pada Subbab IV.3.1.5.

IV.3.4 Implementasi *DOIToken*

DOIToken diimplementasikan sebagai kontrak ERC-721 yang mewarisi *ERC721*, ekstensi *ERC721URIStorage*, dan *AccessControl*, sesuai rancangan pada Subbab III.5.2.4. Fungsi *mint()* dibatasi hanya untuk pemegang *MINTER_ROLE*, yang pada praktiknya adalah *PublicationRegistry* setelah dikonfigurasi pada tahap *deployment*. Setiap *token* yang dicetak menyimpan *tokenURI* melalui mekanisme bawaan *ERC721URIStorage* tanpa penambahan awalan apa pun, sehingga nilai yang tersimpan sama persis dengan yang dikirimkan oleh *PublicationRegistry*.

Selain fungsi pencetakan, kontrak ini menyediakan kanal keterlibatan pembaca pascapublikasi sebagaimana ditunjukkan pada Kode IV.8.

```
function postComment(uint256 doiId, bytes32 hash) external {
    ownerOf(doiId);
    emit CommentPosted(doiId, msg.sender, hash);
}
```

Kode IV.8: Pencatatan komentar pembaca pada naskah terbit

Terdapat dua karakteristik penting pada Kode IV.8. Pertama, fungsi ini sengaja tidak memiliki pembatasan peran sehingga siapa pun dapat memanggilnya, sejalan dengan sifatnya sebagai ruang diskusi publik untuk memenuhi kebutuhan FR-10. Satu-satunya penjagaan adalah pemanggilan *ownerOf()* yang nilainya tidak digunakan, namun akan membatalkan transaksi apabila DOI yang dirujuk belum pernah dicetak. Kedua, isi komentar tidak disimpan pada *storage* kontrak, melainkan hanya nilai *hash*-nya yang dicatat sebagai bagian dari *event CommentPosted*. Teks lengkap komentar disimpan pada basis data *off-chain* dan keasliannya dapat diverifikasi dengan membandingkan *hash*-nya terhadap nilai yang tercatat pada *ledger*. Pendekatan ini dipilih karena penyimpanan *event* jauh lebih murah dibandingkan penyimpanan *storage* pada EVM.

IV.3.5 Konfigurasi Peran

Agar sistem dapat berfungsi, dua konfigurasi peran wajib dilakukan. Pertama, *DOIToken* memberikan *MINTER_ROLE* kepada alamat *proxy PublicationRegistry*, agar kontrak tersebut dapat mencetak DOI. Kedua, *ReviewOracle* memanggil *setPublicationRegistry()* agar

kontrak tersebut mengenali alamat yang berwenang mengirimkan permintaan. Adapun *OPERATOR_ROLE* diberikan kepada dompet operator *backend* secara manual. Rangkuman seluruh peran beserta pemegang dan fungsi yang dijaganya disajikan pada Tabel IV.5.

Tabel IV.5: Matriks peran kontrol akses antarkontrak

Peran	**Pemegang**	**Fungsi yang Dijaga**
ADMIN_ROLE	Administrator	Pembaruan *proxy* UUPS dan pemberian peran lain pada *PublicationRegistry*.
OPERATOR_ROLE	Dompet operator *backend*	Pengajuan naskah, pengajuan ulasan, revisi, keputusan editor, dan verifikasi bidang penelaah.
ORACLE_ROLE (*Registry*)	Kontrak *ReviewOracle*	Pengembalian hasil pemeriksaan plagiarisme dan daftar penelaah ke *PublicationRegistry*.
REGISTRY_ROLE	Kontrak *PublicationRegistry*	Pengiriman permintaan pemeriksaan plagiarisme dan pemilihan penelaah ke *ReviewOracle*.
ORACLE_ROLE (*Oracle*)	Dompet operator *oracle*	Pemenuhan permintaan pada *ReviewOracle*.
MINTER_ROLE (*DOIToken*)	Kontrak *PublicationRegistry*	Pencetakan *token* DOI.
MINTER_ROLE (*JournalToken*)	Administrator	Pencetakan *token* JRT tambahan.

Perlu dicatat bahwa terdapat dua peran tambahan pada *PublicationRegistry*, yaitu *RESEARCHER_ROLE* dan *REVIEWER_ROLE*, yang tetap dipertahankan beserta fungsi pemberiannya. Namun, pada implementasi saat ini kedua peran tersebut tidak lagi menjadi penjaga akses pada fungsi mana pun, karena identitas penulis dipulihkan melalui tanda tangan EIP-712 dan kewenangan penelaah divalidasi melalui keanggotaannya pada daftar

penelaah yang ditugaskan untuk naskah bersangkutan.

IV.4 Implementasi *Backend*

Backend diimplementasikan menggunakan bahasa Go dengan *framework* Gin. Struktur direktorinya mengikuti pola *Clean Architecture* sebagaimana dirancang pada Subbab III.5.1.

Subbab-subbab berikut menjabarkan implementasi empat komponen utama *backend*, yaitu REST API sebagai pintu masuk seluruh permintaan yang membaca data *on-chain* langsung dari kontrak pada setiap permintaan, *EthereumService* sebagai *relayer* transaksi *on-chain*, mekanisme anonimitas berbasis dompet sesi, serta *PinataService* sebagai penghubung ke IPFS.

IV.4.1 Implementasi *REST API*

Seluruh *endpoint* didaftarkan di bawah awalan */api/v1* dan dikelompokkan menurut sumber daya yang dilayaninya. Sistem menyediakan dua puluh tiga *endpoint* sebagaimana dirangkum pada Tabel IV.6, dengan awalan */api/v1* dihilangkan demi keringkasan.

Sejalan dengan rancangan pada Subbab III.5.1, *endpoint* yang menyajikan data *on-chain* (naskah, ulasan, komentar) tidak membaca dari basis data yang disinkronkan sebelumnya, melainkan memanggil kontrak secara langsung pada saat permintaan diterima. Pemanggilan ini ditangani oleh paket internal *chainquery*.

Tabel IV.6: Daftar *endpoint* REST API beserta penjagaan aksesnya

Metode	Path	**Akses**	**Keterangan**
GET	`/health`	Publik	Status layanan beserta status konektivitas RPC *blockchain*.
POST	`/auth/register`	Publik	Pendaftaran akun peneliti atau penelaah.
POST	`/auth/login`	Publik	Verifikasi kredensial dan penerbitan JWT.
POST	`/auth/logout`	Publik	Penanda akhir sesi pada sisi peramban.
GET	`/users/me`	JWT	Pembacaan profil pengguna yang sedang masuk.
PATCH	`/users/wallet-bind`	JWT	Pengikatan alamat dompet asli ke akun pengguna.

Tabel IV.6: Daftar *endpoint* REST API beserta penjagaan aksesnya

Metode	Path	**Akses**	**Keterangan**
GET	`/journals`	Publik	Daftar katalog jurnal beserta kategori bidangnya.
GET	`/manuscripts`	Publik	Daftar naskah dengan penomoran halaman.
GET	`/manuscripts/:id`	Publik	Detail satu naskah beserta riwayat ulasannya.
GET	`/manuscripts/:id/open-review`	Publik	Laporan ulasan terbuka untuk setiap versi naskah.
GET	`/manuscripts/:id/comments`	Publik	Daftar komentar pembaca, dibaca langsung dari *event on-chain*.
POST	`/manuscripts/upload/file`	Publik	Pengunggahan berkas naskah ke IPFS dan pengembalian CID.
POST	`/manuscripts/submit`	JWT	Penerusan naskah bertanda tangan EIP-712 ke kontrak.
POST	`/manuscripts/:id/reviews`	Publik	Penerusan ulasan bertanda tangan EIP-712 ke kontrak.
POST	`/comments/prepare`	JWT	Penyiapan komentar dan pembentukan *hash* keccak256.
POST	`/internal/reviewer-sessions`	Rahasia	Pembangkitan dompet sesi penelaah atas permintaan *oracle*.
GET	`/reviewer/assignments`	JWT	Daftar penugasan ulasan beserta dompet sesinya.
GET	`/reviewer/specialization`	JWT	Status verifikasi bidang keahlian penelaah.
POST	`/reviewer/specialization`	JWT	Pengajuan bidang keahlian untuk diverifikasi editor.
GET	`/editor/reviewer-verifications`	JWT	Daftar pengajuan bidang keahlian yang menunggu putusan.
POST	`/editor/reviewer-verifications/:id/decision`	JWT	Persetujuan atau penolakan bidang keahlian penelaah.
GET	`/editor/manuscripts`	JWT	Daftar naskah yang menunggu penyaringan awal.
POST	`/editor/manuscripts/:id/review`	JWT	Penyaringan awal naskah oleh editor.

Berdasarkan Tabel IV.6, terdapat tiga jenis penjagaan akses. Sebanyak sebelas *endpoint* dilindungi *middleware* JWT, sebelas *endpoint* bersifat publik karena melayani pembacaan data terbuka maupun proses autentikasi itu sendiri, dan satu *endpoint* internal dijaga oleh rahasia bersama melalui *header X-Oracle-Secret* karena hanya dipanggil oleh program *oracle* dan tidak pernah diakses dari peramban pengguna. Perlu dicatat bahwa dua *endpoint* yang bersifat mengubah data, yaitu pengunggahan berkas ke IPFS dan pengiriman ulasan, saat ini belum dilindungi JWT. Keduanya tetap aman pada lapisan *on-chain* karena kontrak menolak transaksi yang tanda tangannya tidak sah, namun pada lapisan *off-chain* keduanya masih terbuka terhadap penyalahgunaan sumber daya.

Proses autentikasi diawali pada *endpoint POST /auth/login*. Kata sandi pengguna disimpan dalam bentuk *hash* bcrypt dengan *cost* sepuluh dan diverifikasi menggunakan *bcrypt.CompareHashAndPassword()*. Apabila kredensial sesuai, sistem menerbitkan JWT bertanda tangan HS256 yang memuat tiga klaim, yaitu *sub* berisi pengenal pengguna, *email*, dan *exp* yang menetapkan masa berlaku satu jam sejak penerbitan. Penjagaan pada setiap permintaan berikutnya dilakukan oleh *middleware JWTAuth()* sebagaimana ditunjukkan pada Kode IV.9, dengan penanganan galat tidak ditampilkan demi keringkasan.

```
token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (interface{}, error) {
    if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
        return nil, jwt.ErrSignatureInvalid
    }
    return []byte(jwtSecret), nil
})

userID, _ := claims["sub"].(string)
user, err := userRepo.FindByID(userID)
c.Set("user", user)
```

Kode IV.9: Penegakan metode penandatanganan dan pemuatan pengguna pada *middleware* JWT

Pembatasan akses berbasis peran sesuai NFR-04 tidak diimplementasikan sebagai *middleware* tersendiri, melainkan ditulis langsung pada awal setiap *handler* yang memerlukannya. Untuk *endpoint* editor, pemeriksaan tersebut dibungkus dalam fungsi pembantu *requireEditor()* sebagaimana ditunjukkan pada Kode IV.10.

```
func requireEditor(c *gin.Context) (*domain.User, bool) {
```

```
    user := c.MustGet("user").(*domain.User)
    if user.Role != "editor" {
        c.JSON(http.StatusForbidden, gin.H{"message": "editor role required"})
        return nil, false
    }
    return user, true
}
```

Kode IV.10: Pemeriksaan peran editor pada awal *handler*

Objek pengguna yang dibaca pada Kode IV.9 diambil kembali dari konteks permintaan, lalu perannya dicocokkan. Pola yang sama diterapkan pada *endpoint* khusus penelaah dengan membandingkan peran terhadap nilai *reviewer*. Pendekatan ini membuat penjagaan peran menjadi eksplisit pada setiap *handler*, dengan konsekuensi bahwa penambahan *endpoint* baru menuntut ketelitian pengembang untuk tidak melewatkan pemeriksaan tersebut.

Adapun *endpoint POST /auth/logout* tidak menyimpan daftar token yang dicabut. *Endpoint* tersebut hanya mengembalikan pesan keberhasilan, sedangkan penghapusan token yang sesungguhnya dilakukan oleh *frontend*. Dengan demikian, token yang telanjur bocor tetap sah sampai masa berlakunya yang satu jam berakhir.

IV.4.2 Implementasi *EthereumService*

EthereumService berperan sebagai *relayer*, yaitu komponen yang mengubah permintaan HTTP menjadi transaksi *on-chain* yang ditandatangani dan dibiayai oleh dompet operator. Kunci privat operator dimuat satu kali ketika aplikasi dijalankan dari variabel lingkungan *OPERATOR_PRIVATE_KEY* lalu disimpan di memori proses. Karena seluruh transaksi dikirim dari dompet tersebut, biaya *gas* sepenuhnya ditanggung operator, sedangkan pengguna akhir tidak perlu memiliki saldo *native token* maupun memasang dompet kripto pada peramban. Inilah yang memungkinkan pola *meta-transaction* pada Subbab IV.3.1.3 bekerja dari sisi *off-chain*.

Terdapat empat fungsi *PublicationRegistry* yang dapat dipanggil melalui komponen ini, dan keempatnya terbagi menjadi dua kelompok yang berbeda secara mendasar. Kelompok pertama, yaitu *submitManuscript()* dan *submitReview()*, membawa tanda tangan pengguna sehingga identitas penulis maupun penelaah dipulihkan langsung oleh kontrak dari tanda tangan tersebut, bukan dari alamat pengirim transaksi. Kelompok kedua, yaitu *submitEditorReview()* dan *verifyReviewerFields()*, dipanggil tanpa tanda tangan pengguna dan hanya

mengandalkan kewenangan *OPERATOR_ROLE* milik dompet operator. Konsekuensinya, keabsahan wewenang editor pada kedua fungsi tersebut sepenuhnya ditegakkan oleh pemeriksaan peran di *backend* sebagaimana ditunjukkan pada Kode IV.10, sedangkan kontrak hanya memastikan bahwa pemanggilnya adalah operator yang sah.

Tanda tangan yang dikirimkan *frontend* berbentuk untai heksadesimal sepanjang 65 bita, sedangkan fungsi kontrak menuntut tiga parameter terpisah *v*, *r*, dan *s*. Pemecahan tersebut dilakukan oleh fungsi *splitSig()* sebagaimana ditunjukkan pada Kode IV.11.

```
b := common.FromHex(hexSig)
if len(b) != 65 {
    return 0, [32]byte{}, [32]byte{}, fmt.Errorf("invalid signature length")
}
copy(r[:], b[0:32])
copy(s[:], b[32:64])
v = b[64]
if v < 27 {
    v += 27
}
```

Kode IV.11: Pemecahan tanda tangan 65 bita menjadi komponen v, r, dan s

Tiga puluh dua bita pertama dibaca sebagai *r*, tiga puluh dua bita berikutnya sebagai *s*, dan bita terakhir sebagai *v*. Baris terakhir pada Kode IV.11 menormalkan nilai *v* dari 0 atau 1 menjadi 27 atau 28. Penyesuaian ini diperlukan karena sebagian pustaka penandatanganan pada peramban mengeluarkan *v* dalam bentuk 0 atau 1, sementara *ECDSA.recover()* pada Kode IV.3 hanya menerima bentuk 27 atau 28. Tanpa normalisasi tersebut, pemulihan alamat penanda tangan akan menghasilkan alamat yang keliru.

Setelah data transaksi tersusun, komponen ini menjalankan tiga penanganan tambahan agar transaksi lebih tahan terhadap kondisi jaringan yang tidak ideal. Pertama, batas *gas* tidak ditetapkan secara tetap melainkan diperoleh melalui *EstimateGas()* lalu dikalikan 1,2 sebagai cadangan sebesar dua puluh persen. Apabila estimasi tersebut gagal, transaksi tidak jadi dikirim karena kegagalan estimasi hampir selalu menandakan bahwa transaksi akan dibatalkan kontrak. Kedua, pengiriman transaksi dicoba paling banyak empat kali apabila jaringan menolaknya dengan alasan *replacement transaction underpriced*, dengan harga *gas* dinaikkan 1,2 kali pada setiap percobaan. Ketiga, ketika transaksi terlanjur masuk blok namun berstatus gagal, komponen ini mensimulasikan ulang pemanggilan melalui

CallContract() untuk memperoleh muatan galatnya, kemudian mencocokkan empat bita pertama muatan tersebut dengan daftar *custom error* pada ABI. Dengan cara ini, pesan galat yang sampai ke pengguna berupa nama galat yang bermakna beserta argumennya, misalnya *InvalidState(msId=3, expected=2, actual=1)*, bukan sekadar untai heksadesimal.

IV.4.3 Implementasi Mekanisme Anonimitas (*Session Wallet*)

Kebutuhan anonimitas pada FR-08 diwujudkan dengan memastikan bahwa alamat yang tercatat pada *ledger* bukanlah alamat dompet asli pengguna, melainkan alamat dompet sesi yang dibangkitkan khusus untuk keperluan tersebut. Implementasinya membedakan dua jenis dompet sesi dengan perlakuan yang berbeda.

Jenis pertama adalah *submission wallet* milik penulis. Dompet ini dibangkitkan di sisi peramban, sehingga kunci privatnya tidak pernah meninggalkan perangkat penulis. *Backend* hanya menerima alamat publiknya melalui medan *submissionAddress* pada permintaan pengajuan naskah, lalu menyimpannya pada tabel *submission_wallets* bersama pengenal pengguna. Penyimpanan tersebut dilakukan setelah transaksi *on-chain* berhasil dan bersifat *best-effort*, yaitu kegagalan penyimpanan tidak membatalkan pengajuan yang sudah tercatat pada *ledger* melainkan hanya dikembalikan sebagai peringatan.

Jenis kedua adalah *session wallet* milik penelaah. Berbeda dengan dompet penulis, dompet ini dibangkitkan di sisi peladen ketika program *oracle* meminta penugasan penelaah, sebagaimana ditunjukkan pada Kode IV.12.

```
key, err := crypto.GenerateKey()
privHex := "0x" + hex.EncodeToString(crypto.FromECDSA(key))
sessionAddr := crypto.PubkeyToAddress(key.PublicKey).Hex()
```

Kode IV.12: Pembangkitan dompet sesi penelaah pada sisi peladen

Pasangan kunci pada Kode IV.12 disimpan pada tabel *reviewer_sessions* dengan kunci unik gabungan nomor naskah dan alamat asli penelaah, sehingga setiap penelaah memperoleh satu dompet sesi yang berbeda untuk setiap naskah yang ditugaskan kepadanya. Dengan pola ini, dua ulasan dari penelaah yang sama pada dua naskah berbeda tidak dapat dikaitkan satu sama lain hanya dengan membaca data *on-chain*.

Kunci privat dompet sesi penelaah disimpan pada kolom *session_privkey* dalam bentuk teks biasa tanpa enkripsi, dan dikirimkan kepada penelaah yang bersangkutan melalui

endpoint GET /reviewer/assignments. Pilihan ini diambil karena penelaah harus dapat menandatangani ulasannya secara EIP-712 dari peramban menggunakan dompet sesi tersebut, sementara salah satu tujuan rancangan adalah membebaskan pengguna dari keharusan memasang dan mengelola dompet kripto. Kunci tersebut hanya dapat diambil oleh pemilik akun yang bersangkutan karena *endpoint*-nya dilindungi JWT dan kuerinya disaring berdasarkan pengenal pengguna. Meskipun demikian, penyimpanan kunci tanpa enkripsi berarti *backend* secara teknis mampu memalsukan ulasan atas nama penelaah mana pun, dan kebocoran basis data akan langsung membongkar seluruh dompet sesi.

Perlu dicatat pula bahwa anonimitas yang dijamin implementasi ini bersifat *on-chain*, dan perlakuannya sengaja dibuat tidak simetris antara penulis dan penelaah begitu naskah mencapai status akhir (*ACCEPTED*, *REJECTED*, atau *PUBLISHED*). Untuk penulis, pemetaan antara alamat dompet sesi dan identitas asli tetap tersimpan pada basis data *off-chain*, dan sengaja dibuka kembali oleh komponen *IdentityResolver* melalui *ArticleHandler.List()* dan *ArticleHandler.OpenReview()* begitu status akhir tercapai, sebagai wujud transparansi pascapublikasi pada NFR-01. Untuk penelaah, identitas asli tidak pernah dibuka pada *endpoint* publik mana pun, bahkan setelah naskah mencapai status akhir sekalipun. Kedua *handler* tersebut selalu mengembalikan alamat dompet sesi penelaah apa adanya, terlepas dari status naskah. Ketidaksimetrisan ini diambil karena penelaah yang mengetahui identitasnya akan tetap dibuka pada akhirnya berpotensi menyesuaikan putusannya secara tidak jujur selama proses ulasan berlangsung, misalnya menjadi segan mengkritik naskah dari penulis yang lebih senior karena mengantisipasi konsekuensi sosial pascapublikasi, persoalan yang telah diidentifikasi pada Subbab III.1.1.4. Dengan demikian, identitas penulis dapat ditelusuri melalui *endpoint* publik setelah keputusan akhir ditetapkan, sedangkan identitas penelaah tidak dapat ditelusuri kapan pun, sesuai FR-08.

IV.4.4 Implementasi Modul Pemilihan *Reviewer*

Penelaah yang berhak ditugaskan pada suatu naskah ditentukan melalui kueri pada Kode IV.13, yang dijalankan *backend* setelah menerima permintaan dari *Reviewer Selector* pada program *oracle* (Subbab IV.5.3).

```
1 SELECT id, wallet_address FROM users
2 WHERE role = 'reviewer'
3   AND $1 = ANY(verified_fields)
4   AND wallet_address IS NOT NULL AND wallet_address <> ''
```

```
5 ORDER BY random()
6 LIMIT $2
```

Kode IV.13: Kueri pemilihan penelaah berdasarkan bidang kajian yang telah diverifikasi

Kueri pada Kode IV.13 menerapkan tiga penyaringan sekaligus, yaitu peran pengguna harus penelaah, bidang kajian naskah harus termasuk dalam daftar bidang yang telah diverifikasi editor sesuai FR-09, dan penelaah harus telah mengikat alamat dompet aslinya. Urutan hasil diacak melalui *ORDER BY random()* dan dibatasi sebanyak jumlah penelaah yang diminta, yaitu tiga orang secara bawaan. Apabila jumlah penelaah yang memenuhi syarat kurang dari tiga, *backend* mengembalikan galat HTTP 409 dengan pesan yang jelas alih-alih membiarkan transaksi dibatalkan kontrak oleh penjagaan pada Kode IV.7. Perlu dinyatakan bahwa keacakan pemilihan penelaah ini berlangsung di dalam basis data *backend* dan bukan di atas rantai, sehingga sifat acaknya masih bergantung pada kejujuran penyelenggara sistem.

Setiap penelaah yang terpilih melalui kueri ini kemudian dibangkitkan dompet sesinya sebagaimana telah dijelaskan pada Subbab IV.4.3.

IV.4.5 Implementasi *PinataService*

PinataService merupakan komponen terkecil pada *backend* dan hanya menyediakan satu fungsi, yaitu *UploadFile()*. Fungsi tersebut menyusun permintaan *multipart* dengan satu medan bernama *file*, menyertakan *token* Pinata pada *header Authorization*, lalu mengirimkannya ke *endpoint pinFileToIPFS* milik Pinata. Nilai *IpfsHash* pada balasan diambil sebagai CID berkas.

Terdapat empat jenis data yang diunggah melalui komponen ini. Pertama, berkas naskah asli yang diunggah penulis pada tahap pertama alur pengajuan. Kedua, isi ulasan penelaah yang terlebih dahulu dibungkus menjadi dokumen JSON berisi komentar dan putusan. Ketiga, dokumen keputusan penyaringan awal editor yang juga berbentuk JSON. Keempat, teks komentar pembaca pada naskah yang telah terbit. Pada keempat kasus tersebut, CID yang dikembalikan Pinata itulah yang kemudian dicatatkan pada *ledger*, baik sebagai atribut naskah, sebagai *reviewCid* pada ulasan, maupun sebagai *editorCid* pada keputusan editor. Dengan pola ini, kebutuhan FR-07 terpenuhi karena *blockchain* cukup menyimpan referensi berupa CID, sedangkan berkas yang berukuran besar berada pada jaringan penyimpanan terdistribusi.

Keputusan untuk turut menyimpan isi ulasan penelaah, bukan hanya putusan *ACCEPT*/*REJECT*/*REVISE*-nya, berangkat dari kebutuhan yang telah dirumuskan pada Subbab III.2, yaitu bahwa keputusan editorial harus dapat dipertanggungjawabkan dan bukan sekadar hasil akhir yang tidak dapat ditelusuri alasannya. Putusan tanpa komentar hanya memberi tahu apa yang diputuskan, tetapi tidak memberi tahu mengapa. Penulis yang menerima putusan *REVISE* tanpa penjelasan tidak akan tahu bagian mana dari naskahnya yang perlu diperbaiki, dan pihak luar yang membaca riwayat naskah tidak dapat menilai apakah putusan tersebut beralasan atau sewenang-wenang. Komentar tertulis penelaah mengisi kekosongan tersebut dengan memuat alasan dan rincian yang menjadi dasar putusan, sehingga penulis memperoleh panduan konkret untuk revisi dan pihak lain memperoleh dasar untuk menilai kualitas proses *review* itu sendiri.

Komentar tersebut disimpan *off-chain* sebagai dokumen JSON pada IPFS, bukan sebagai teks langsung pada *ledger*, karena panjang teksnya tidak dapat diperkirakan sebelumnya dan berpotensi jauh melebihi ukuran yang wajar untuk disimpan *on-chain*, sebagaimana pertimbangan biaya dan ukuran data pada Subbab IV.1.1. Meskipun demikian, integritasnya tetap dapat diverifikasi melalui mekanisme CID yang sama seperti berkas naskah: CID komentar dicatat sebagai *reviewCid* pada *struct* ulasan di kontrak *PublicationRegistry*, sehingga isi komentar yang diunduh ulang dari IPFS dapat dicocokkan *hash*-nya terhadap CID tersebut untuk memastikan komentar yang dibaca memang komentar yang sama dengan yang tercatat saat ulasan pertama kali dikirimkan.

IV.4.6 Implementasi Manajemen Jurnal

Katalog jurnal disimpan pada dua tabel *off-chain*, yaitu *categories* yang berisi lima bidang kajian yang sama dengan yang digunakan pada verifikasi bidang keahlian penelaah, dan *journals* yang memuat nama jurnal beserta kategori bidangnya, dengan beberapa jurnal dapat tergabung dalam satu kategori yang sama. Demi mempermudah implementasi pada tugas akhir ini, daftar jurnal pada tabel *journals* ditanamkan langsung (*hardcoded*) pada basis data, bukan dikelola melalui antarmuka administratif tersendiri. *Endpoint* publik *GET /journals* mengembalikan seluruh isi katalog ini untuk mengisi pilihan jurnal pada *frontend* saat penulis mengajukan naskah.

Jurnal yang dipilih penulis disertakan sebagai bagian dari data metadata naskah yang ditandatangani secara EIP-712, berdampingan dengan judul dan abstrak, dalam bentuk objek JSON berisi pengenal, nama, dan kategori jurnal. Begitu *event ManuscriptSubmitted*

diproses, *backend* membaca kembali metadata tersebut dari kontrak, mem-*parsing* pengenal jurnal yang tersimpan di dalamnya, lalu menyimpannya pada kolom *journal_id* milik naskah. Penugasan editor kepada naskah dilakukan secara otomatis pada tahap yang sama melalui kueri pada Kode IV.14.

```
1 UPDATE manuscripts m SET assigned_editor_id = COALESCE(
2   (SELECT u.id FROM users u
3      JOIN journals j ON j.id = m.journal_id
4     WHERE u.role = 'editor' AND j.category_slug = ANY(u.verified_fields)
5     ORDER BY random() LIMIT 1),
6   (SELECT u.id FROM users u WHERE u.role = 'editor' ORDER BY random() LIMIT
        1)
7 ), updated_at = NOW()
8 WHERE m.ms_id = $1 AND m.assigned_editor_id IS NULL
```

Kode IV.14: Penugasan editor otomatis berdasarkan kategori jurnal

Kueri pada Kode IV.14 lebih dahulu mencoba memilih editor secara acak di antara editor yang bidang keahliannya, sebagaimana tercatat pada kolom *verified_fields* yang sama dengan yang digunakan pemilihan penelaah pada Kode IV.13, mencakup kategori jurnal naskah tersebut. Apabila tidak ditemukan kecocokan, editor dipilih secara acak dari seluruh editor yang terdaftar, sehingga naskah tidak pernah dibiarkan tanpa penanggung jawab meskipun tidak ada editor yang sesuai kategorinya. Klausa *WHERE m.assigned_editor_id IS NULL* membuat kueri ini hanya mengisi penugasan yang masih kosong, sehingga pemrosesan ulang *event* yang sama tidak pernah menggantikan editor yang telah ditugaskan pada naskah yang sedang berjalan.

Penugasan ini ditegakkan pada dua *endpoint* editor. Daftar naskah yang menunggu penyaringan (*GET /editor/manuscripts*) disaring agar editor hanya melihat naskah yang ditugaskan kepadanya atau naskah yang belum memiliki penugasan sama sekali. Adapun *endpoint* pengambilan keputusan penyaringan (*POST /editor/manuscripts/:id/review*) terlebih dahulu memeriksa kolom *assigned_editor_id* naskah tersebut, dan mengembalikan galat HTTP 403 apabila naskah telah ditugaskan kepada editor lain.

Bidang kajian yang dipilih penulis melalui jurnal hanya berfungsi sebagai nilai bawaan pada formulir penyaringan editor di *frontend*. Nilai yang benar-benar dikirimkan ke fungsi *submitEditorReview()* tetap berupa pilihan editor sendiri, yang dapat berbeda dari kategori jurnal apabila editor menilai naskah lebih sesuai untuk bidang kajian lain.

IV.5 Implementasi *Oracle*

Program *oracle* diimplementasikan sebagai proses Node.js tersendiri yang terpisah dari *backend*, menggunakan pustaka *ethers.js* versi 6 untuk berinteraksi dengan jaringan *blockchain*. Sesuai perannya pada Subbab III.4.2, komponen ini menjembatani *smart contract ReviewOracle* dengan dua sumber data eksternal, yaitu layanan pemeriksaan plagiarisme dan basis data profil penelaah milik *backend*. Program berjalan sebagai satu fungsi *main()* pada berkas *src/index.js* yang membangun koneksi jaringan, memuat dompet operator, lalu mendelegasikan setiap permintaan yang masuk kepada salah satu dari dua penanganan (*handler*): *Plagiarism Handler* dan *Reviewer Selector*.

IV.5.1 *Event Listener*

Koneksi jaringan dibangun oleh fungsi *createProvider()*, yang memilih antara langganan *WebSocket* (*SEPOLIA_RPC_WSS*) atau HTTP dengan penjajakan berkala apabila variabel tersebut tidak diatur. Berkas ini tidak menyertakan mekanisme pencobaan ulang tersendiri ketika koneksi terputus, sehingga ketahanan koneksi sepenuhnya bergantung pada penanganan bawaan pustaka *ethers.js*.

Dompet operator dibungkus dengan *ethers.NonceManager* agar nomor urut transaksi (*nonce*) dikelola otomatis di sisi klien, alih-alih meminta ulang ke jaringan pada setiap transaksi. Kode IV.15 menunjukkan pola berlangganan terhadap dua *event* yang dipancarkan *ReviewOracle*.

```
contract.on("PlagiarismCheckRequested", async (requestId, msId, cid) => {
  await wallet.reset();
  handlePlagiarismRequest(contract, requestId, msId, cid)
    .catch(console.error);
});
contract.on("ReviewerSelectionRequested", async (requestId, msId) => {
  await wallet.reset();
  handleReviewerSelection(contract, requestId, msId, field)
    .catch(console.error);
});
```

Kode IV.15: Berlangganan event ReviewOracle dan penyetelan ulang nonce

Sebelum setiap permintaan diteruskan ke penanganannya, *wallet.reset()* dipanggil untuk mengosongkan nonce yang dipatok *NonceManager*, sehingga nilai nonce berikutnya selalu

dibaca ulang dari jaringan. Pemanggilan ini mencegah *oracle* mengirim transaksi dengan nonce yang sudah kedaluwarsa apabila terdapat transaksi lain yang terkirim dari dompet operator di luar sepengetahuan *NonceManager*, misalnya akibat proses yang sempat dimulai ulang.

Khusus untuk permintaan pemilihan penelaah, *index.js* turut membaca bidang kajian naskah langsung dari *PublicationRegistry* melalui pemanggilan *getField()*, menggunakan potongan ABI baca-saja yang hanya berisi satu fungsi tersebut. Bidang kajian ini diperlukan agar *backend* dapat mencocokkan penelaah yang sesuai keahliannya, sebagaimana dijelaskan pada Subbab IV.4.4. Apabila pembacaan tersebut gagal atau alamat kontrak registri belum dikonfigurasi, bidang kajian diteruskan sebagai untai kosong dan permintaan tetap dineruskan ke *Reviewer Selector*, yang kemudian menolaknya dengan pesan galat yang jelas.

Sebagai pemeriksaan tambahan, program mencetak saldo Ether dompet operator saat dimulai dan menampilkan peringatan apabila saldonya nol, namun program tetap berjalan karena kegagalan pengiriman transaksi baru akan terjadi ketika permintaan pertama diproses. Program juga menangani sinyal *SIGINT* dan *SIGTERM* untuk berhenti secara rapi dengan melepas seluruh langganan *event* sebelum keluar.

IV.5.2 *Plagiarism Handler*

Setiap permintaan *PlagiarismCheckRequested* akan ditangani oleh fungsi *handle PlagiarismRequest()*, yang terlebih dahulu melakukan dua pemeriksaan deduplikasi sebelum memproses permintaan, sebagaimana ditunjukkan pada Kode IV.16.

```
const reqKey = requestId.toString();
if (inflight.has(reqKey)) {
  return;
}
const fulfilled = await contract.plagiarismRequestFulfilled(requestId);
if (fulfilled) {
  return;
}
inflight.add(reqKey);
```

Kode IV.16: Deduplikasi permintaan berdasarkan status di memori dan status on-chain

Pemeriksaan pertama menggunakan himpunan (*Set*) di memori untuk menandai permintaan

yang sedang diproses, mencegah pemrosesan ganda apabila *event* yang sama diterima lebih dari sekali dalam satu masa hidup proses. Pemeriksaan kedua membaca status pemenuhan langsung dari kontrak *ReviewOracle* melalui *plagiarismRequestFulfilled()*, sehingga permintaan yang sudah terpenuhi pada transaksi sebelumnya, misalnya karena proses *oracle* sempat dihentikan lalu dijalankan ulang, tidak diproses kembali. Pemanggilan tersebut dibungkus *try-catch* karena kegagalan pemanggilan *view* tidak dianggap fatal, sehingga permintaan tetap diproses seolah belum terpenuhi. Penanda *inflight* dihapus kembali pada blok *finally* setelah permintaan selesai diproses, baik berhasil maupun gagal.

Skor plagiarisme sendiri diperoleh melalui fungsi *checkPlagiarism()*, yang meneruskan CID naskah ke layanan eksternal pada *PLAGIARISM_API_URL* apabila variabel tersebut dikonfigurasi. Balasan API divalidasi agar bertipe angka dalam rentang 0 hingga 100 sebelum dibulatkan.

Setelah skor diperoleh, hasilnya kemudian dikirim ke kontrak melalui pemanggilan *fulfill PlagiarismCheck(requestId, score)*. Program mencatat *hash* transaksi segera setelah terkirim, lalu menunggu konfirmasi dan mencatat nomor blok beserta jumlah *gas* yang terpakai.

IV.5.3 *Reviewer Selector*

Penanganan permintaan *ReviewerSelectionRequested* mengikuti pola deduplikasi yang sama seperti Kode IV.16, hanya saja memeriksa *reviewerRequestFulfilled()* sebagai gantinya. Apabila bidang kajian naskah yang diteruskan *index.js* berupa untai kosong, yang menandakan editor belum menetapkan bidang kajian pada naskah tersebut, penanganan dihentikan lebih awal dengan pesan galat yang jelas, alih-alih meneruskan permintaan kosong ke *backend*.

Pemilihan penelaah yang sesungguhnya sepenuhnya didelegasikan kepada *backend*. *Reviewer Selector* hanya mengirimkan bidang kajian naskah ke *endpoint* internal *backend*, sebagaimana ditunjukkan pada Kode IV.17.

```
1 const res = await fetch(
2   `${BACKEND_URL}/api/v1/internal/reviewer-sessions`, {
3   method: "POST",
4   headers: { "Content-Type": "application/json",
5              "X-Oracle-Secret": ORACLE_SECRET },
6   body: JSON.stringify({ msId: Number(msId), field }),
```

```
});
```

Kode IV.17: Permintaan dompet sesi penelaah ke endpoint internal backend

Permintaan tersebut dilindungi kunci bersama pada tajuk *X-Oracle-Secret*, bukan mekanisme otentikasi berbasis pengguna, karena pemanggilnya adalah proses *oracle* itu sendiri dan bukan klien akhir. *Backend* kemudian menjalankan kueri pemilihan penelaah yang telah dijelaskan pada Kode IV.13 (Subbab IV.4.4), membangkitkan dompet sesi bagi setiap penelaah terpilih, lalu mengembalikan daftar alamatnya. Apabila *backend* membalas dengan status bukan 2xx, fungsi ini melempar galat yang menghentikan seluruh penanganan permintaan.

Perlu dicatat bahwa berkas *src/reviewer.js* dan *src/registry.js* pada kode sumber *oracle*, yang masing-masing berisi algoritma seleksi berjenjang dan daftar sepuluh penelaah tetap, tidak pernah dipanggil oleh *index.js* maupun *handlers/reviewer.js*. Kedua berkas tersebut tidak memengaruhi perilaku sistem yang berjalan karena pemilihan penelaah sepenuhnya ditentukan oleh kueri *backend* pada Kode IV.13.

Daftar alamat dompet sesi yang diterima dari *backend* kemudian dikirimkan ke kontrak melalui *fulfillReviewerSelection(requestId, addresses)*, dengan pola pencatatan *hash* transaksi dan konfirmasi yang sama seperti pada *Plagiarism Handler*.

BAB V

PENGUJIAN DAN EVALUASI

Bab ini menyajikan pengujian terhadap hasil implementasi pada Bab IV, dilanjutkan dengan evaluasi untuk menilai sejauh mana sistem yang dibangun memenuhi kebutuhan pada Subbab III.2 dan memanfaatkan sifat dasar *blockchain* yang dianalisis pada Subbab III.1.

V.1 Pengujian Implementasi

Pengujian terhadap implementasi dilakukan pada dua lapisan utama, yaitu lapisan *on-chain* berupa keempat *smart contract* dan lapisan *off-chain* berupa *backend*. Pengujian meliputi pengujian unit, pengujian terintegrasi *end-to-end*, serta pengujian terhadap basis data.

V.1.1 Strategi dan Lingkup Pengujian

Strategi pengujian yang digunakan adalah metode uji kasus (*test case*). Pengujian dilakukan pada dua tingkatan. Pertama, pengujian unit yang memverifikasi kebenaran perilaku setiap fungsi *smart contract* secara terisolasi, mencakup skenario keberhasilan maupun skenario kegagalan yang diharapkan. Kedua, pengujian terintegrasi *end-to-end* yang mensimulasikan aktor sesungguhnya, yaitu peneliti, editor, dan penelaah, melalui keseluruhan alur publikasi menggunakan seluruh kontrak yang saling berinteraksi tanpa *mock*.

Pemilihan kasus uji pada bab ini mengikuti rantai Kebutuhan $\rightarrow$ Solusi $\rightarrow$ Implementasi yang telah dibangun pada Bab III dan Bab IV, bukan disusun secara terpisah dari keduanya. Setiap keputusan desain pada Bab III lahir untuk menjawab kebutuhan atau masalah tertentu pada Subbab III.2, direalisasikan sebagai mekanisme konkret pada Bab IV, dan mekanisme itulah yang kemudian diuji. Dengan kata lain, suatu fungsi kontrak tidak diuji semata-mata karena fungsi tersebut ada, melainkan karena fungsi tersebut merupakan wujud dari solusi yang harus terbukti bekerja sebagaimana dirancang. Hubungan ini dirangkum secara eksplisit pada Tabel V.4 (Subbab V.1.5).

Secara umum, skenario pengujian kontrak meliputi empat kategori berikut, masing-masing menyasar aspek berbeda dari solusi yang telah dirancang.

1. **Skenario keberhasilan:** pemanggilan fungsi dengan argumen valid pada keadaan kontrak yang tepat, kemudian diverifikasi bahwa *event* yang benar dipancarkan dan status berubah sesuai rancangan. Kategori ini membuktikan bahwa mekanisme inti sistem, yaitu pencatatan naskah, keputusan mayoritas, distribusi insentif, dan sejenisnya, benar-benar berjalan sesuai rancangan pada jalur normal, sehingga menjadi dasar pembuktian bagi hampir seluruh kebutuhan fungsional pada Subbab III.2.

2. **Skenario kesalahan otorisasi:** pemanggilan fungsi oleh alamat yang tidak berwenang untuk memastikan kontrak menolak operasi tersebut. Kategori ini penting karena rancangan pada Subbab III.5.2 menyandarkan keamanan sistem pada kontrol akses berbasis peran (*onlyRole*) sebagai satu-satunya penjaga terhadap penyalahgunaan fungsi yang mengubah status. Apabila penjagaan ini gagal, siapa pun berpotensi memalsukan keputusan editor maupun mencetak insentif tanpa hak, sehingga kategori ini secara langsung membuktikan pemenuhan NFR-04.

3. **Skenario kesalahan validasi:** pemanggilan fungsi dengan argumen atau status yang tidak sesuai untuk memastikan kesalahan validasi yang tepat dikembalikan. Kategori ini memastikan mesin status yang dirancang pada Subbab III.5.2.1 tidak dapat dilompati atau dipaksa masuk ke keadaan yang tidak valid, misalnya mengirim ulasan pada naskah yang belum berstatus *UNDER_REVIEW*, sehingga integritas alur kerja publikasi pada FR-05 tetap terjaga sekalipun dipanggil dengan urutan yang keliru.

4. **Skenario duplikasi:** pemanggilan fungsi secara berulang dengan argumen yang sama untuk memastikan kontrak menolak penulisan ulang data yang telah tercatat. Kategori ini relevan karena beberapa transisi status pada rancangan bersifat satu kali dan tidak dapat diulang, seperti pengiriman ulasan oleh penelaah yang sama atau pencetakan DOI untuk naskah yang sama. Tanpa penjagaan ini, catatan pada *ledger* dapat digandakan sehingga jejak audit yang menjadi wujud NFR-01 dan NFR-02 kehilangan keandalannya.

Pengujian ini secara eksplisit tidak mencakup pengujian beban tinggi (*load testing*) maupun audit kerentanan keamanan. Batasan ini dipilih karena kedua jenis pengujian tersebut memerlukan keahlian dan perangkat khusus di luar fokus tugas akhir ini, yaitu pembuktian bahwa mekanisme fungsional yang dirancang bekerja sebagaimana mestinya. Akibatnya,

hasil pengujian pada bab ini membuktikan kebenaran perilaku (*correctness*) sistem pada skenario yang diuji, tetapi tidak dapat dijadikan dasar untuk menyimpulkan ketahanan sistem terhadap serangan yang disengaja maupun performa sistem di bawah beban pengguna yang tinggi.

V.1.2 Ringkasan Hasil Pengujian

Pengujian fungsional dilakukan pada komponen *smart contract* menggunakan Hardhat serta pada *backend* menggunakan pustaka pengujian bawaan Go. Pada lapisan *on-chain*, pengujian mencakup 88 kasus uji yang dijalankan pada jaringan lokal Hardhat. Pada lapisan *off-chain*, *backend* diuji melalui pengujian terintegrasi terhadap basis data PostgreSQL yang sesungguhnya. Rekapitulasi eksekusi pengujian dirangkum pada Tabel V.1.

Tabel V.1: Rekapitulasi eksekusi pengujian

Komponen	**Jenis Pengujian**	**Kasus Uji**	**Status**
Smart contract	Pengujian unit dan kontrol akses	57	Lulus
Smart contract	Pengujian terintegrasi *end-to-end*	31	Lulus
Backend	Pengujian terintegrasi basis data	3	Lulus
	Total	**91**	**Lulus**

Secara keseluruhan, terdapat 91 kasus uji yang seluruhnya lulus. Hasil ini menunjukkan bahwa mekanisme yang diimplementasikan berperilaku sesuai rancangan pada seluruh skenario yang diuji, baik pada lapisan kontrak maupun pada lapisan *backend*.

Tabel V.2 menampilkan satu kasus uji nyata dari setiap baris pada Tabel V.1 sebagai gambaran konkret bentuk pengujian yang dijalankan.

Tabel V.2: Contoh kasus uji dari setiap komponen pengujian

Komponen	**Kasus Uji**	**Yang Diverifikasi**
Smart contract	*should reject a replayed nonce (anti-replay protection)* *(unit dan kontrol akses)*	Tanda tangan EIP-712 yang sama digunakan untuk mengajukan naskah dua kali. Percobaan kedua harus ditolak dengan galat *InvalidSignature* karena *nonce* milik penanda tangan sudah tidak sesuai.
Smart contract	*Step 3: All 3 reviewers sign and submit ACCEPT verdicts → ACCEPTED* *(terintegrasi end-to-end)*	Ketiga penelaah yang ditugaskan mengirimkan putusan *ACCEPT* secara berurutan melalui alur penuh pengajuan tanda tangan hingga transaksi *on-chain*. Status naskah harus otomatis berubah menjadi *ACCEPTED* tanpa campur tangan editor, begitu suara ketiga tercatat.
Backend	*TestUserRepository WalletBinding* *(terintegrasi basis data)*	Alamat dompet yang sudah terikat pada satu akun dicoba diikatkan ulang. *Repository* harus dapat menemukan pemilik akun tersebut melalui pencarian berdasarkan alamat dompet setelah pengikatan, membuktikan uniknya pemetaan dompet-ke-akun.

V.1.3 Pengujian Lapisan *On-Chain*

Pengujian lapisan *on-chain* dilakukan menggunakan kerangka pengujian Hardhat dengan pustaka asersi Chai. Pengujian terbagi ke dalam dua berkas dengan fokus yang berbeda. Berkas pertama memuat pengujian unit terhadap fungsi-fungsi utama dan kontrol akses, dan berkas kedua memuat pengujian alur publikasi *end-to-end* menggunakan seluruh

kontrak sesungguhnya.

Tabel V.3 merangkum fokus utama kasus uji pada setiap kelompok pengujian.

Tabel V.3: Ringkasan cakupan pengujian lapisan *on-chain*

Kelompok Pengujian	**Aspek yang Diuji**
Deployment	Ketepatan suplai awal *JournalToken*, pemberian *MINTER_ROLE* kepada *PublicationRegistry*, *deployment proxy* UUPS, dan ketepatan peran yang diberikan pada saat inisialisasi.
Pengajuan Naskah	Penolakan pengajuan oleh alamat tanpa *OPERATOR_ROLE*.
Siklus Hidup Penuh	Alur lengkap dari pengajuan hingga penerbitan beserta pembayaran biaya publikasi, penolakan pembayaran tanpa *allowance* yang cukup, dan penolakan pembayaran oleh pihak selain penulis.
Mesin Status	Transisi status naskah dari *CHECKING* menuju *PENDING_EDITOR*, *UNDER_REVIEW*, hingga *ACCEPTED*, *REJECTED*, atau *REVISION_REQUESTED*, ambang batas plagiarisme pada nilai batas, keputusan mayoritas untuk tiga dan lima penelaah, cabang konservatif ketika tidak ada mayoritas mutlak, pencegahan ulasan ganda, penolakan penelaah yang tidak ditugaskan, serta penolakan transisi status yang tidak sah.
Keamanan Tanda Tangan dan *Nonce*	Pemulihan penanda tangan yang sesungguhnya dari tanda tangan EIP-712, penolakan penggunaan ulang *nonce*, ketiadaan atribusi terhadap penanda tangan asli ketika tanda tangan atau muatan dimodifikasi maupun ketika *domain* berbeda, penolakan revisi oleh pihak selain penulis, serta kebenaran *domain separator*.

Tabel V.3: Ringkasan cakupan pengujian lapisan *on-chain*

Kelompok Pengujian	**Aspek yang Diuji**
DOIToken	Penolakan pencetakan oleh pihak tanpa *MINTER_ROLE* maupun ke alamat nol, ketepatan penyimpanan *tokenURI*, pemberian komentar pada DOI yang ada, serta penolakan komentar pada DOI yang tidak ada.
JournalToken	Pembatasan pencetakan hanya untuk pemegang *MINTER_ROLE*, pencetakan tambahan oleh administrator, serta penolakan pencetakan bernilai nol maupun ke alamat nol.
Kontrol Akses	Penolakan pembaruan *proxy* UUPS oleh pihak tidak berwenang, serta penolakan pemanggilan fungsi oleh alamat tanpa *ORACLE_ROLE*, *OPERATOR_ROLE*, *ADMIN_ROLE*, maupun *REGISTRY_ROLE*.
Upgradeability	Keberhasilan pembaruan *proxy* UUPS oleh pihak berwenang beserta terjaganya data naskah setelah pembaruan.
Alur *End-to-End*	Enam skenario alur penuh, yaitu jalur normal hingga terbit, penolakan akibat plagiarisme, revisi hingga terbit, *guard* pemilihan penelaah, penolakan oleh editor, serta penolakan oleh mayoritas penelaah.

V.1.4 Pengujian Lapisan *Backend*

Pengujian pada lapisan *backend* dilakukan menggunakan pustaka pengujian bawaan Go terhadap basis data PostgreSQL. Setiap pengujian membuat basis data sementara, menjalankan seluruh migrasi skema, lalu menghapus basis data tersebut setelah pengujian selesai, sehingga pengujian tidak saling memengaruhi maupun mengganggu data pengembangan.

Cakupan pengujian pada lapisan ini meliputi keberhasilan eksekusi seluruh migrasi skema beserta ketersediaan seluruh tabel yang dibutuhkan, serta operasi baca dan tulis pada

repository pengguna, termasuk pencarian berdasarkan surel dan alamat dompet serta keunikan pemetaan dompet-ke-akun setelah pengikatan.

Pemilihan aspek yang diuji pada lapisan ini menyasar bagian basis data yang masih ditulis dan dibaca langsung oleh *REST API*, yaitu data akun pengguna dan pemetaan dompet, sejalan dengan rancangan pada Subbab III.5.1 yang tidak lagi menyimpan salinan data *on-chain* pada basis data ini. Pengujian skema memastikan seluruh tabel yang dibutuhkan komponen di luar data *on-chain*, yaitu pengguna, dompet sesi, permintaan verifikasi bidang keahlian, dan isi komentar, benar-benar terbentuk setelah migrasi dijalankan pada basis data kosong, sedangkan pengujian *repository* memastikan kueri pencarian dan batasan keunikan pada tabel tersebut berperilaku benar terhadap basis data PostgreSQL yang sesungguhnya, bukan terhadap tiruan.

V.1.5 *Traceability* Kebutuhan Fungsional dan Non-Fungsional

Tabel V.4 memetakan setiap kebutuhan pada Subbab III.2 ke metode pembuktian dan status pemenuhannya, sekaligus menjadi rangkuman akhir dari rantai kebutuhan → solusi → implementasi → pengujian yang mendasari Subbab V.1.1: kolom **kode** merujuk pada kebutuhan di Subbab III.2, keputusan desain yang menjawabnya dapat ditelusuri pada Subbab III.5, unit implementasinya pada Bab IV, dan kolom **metode pembuktian** menunjukkan pengujian atau bukti nyata yang memvalidasinya. Status **sebagian** menandakan mekanisme utama sudah berjalan namun ditemukan penyimpangan pada implementasinya. Rinciannya dibahas pada subbab evaluasi yang dirujuk.

Tabel V.4: *Traceability* kebutuhan fungsional dan non-fungsional

Kode	Metode Pembuktian	Subbab Rujukan	Status
FR-01	Pengujian *smart contract*, naskah nyata tercatat pada Sepolia	V.1.3	Terpenuhi
FR-02	Pengujian ambang plagiarisme dan penyaringan editor	V.1.3	Terpenuhi
FR-03	Pengujian terintegrasi *end-to-end* dan *event* penunjukan penelaah nyata teramati *on-chain*	V.1.3	Terpenuhi

Tabel V.4: *Traceability* kebutuhan fungsional dan non-fungsional

Kode	Metode Pembuktian	Subbab Rujukan	Status
FR-04	Pengujian pencatatan ulasan dan ulasan nyata teramati pada *ledger*	V.1.3	Terpenuhi
FR-05	Pengujian mesin status dan keputusan mayoritas nyata teramati *on-chain*	V.1.3, V.2.1.3	Terpenuhi
FR-06	Insentif nyata teramati terdistribusi atomik bersama pencetakan DOI	V.2.1.3	Terpenuhi
FR-07	Inspeksi implementasi *PinataService* dan referensi CID *on-chain*	IV.4.5	Terpenuhi
FR-08	Inspeksi data *on-chain* dan *endpoint* publik	V.2.1.4	Terpenuhi
FR-09	Pengujian alur verifikasi bidang keahlian penelaah	V.1.3	Terpenuhi
FR-10	Pengujian *postComment*	V.1.3	Terpenuhi
NFR-01	Penelusuran 28 *event* nyata untuk satu naskah dari pengajuan hingga terbit	V.2.1.3	Terpenuhi
NFR-02	Inspeksi kode kontrak, tidak ada fungsi penghapusan atau pengubahan data historis	V.2.1.3	Terpenuhi
NFR-03	Inspeksi arsitektur penyimpanan	V.2.2.2	Sebagian
NFR-04	Pengujian kontrol akses pada seluruh peran kontrak dan *backend*	V.1.3	Terpenuhi

V.2 Evaluasi

Subbab ini mengevaluasi sistem yang telah dirancang pada Bab III dan diimplementasikan sebagai realisasi *framework* pada Bab IV. Evaluasi diarahkan untuk menjawab apakah sistem yang diusulkan memenuhi kebutuhan yang ditetapkan pada Subbab III.2 serta memanfaatkan sifat dasar *blockchain* sebagaimana dianalisis pada Subbab III.1. Evaluasi terdiri atas dua bagian, yaitu evaluasi kualitatif serta pembahasan kelebihan dan keterbatasan.

V.2.1 Evaluasi Kualitatif

Evaluasi kualitatif berfokus pada perilaku sistem pada tingkat proses, data, dan peran, serta keselarasan antara implementasi dan kebutuhan yang telah dianalisis.

V.2.1.1 Pemenuhan Kebutuhan Fungsional

Tabel V.5 menguraikan mekanisme pemenuhan dan bukti untuk setiap kebutuhan fungsional. Bukti diutamakan dari data nyata pada jaringan uji Sepolia, tempat kontrak yang dideskripsikan pada Bab IV telah disebarkan dan digunakan untuk mengajukan manuskrip sepanjang pengembangan tugas akhir ini. Sisanya bersumber dari pengujian kasus pada Subbab V.1.

Tabel V.5: Pemenuhan kebutuhan fungsional

Kode	Mekanisme Pemenuhan	Bukti
FR-01	*submitManuscript()* mencatatkan naskah dan CID-nya sebagai transaksi *on-chain*.	*nextManuscriptId()* pada kontrak *PublicationRegistry* bertambah pada setiap pengajuan, menandakan naskah tercatat permanen di *ledger* secara berurutan.

Tabel V.5: Pemenuhan kebutuhan fungsional

Kode	Mekanisme Pemenuhan	Bukti
FR-02	*Event PlagiarismCheckRequested* otomatis terpicu setelah pengajuan. Status *PENDING_EDITOR* baru tercapai setelah skor plagiarisme lolos ambang batas dan editor menyetujui melalui *submitEditorReview()*.	Dari naskah nyata yang diproses, ditemukan kombinasi status *CHECKING* (menunggu pemeriksaan), *UNDER_REVIEW*, dan *REJECTED* (akibat penolakan editor maupun plagiarisme), menunjukkan kedua tahap saringan berjalan pada data sesungguhnya.
FR-03	*ReviewOracle* meneruskan permintaan pemilihan penelaah ke *oracle off-chain*, yang meminta *backend* membangkitkan dompet sesi penelaah terverifikasi sesuai bidang kajian.	*Event ReviewerSelectionRequested*, *ReviewerSelectionFulfilled*, dan *ReviewersAssigned* teramati berpasangan pada seluruh naskah berstatus *UNDER_REVIEW* atau lebih lanjut.
FR-04	*submitReview()* mencatat putusan penelaah sebagai transaksi *on-chain* yang tidak dapat diubah.	Naskah uji (*ms_id* 32) memiliki enam ulasan nyata dari enam alamat dompet sesi penelaah berbeda, masing-masing dengan *tx hash* dan CID ulasan tersendiri.
FR-05	*_evaluateDecision()* menetapkan status *ACCEPTED*, *REJECTED*, atau *REVISION_REQUESTED* secara deterministik begitu seluruh penelaah menyampaikan ulasan.	*Event DecisionMade* pada naskah uji menunjukkan keputusan *REVISION_REQUESTED* untuk ronde pertama (tiga putusan *REVISE*) dan *ACCEPTED* untuk ronde kedua (dua *ACCEPT*, satu *REJECT*), sesuai aturan mayoritas pada Subbab III.5.8.
FR-06	*payPublicationFee()* mendistribusikan 10 JRT ke setiap penelaah yang ditugaskan dalam transaksi yang sama dengan penarikan biaya publikasi.	Tiga *event IncentivePaid* untuk naskah uji tercatat pada *tx hash* yang identik dengan *event DOIMinted*, membuktikan distribusi insentif benar-benar atomik.

Tabel V.5: Pemenuhan kebutuhan fungsional

Kode	Mekanisme Pemenuhan	Bukti
FR-07	Berkas diunggah ke IPFS melalui *PinataService*, dan CID-nya dicatatkan pada bidang *cid* naskah.	Naskah uji (*ms_id* 32, *PUBLISHED*) memiliki CID nyata tercatat baik pada bidang *cid* maupun pada *tokenURI DOIToken*-nya, sesuai implementasi pada Subbab IV.3.1.5 dan IV.4.5.
FR-08	Dompet sesi menggantikan identitas asli pada seluruh transaksi *on-chain*. *Endpoint* publik membuka identitas penulis setelah naskah mencapai keputusan akhir, tetapi tidak pernah membuka identitas penelaah pada status mana pun.	Naskah uji (*ms_id* 32, *PUBLISHED*) menampilkan identitas asli penulis, sedangkan identitas penelaahnya tetap berupa alamat dompet sesi meskipun naskah telah *PUBLISHED*, dirinci pada Subbab V.2.1.4.
FR-09	Penelaah mengajukan bidang keahlian melalui *POST /reviewer/specialization*. Editor menyetujuinya sebelum *verifyReviewerFields()* dipanggil *on-chain*.	Diverifikasi melalui pengujian kasus pada Subbab V.1.3. Belum ada data produksi karena seluruh penelaah pada naskah uji sudah berstatus terverifikasi sebelum pengambilan data.
FR-10	*postComment()* mencatat *hash* komentar pembaca sebagai *event* tanpa pembatasan peran.	Diverifikasi melalui pengujian kasus pada Subbab V.1.3. Belum ada komentar pembaca pada data produksi saat pengambilan data.

Secara keseluruhan, seluruh sepuluh kebutuhan fungsional terpenuhi penuh dengan bukti data nyata maupun pengujian kasus.

V.2.1.2 Pemenuhan Kebutuhan Non-Fungsional

Tabel V.6 menguraikan pemenuhan setiap kebutuhan non-fungsional.

Tabel V.6: Pemenuhan kebutuhan non-fungsional

Kode	Mekanisme Pemenuhan	Bukti
NFR-01	Seluruh keputusan dan hasil tahapan alur kerja dicatat sebagai *event on-chain*.	Naskah uji memiliki 28 *event* tercatat dari pengajuan hingga penerbitan, seluruhnya tertaut *tx hash* dan nomor blok yang dapat diverifikasi publik di Sepolia.
NFR-02	Kontrak tidak menyediakan fungsi penghapusan maupun pengubahan data historis. Setiap perubahan status hanya dapat maju melalui alur yang telah ditentukan.	Inspeksi kode kontrak (Subbab IV.3) tidak menemukan fungsi *delete* maupun *override* status naskah selain melalui alur resmi.
NFR-03	Berkas disimpan melalui protokol IPFS yang secara desain bersifat terdistribusi.	Sebagian terpenuhi. Implementasi saat ini hanya menggunakan satu penyedia *pinning* (Pinata), sehingga ketersediaan jangka panjang berkas yang tidak di-*pin* ulang oleh pihak lain masih bergantung pada satu penyedia, dirinci pada Subbab V.2.2.2.
NFR-04	Kontrol akses ditegakkan pada dua lapisan: *modifier onlyRole* pada kontrak, dan pemeriksaan peran pada *handler backend*.	Seluruh skenario pengujian kontrol akses pada Tabel V.3 (kesalahan otorisasi untuk setiap peran) lulus tanpa kecuali.

Tiga dari empat kebutuhan non-fungsional terpenuhi penuh. NFR-03 berstatus sebagian terpenuhi, sebuah keterbatasan yang telah disadari sejak perancangan (Subbab IV.4.5) dan dibahas lebih lanjut pada Subbab V.2.2.2.

V.2.1.3 Pemanfaatan Sifat Dasar *Blockchain*

Subbab ini mengevaluasi apakah keenam rancangan solusi pada Subbab III.3 benar-benar terwujud pada implementasi, menggunakan data naskah uji (*ms_id* 32) sebagai kasus penelusuran karena satu-satunya naskah yang telah melalui seluruh alur kerja secara penuh,

dari pengajuan hingga penerbitan, pada jaringan Sepolia.

1. **Kekekalan riwayat.** Penelusuran naskah uji menghasilkan 28 *event* berurutan, mencakup dua ronde pemeriksaan plagiarisme, dua ronde penyaringan editor, dua ronde pemilihan penelaah, enam ulasan, satu revisi, dan satu penerbitan, seluruhnya tertaut *tx hash* dan nomor blok yang tidak dapat diubah maupun dihapus. Riwayat ini dapat diverifikasi independen oleh siapa pun melalui *block explorer* Sepolia tanpa memerlukan akses ke *backend* tugas akhir ini.

2. **Integritas berkas.** CID naskah uji tercatat konsisten pada bidang *cid* maupun pada *tokenURI DOIToken*-nya, sesuai mekanisme *content-addressing* yang dijelaskan pada Subbab IV.3.1.5, tempat CID berfungsi sebagai sidik jari kriptografis dari isi berkas sehingga perubahan berkas sekecil apa pun akan menghasilkan CID yang berbeda dan dapat terdeteksi.

3. **Kepercayaan pada keputusan.** *Event DecisionMade* pada naskah uji menunjukkan bahwa keputusan *REVISION_REQUESTED* (ronde pertama) dan *ACCEPTED* (ronde kedua) dihasilkan langsung oleh logika *_evaluateDecision()* berdasarkan jumlah suara yang tercatat, bukan oleh keputusan sepihak editor. Tidak ditemukan jalur pada kontrak yang memungkinkan editor maupun administrator mengubah hasil keputusan setelah suara mayoritas tercapai.

4. **Kepercayaan pada insentif.** Ketiga *event IncentivePaid* untuk naskah uji berbagi *tx hash* yang identik dengan *event DOIMinted* dan *DOIRegistered*, membuktikan bahwa penarikan biaya publikasi, distribusi insentif kepada penelaah, dan pencetakan DOI benar-benar dieksekusi sebagai satu transaksi atomik sebagaimana dirancang pada Subbab III.5.7, bukan tiga operasi terpisah yang rentan gagal sebagian.

5. **Keterverifikasian publik.** Seluruh bukti pada subbab ini diperoleh melalui pemanggilan RPC publik Sepolia tanpa kredensial maupun akses istimewa apa pun, menegaskan bahwa jejak proses benar-benar dapat diverifikasi oleh pihak luar sistem.

V.2.1.4 Evaluasi Anonimitas

Subbab ini menelusuri naskah uji (*ms_id* 32, *PUBLISHED*) melalui *endpoint* publik yang sama untuk memverifikasi kebijakan anonimitas asimetris pada FR-08 (Subbab III.3 poin 6).

1. **Identitas penulis terbuka.** Identitas asli penulis naskah uji tampil pada respons *endpoint* publik karena naskah telah mencapai keputusan akhir, sesuai kebijakan bahwa identitas penulis dibuka kembali sebagai bentuk transparansi pascapublikasi.
2. **Identitas penelaah tersembunyi.** Keenam penelaah naskah uji tetap tampil sebagai alamat dompet sesi anonim pada *endpoint* yang sama, meskipun naskah telah *PUBLISHED* dan seluruh ulasannya final. Perbedaan perlakuan ini konsisten dengan implementasi *OpenReview()* pada Subbab IV.4.3, tempat pemetaan identitas penelaah sengaja tidak pernah diselesaikan (*resolved*) pada status manuskrip apa pun, untuk mencegah penelaah mengantisipasi pembukaan identitasnya di masa depan dan membiaskan putusannya.

Dengan bukti ini, FR-08 dinyatakan terpenuhi.

V.2.2 Kelebihan dan Keterbatasan

Berdasarkan hasil pengujian dan evaluasi kualitatif pada Subbab V.2.1, subbab ini merangkum kelebihan utama serta keterbatasan yang perlu diperhitungkan sebelum sistem dipertimbangkan untuk pengembangan maupun penggunaan lebih lanjut.

V.2.2.1 Kelebihan

Kelebihan berikut disarikan dari bukti pemenuhan kebutuhan dan hasil pengujian yang telah dipaparkan pada subbab-subbab sebelumnya.

1. **Kebutuhan terpenuhi penuh.** Seluruh sepuluh kebutuhan fungsional (Tabel V.5) dan tiga dari empat kebutuhan non-fungsional (Tabel V.6) terpenuhi penuh, dengan bukti yang diutamakan dari naskah nyata yang diproses di jaringan uji publik Sepolia.
2. **Otonomi eksekusi on-chain.** Penelusuran naskah uji pada Subbab V.2.1.3 menunjukkan bahwa keputusan penerimaan maupun revisi dihasilkan langsung oleh logika *_evaluateDecision()*, dan bahwa penarikan biaya publikasi, distribusi insentif penelaah, serta pencetakan DOI dieksekusi sebagai satu transaksi atomik, tanpa jalur bagi editor maupun administrator untuk mengubah hasilnya secara sepihak.
3. **Keterbukaan dan anonimitas konsisten.** Sebagaimana ditunjukkan pada Subbab V.2.1.4, *endpoint* yang sama yang membuka akses publik tanpa otentikasi juga menegakkan kebijakan anonimitas asimetris FR-08 secara konsisten, sehingga karakteristik *open* pada Bab I tidak dicapai dengan mengorbankan perlindungan identitas

partisipan.

4. **Menjawab fragmentasi solusi.** Dibandingkan dengan *PubChain*, *Eureka*, dan *SJP* yang dikaji pada Bab II, tugas akhir ini menyatukan otomatisasi keputusan editorial, insentif kontributor, penyimpanan terdesentralisasi, dan mekanisme anonimitas dalam satu alur kerja *end-to-end* yang teruji dengan data nyata, sebagaimana diidentifikasi pada Subbab III.1.1.

V.2.2.2 Keterbatasan

Keterbatasan solusi ini banyak terletak pada batasan implementasi yang telah dijelaskan pada Subbab IV.2.3.

BAB VI

KESIMPULAN DAN SARAN

VI.1 Kesimpulan

Tugas akhir ini bertujuan menghasilkan rancangan *framework* sistem publikasi ilmiah berbasis *blockchain* yang mengintegrasikan alur *submission*, *peer review*, hingga *publishing* dalam satu alur kerja *end-to-end*, beserta implementasi awal sebagai pembuktian kelayakan rancangan, sebagaimana ditetapkan pada Bab I. Berdasarkan analisis dan perancangan pada Bab III, implementasi pada Bab IV, serta pengujian dan evaluasi pada Bab V, dapat disimpulkan hal-hal berikut.

1. **Jawaban rumusan masalah pertama: kebutuhan sistem.** Agar integritas riwayat, keputusan editorial, dan distribusi insentif dapat diverifikasi secara independen, sementara anonimitas pihak yang terlibat tetap terjaga, sistem publikasi ilmiah berbasis *blockchain* perlu memenuhi sepuluh kebutuhan fungsional dan empat kebutuhan non-fungsional (Subbab III.2). Integritas riwayat menuntut pencatatan siklus hidup naskah, ulasan, dan tautan berkas secara *on-chain* (FR-01, FR-04, FR-07, FR-10) yang transparan, kekal, dan tidak bergantung pada satu peladen penyimpanan (NFR-01 s.d. NFR-03). Keputusan editorial menuntut pemeriksaan awal, penunjukan penelaah, verifikasi bidang keahlian, dan pengambilan keputusan yang berjalan berdasarkan aturan otomatis (FR-02, FR-03, FR-05, FR-09) dengan kontrol akses berbasis peran (NFR-04). Distribusi insentif menuntut eksekusi pembayaran penelaah secara otomatis (FR-06), sedangkan anonimitas menuntut penyembunyian identitas penulis dan penelaah selama penelaahan (FR-08). Setiap kebutuhan tersebut dapat ditelusuri ke persoalan asalnya melalui Tabel III.1.

2. **Jawaban rumusan masalah kedua: rancangan sistem.** Kebutuhan tersebut dipenuhi oleh *framework* dengan arsitektur *blockchain* hibrida yang memisahkan data berdasarkan kebutuhannya. Riwayat siklus hidup naskah dicatat oleh *smart contract*, keputusan penerimaan ditentukan oleh suara mayoritas penelaah yang dievaluasi *on-chain*, serta penarikan biaya publikasi, distribusi insentif, dan pencetakan DOI dieksekusi dalam satu transaksi atomik. Berkas naskah disimpan pada

IPFS dengan hanya pengenal kontennya yang dicatat pada *blockchain*, sedangkan identitas penulis dan penelaah dijaga melalui alamat samaran. Seluruh mekanisme ini terintegrasi dalam satu alur kerja *end-to-end* dari *submission* hingga *publishing*, sehingga menjawab kesenjangan fragmentasi solusi pada *PubChain*, *Eureka*, dan *SJP* yang diidentifikasi pada Subbab III.1.1.

3. **Validitas rancangan.** Kelayakan rancangan dibuktikan melalui implementasi awal. Sebanyak 91 kasus uji lulus seluruhnya, terdiri atas 57 pengujian unit dan kontrol akses *smart contract*, 31 pengujian terintegrasi *end-to-end smart contract*, dan 3 pengujian terintegrasi basis data *backend*. Penelusuran naskah uji pada jaringan Sepolia menunjukkan 28 *event* berurutan yang dapat diverifikasi melalui RPC publik tanpa akses istimewa, keputusan yang dihasilkan langsung oleh logika kontrak, serta distribusi insentif yang atomik bersama pencetakan DOI. *Traceability* pada Tabel V.4 menunjukkan tiga belas dari empat belas kebutuhan terpenuhi penuh.

4. **Batas validitas.** Validitas tersebut berlaku dengan dua catatan. Pertama, NFR-03 hanya terpenuhi sebagian karena berkas disimpan melalui satu penyedia *pinning* IPFS, sehingga karakteristik terdistribusi baru tercapai pada lapisan pencatatan *blockchain* dan belum pada ketersediaan berkas. Kedua, pengujian otomatis pada lapisan *off-chain* baru mencakup basis data, sedangkan *endpoint* REST API, *relayer* transaksi, dan *oracle* belum diuji secara otomatis dan kebenarannya hanya didukung oleh penelusuran naskah uji. Pengujian beban tinggi dan audit keamanan juga belum dilakukan.

Dengan demikian, kedua rumusan masalah terjawab dan tujuan tercapai dalam bentuk rancangan beserta implementasi awal yang kelayakannya terbukti pada lingkup pengujian di atas. *Framework* ini masih memiliki keterbatasan, khususnya pada ketersediaan penyimpanan terdesentralisasi, waktu tanggap pembacaan data yang bergantung langsung pada *throughput* penyedia RPC *blockchain*, kesederhanaan algoritma pemilihan penelaah, konsensus, dan insentif, serta sentralisasi kepercayaan pada beberapa komponen kunci, sebagaimana dirinci pada Subbab V.2.2.2 dan dijadikan dasar saran pengembangan berikut.

VI.2 Saran

Berdasarkan keterbatasan (Subbab V.2.2.2), berikut saran pengembangan lanjutan bagi penelitian maupun pengembangan sistem serupa di masa mendatang.

1. **Mengatasi keterbatasan implementasi.** Ketergantungan pada satu penyedia *pinning* IPFS dapat dikurangi dengan meng-*pin* berkas ke lebih dari satu penyedia atau ke simpul IPFS yang dikelola sendiri, sehingga ketersediaan jangka panjang berkas tidak bergantung pada kelangsungan satu penyedia komersial. Algoritma pemilihan penelaah, konsensus, dan insentif yang saat ini masih sederhana, yaitu pengacakan sederhana di sisi *backend*, suara mayoritas tanpa pembobotan, dan *token* bernilai tetap tanpa nilai ekonomi riil, dapat dikembangkan lebih lanjut misalnya melalui keacakan pemilihan penelaah yang dapat diverifikasi *on-chain*, pembobotan suara berdasarkan reputasi atau tingkat keyakinan penelaah, mekanisme anti-kolusi, serta studi kelayakan penerbitan *token* pada jaringan utama (*mainnet*) dengan nilai ekonomi nyata. Sentralisasi kepercayaan pada *backend* sebagai satu-satunya *relayer* transaksi dan pemegang pemetaan identitas, serta pada pemegang *ADMIN_ROLE* yang berwenang memperbarui logika *PublicationRegistry*, dapat dikurangi misalnya melalui *relayer* yang terdesentralisasi atau kewenangan pembaruan kontrak yang disyaratkan melalui persetujuan *multi-signature* maupun tata kelola (*governance*) komunitas, agar tidak ada satu pihak tunggal yang mampu mengubah aturan sistem secara sepihak.

2. **Memperluas cakupan pengujian sebelum sistem dipertimbangkan untuk produksi.** Tugas akhir ini tidak mencakup pengujian beban tinggi maupun audit keamanan *smart contract* secara formal. Kedua jenis pengujian tersebut perlu dilakukan sebelum sistem serupa dipertimbangkan untuk digunakan pada lingkungan produksi dengan nilai transaksi maupun jumlah pengguna yang signifikan.

DAFTAR PUSTAKA

Banks, David (2018). "Thoughts on Publishing the Research Article over the Centuries". Dalam: *Publications* 6.1, hal. 10. DOI: 10.3390/publications6010010.

Benet, Juan (2014). *IPFS - Content Addressed, Versioned, P2P File System (DRAFT 3)*. Draft.

Beştaş, M. dkk. (2023). "A Novel Blockchain-Based Scientific Publishing System". Dalam: *Sustainability* 15.4, hal. 3354. DOI: 10.3390/su15043354.

Carter-Templeton, Heather dkk. (2021). "Digital object identifier: The DOI". Dalam: *Nurse Author & Editor* 31.3-4, hal. 58–61. DOI: 10.1111/nae2.25.

Donovan, Stephen K. (2018). "Reflective Practice: Eight Stages of Publishing a Scientific Research Paper". Dalam: *Publications* 6.1, hal. 8. DOI: 10 . 3390 / publications6010008.

Fayad, Mohamed dan Douglas C. Schmidt (1997). "Object-oriented application frameworks". Dalam: *Communications of the ACM* 40.10, hal. 32–38. DOI: 10.1145/262793.262798.

Gasparyan, Armen Yuri dkk. (2019). "Comprehensive Approach to Open Access Publishing: Platforms and Tools". Dalam: *Journal of Korean Medical Science* 34.27, e184. DOI: 10.3346/jkms.2019.34.e184.

Janowicz, Krzysztof dkk. (2018). "On the prospects of blockchain and distributed ledger technologies for open science and academic publishing". Dalam: *Semantic Web* 9.5, hal. 545–555. DOI: 10.3233/SW-180322.

Nakamoto, Satoshi (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf.

Niya, Sina Rafati dkk. (2019). "A Blockchain-based Scientific Publishing Platform". Dalam: *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, hal. 329–336. DOI: 10.1109/BLOC.2019.8751379.

Rampelotto, Pabulo H. (2023). "A Critical Assessment of the Peer Review Process in Life: From Submission to Final Decision". Dalam: *Life* 13.7, hal. 1603. DOI: 10.3390/life13071603.

Rauchs, Michel dkk. (2018). *Distributed Ledger Technology Systems: A Conceptual Framework*. techreport. Cambridge Centre for Alternative Finance, University of Cambridge Judge Business School. DOI: 10.2139/ssrn.3230013.

Romero Ugarte, José Luis (Okt 2018). *Distributed Ledger Technology (DLT): Introduction*. Banco de España Article 19/18. Available at SSRN. Banco de España. URL: https://ssrn.com/abstract=3269731.

Ross-Hellauer, Tony (2017). "What is open peer review? A systematic review". Dalam: *F1000Research* 6, hal. 588. DOI: 10.12688/f1000research.11369.2.

Solaiman, Ellis, Todd Wike, dan Ioannis Sfyrakis (2021). "Implementation and evaluation of smart contracts using a hybrid on- and off-blockchain architecture". Dalam: *Concurrency and Computation: Practice and Experience* 33, e5811. DOI: 10.1002/cpe.5811.

Suber, Peter (2012). *Open Access*. MIT Press Essential Knowledge Series. Cambridge, Massachusetts: The MIT Press. ISBN: 9780262301732. URL: https://mitpress.mit.edu/9780262517638/open-access/.

Tenorio-Fornés, Antonio dkk. (2019). "Towards a Decentralized Process for Scientific Publication and Peer Review using Blockchain and IPFS". Dalam: *Proceedings of the 52nd Hawaii International Conference on System Sciences (HICSS)*, hal. 4635–4644. ISBN: 978-0-9981331-2-6. URL: https://hdl.handle.net/10125/59901.

vom Brocke, Jan, Alan Hevner, dan Alexander Maedche (2020). "Introduction to Design Science Research". Dalam: *Design Science Research. Cases*. Cham: Springer. DOI: 10.1007/978-3-030-46781-4_1.

Wang, Taotao, Soung Chang Liew, dan Shengli Zhang (2020). "PubChain: A Decentralized Open-Access Publication Platform with Participants Incentivized by Blockchain Technology". Dalam: *2020 International Symposium on Networks, Computers and Communications (ISNCC)*. DOI: 10.1109/ISNCC49221.2020.9297213.

Zheng, Zibin dkk. (2017). "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends". Dalam: *2017 IEEE International Congress on Big Data (BigData Congress)*, hal. 557–564. DOI: 10.1109/BigDataCongress.2017.85.

Zheng, Zibin dkk. (2020). "An overview on smart contracts: Challenges, advances and platforms". Dalam: *Future Generation Computer Systems* 105, hal. 475–491. DOI: 10.1016/j.future.2019.12.019.

Lampiran A. Perbandingan Sistem Publikasi Ilmiah

Tabel-tabel berikut membandingkan penanganan setiap tahap alur kerja publikasi ilmiah (Tabel II.1) oleh sistem konvensional, ketiga *related works* yang dikaji pada Subbab II.6, dan solusi yang diajukan pada tugas akhir ini. Tahap-tahap dikelompokkan tiga per tabel agar tetap dapat dibaca dalam satu halaman. Keterangan "Tidak dijelaskan dalam paper" menandakan bahwa sumber rujukan sistem terkait tidak membahas mekanisme untuk tahap tersebut sama sekali, sedangkan "Tidak didukung berdasarkan informasi yang tersedia" menandakan bahwa arsitektur sistem tersebut secara eksplisit tidak memiliki komponen untuk tahap tersebut.

Tahap	**Sistem Konvensional**	***PubChain***	***Eureka***	***SJP***	**Solusi yang Diajukan**
Submission	Penulis mengajukan naskah beserta seluruh materi dan dokumentasi pendukung melalui sistem *submission* jurnal.	Penulis mengirim transaksi *Tpost* berisi metadata naskah dan membayar sejumlah *PubCoin*, dan naskah langsung tercatat permanen di *blockchain*.	Penulis mengunggah manuskrip (tautan eksternal dan *hash*) melalui *DApp* dan membayar biaya pengajuan *token* EKA yang dikunci pada *smart contract*.	Penulis dengan saldo *token* SJP mencukupi mengunggah naskah beserta bukti kepenulisan dan pembayaran di muka.	Penulis mengunggah berkas manuskrip melalui antarmuka aplikasi, dan pengajuan dicatat pada *smart contract* secara anonim melalui Modul Anonimitas (Subbab III.6.1.1).
Initial Check	Staf jurnal memeriksa kepatuhan terhadap kebijakan editorial, standar etika, format penulisan, serta *screening* plagiarisme.	Tidak dijelaskan dalam paper.	Tidak dijelaskan dalam paper. Hanya disebutkan validasi saldo *token* milik penulis.	Editor memeriksa kesesuaian ruang lingkup, format jurnal, dan laporan plagiarisme dari perkakas eksternal iThenticate.	Pemeriksaan awal dilakukan otomatis melalui *oracle*, dengan plagiarisme sebagai parameter utama dan ambang batas 30% (Subbab III.5.2.1).
Editorial Pre-check	Editor mengevaluasi *merit* ilmiah, metodologi, dan kesesuaian ruang lingkup naskah untuk menentukan kelanjutan ke *peer review* atau penolakan awal.	Tidak didukung berdasarkan informasi yang tersedia, karena arsitektur *PubChain* sengaja dirancang tanpa entitas editor.	Tidak didukung berdasarkan informasi yang tersedia karena tidak ada penyaringan editor sebelum penelaah diberi tahu.	Tidak dipisahkan secara eksplisit dari *Initial Check*. Editor dipilih sistem berdasarkan kesesuaian bidang dan reputasi sitasi, dengan penyaringan konflik kepentingan.	Editor yang ditugaskan otomatis berdasarkan kategori jurnal tujuan naskah (Manajemen Jurnal, Subbab III.5.1.5) meninjau naskah, menetapkan bidang kajian, dan memutuskan kelanjutan ke *peer review* atau penolakan.

Tabel A.1: Perbandingan tahap *Submission*, *Initial Check*, dan *Editorial Pre-check*

Tahap	Sistem Konvensional	*PubChain*	*Eureka*	*SJP*	Solusi yang Diajukan
Peer Review	*Reviewer* eksternal menilai kualitas, kekokohan ilmiah, dan signifikansi naskah, serta memberikan rekomendasi.	Penelaah mengirim skor numerik melalui transaksi *Treview*, sedangkan skor efektif ulasan dihitung dari penilaian pembaca (*reader scoring*) setelah naskah terbit.	Penelaah dipilih otomatis dari daftar terdaftar berdasarkan topik naskah menggunakan teknik *text-mining*. Skor numerik dicatat *on-chain*, sedangkan teks ulasan disimpan *off-chain* pada *convenience layer*.	Editor menyediakan daftar kandidat penelaah berdasarkan kesesuaian bidang dan reputasi sitasi dengan penyaringan konflik kepentingan, lalu *smart contract* menetapkan penelaah dari daftar tersebut, dan keterlambatan ulasan dikenai penalti.	Penelaah dipilih otomatis berdasarkan kecocokan bidang kajian (Modul Pemilihan *Reviewer*, Subbab III.5.6), lalu memberi putusan secara anonim melalui Modul Anonimitas (Subbab III.5.5).
Editorial Post-review	Editor mempertimbangkan masukan *reviewer* dan menetapkan keputusan editorial, diterima, revisi, atau ditolak.	Tidak didukung berdasarkan informasi yang tersedia karena tidak ada keputusan editorial, dan skor hanya menentukan besaran imbalan penulis.	Editor menyatakan ulasan layak dan menyetujui manuskrip secara manual di akhir proses ulasan.	*Smart contract* menghitung skor keputusan (*accept/reject/revision*) dari hasil penelaah, namun algoritme penghitungannya tidak dirinci dalam paper.	*Smart contract* otomatis mengevaluasi suara mayoritas penelaah tanpa keterlibatan editor (Modul Konsensus, Subbab III.5.8).
Revision and Resubmission	Penulis merevisi naskah berdasarkan komentar *reviewer* dan mengajukan ulang disertai respons *point-by-point*.	Tidak dijelaskan dalam paper. Kode *smart contract* menerapkan batasan *"one paper one upload"* yang mengindikasikan tidak ada mekanisme revisi bawaan.	Naskah yang ditolak dapat diajukan ulang menggunakan *id submission* yang sama tanpa biaya tambahan.	Penulis merevisi naskah untuk disetujui, dan naskah yang ditolak dapat dialihkan ke jurnal lain beserta riwayat ulasannya tanpa mengulang evaluasi dari awal.	Penulis mengajukan revisi yang menaikkan versi naskah, menyetel ulang riwayat ulasan, dan mengembalikan naskah ke pemeriksaan awal untuk diperiksa ulang dari awal (Subbab III.6.1.3).

Tabel A.2: Perbandingan tahap *Peer Review*, *Editorial Post-review*, dan *Revision and Resubmission*

Tahap	Sistem Konvensional	*PubChain*	*Eureka*	*SJP*	Solusi yang Diajukan
Final Acceptance	Editor mengeluarkan keputusan penerimaan editorial bersifat *provisional* sambil menunggu pemeriksaan teknis dan format akhir.	Tidak berlaku karena tidak ada tahap "diterima" yang terpisah dari pencatatan *submission*.	Menyatu dengan *Editorial Post-review*, yaitu editor menyetujui manuskrip di akhir siklus ulasan.	Tidak dijelaskan sebagai tahap terpisah dari *Publication* dalam paper.	Menyatu dengan *Editorial Post-review*, dengan penerimaan tercapai begitu suara mayoritas terpenuhi tanpa tahap *provisional* terpisah.
Publication	Artikel dipublikasikan secara resmi dan tersedia bagi pembaca.	Bersamaan dengan momen *Submission*, karena pencatatan transaksi *Tpost* yang berhasil adalah publikasi itu sendiri.	Tidak dijelaskan secara rinci sebagai tahap terpisah dari *Editorial Post-review*. Naskah yang disetujui tercatat pada *smart contract* jurnal dan dapat dibaca bebas oleh publik.	Versi pertama artikel tersedia bagi pembaca sejak sebelum *peer review* selesai, dan versi akhir dipublikasikan setelah revisi disetujui.	Penulis melunasi biaya publikasi, lalu insentif didistribusikan kepada penelaah (Modul Distribusi Insentif, Subbab III.5.7) dan DOI dicetak sebagai *token non-fungible* yang tertaut naskah sehingga naskah terbit (Subbab III.6.1.4).
Post-Publication	Laporan *open peer review* (jika tersedia) dipublikasikan bersama artikel.	Tidak dijelaskan sesuai definisi tersebut. Penilaian pembaca dan imbalan sitasi berjalan berkelanjutan pascaterbit.	Tidak dijelaskan sesuai definisi tersebut. Penulis yang disitasi menerima imbalan *token* secara otomatis, dan penulis lain yang mereplikasi atau membantah studi terbit turut memperoleh imbalan *token*.	Tidak dijelaskan sesuai definisi tersebut. Mekanisme yang tersedia berupa imbalan sitasi berkala dan kontribusi perbaikan berbayar.	Laporan ulasan dipublikasikan secara terbuka dengan identitas penelaah tetap dirahasiakan secara permanen (Modul Anonimitas, Subbab III.5.5), dan pembaca dapat memberikan komentar publik pascaterbit melalui fitur komentar pada *DOIToken* (Subbab III.5.2.4).

Tabel A.3: Perbandingan tahap *Final Acceptance*, *Publication*, dan *Post-Publication*

Lampiran B. Repositori Implementasi

Kode sumber implementasi sistem yang dijelaskan pada Bab IV tersedia pada repositori publik GitHub berikut.

`https://github.com/mroihn/Publication_System_DLT`